

Sequence Containers

CSCI 33500: Software Design and Analysis III

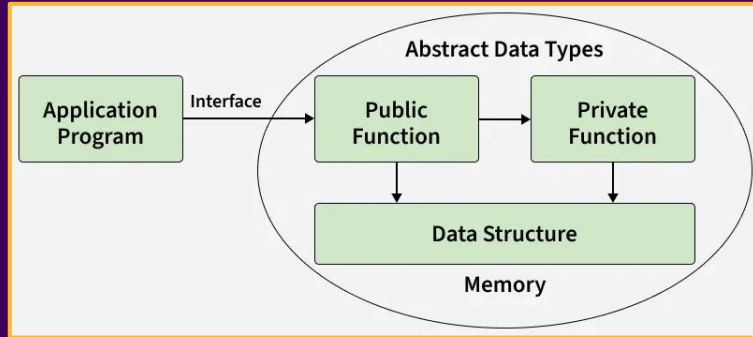
Sadab Hafiz

sh3646@hunter.cuny.edu

04

Abstract Data Type (ADT)

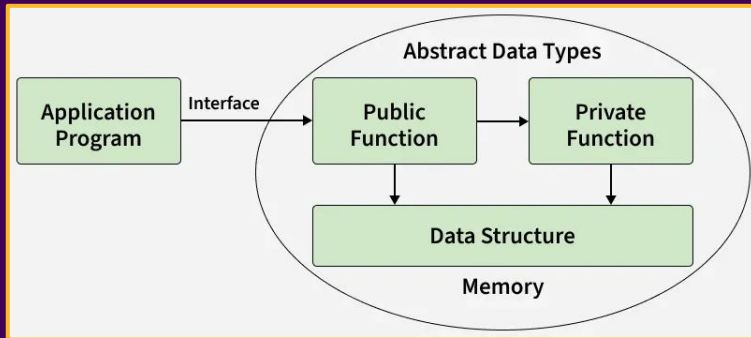
Characteristics



- Any form of information processing/ or communication requires data to be stored in and accessed from either main or secondary memory
- Questions to ask about data storage
 - ◆ What do we want to do with the data?
 - ◆ How do we do it?
- An Abstract Data Type (ADT) is a model of the storage and access of information from memory
- Data structures are concrete methods for organizing and accessing data in the computer

ADT vs Data Structure

Theory vs Implementation



→ Abstract Data Types:

An **abstract data type (ADT)** is a mathematical model for data types, defined by its behavior (semantics) from the point of view of a *user* of the data, specifically in terms of possible values, possible operations on data of this type, and the behavior of these operations.

- https://en.wikipedia.org/wiki/Abstract_data_type

→ Data Structure:

A **data structure** is a way to organize and store data that is usually chosen for efficient access to data. More precisely, a data structure is the physical implementation of a data type, including specifications of the data organization and storage format, as well functions or operations for working with this data.

- https://en.wikipedia.org/wiki/Data_structure

→ Example:

- ◆ **ADT:** Stack (LIFO behavior)
- ◆ **Data Structure:** Linked List

Container

Most Basic ADT

- The most general Abstract Data Type (ADT) can be called the **Container** ADT
- A container describes structures that store and give access to objects
- The queries and operations of interest may be defined on:
 - ◆ The container itself
 - E.g., `isEmpty()`
 - ◆ The objects stored within the container
 - E.g., `contains(const Item& x)`

Containers

Characteristics

- Creating a new container
 - ◆ Constructors
- Big five:
 - ◆ Copy an existing container
 - Copy constructor
 - Copy assignment operator
 - ◆ Move an existing container (since C++11)
 - Move constructor
 - Move assignment operator
 - ◆ Destructor (deallocate dynamic memory)
- Operations:
 - ◆ Adding a new element
 - ◆ Finding an existing element
 - ◆ Removing an existing element
 - ◆ Empty the container
 - ◆ Count number of element
 - ◆ Retrieving/Accessing the elements
 - ◆ And many more . . .

Things to Consider

Important ADT Decisions

- Duplicate or Unique Elements
 - ◆ Some ADT may require all objects in the container to be unique
 - ◆ Other ADT may allow duplicate objects
 - ◆ Some containers we will look at will assume uniqueness.
 - E.g., Binary Search Trees
 - ◆ Dealing with duplicate objects is often just additional, and subtle code
- In any application, the actual required operations may be only a subset of the possible operations
 - ◆ While designing a solution, it is important to consider both current and future requirements and plan accordingly
 - ◆ In some cases, reducing specific operations can increase memory and speed efficiency

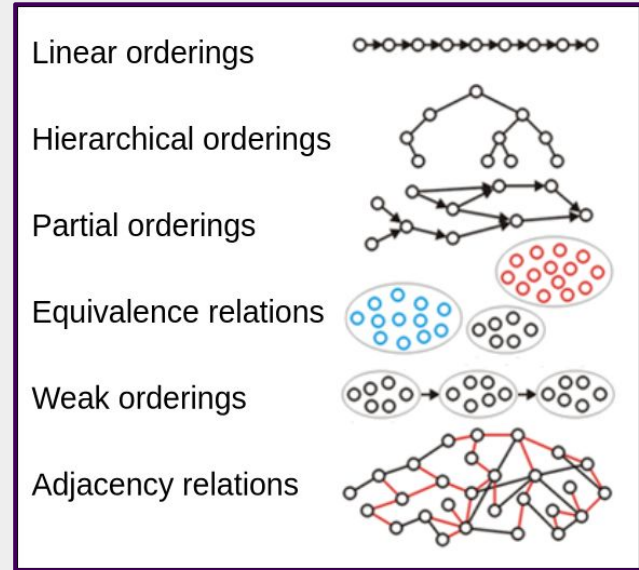
Relationships

Modeling Data

- We may want to store data in a way which captures relationship between the data points
 - ◆ This may add more operations
- Most interesting problems have questions associated with data relationships. For example:
 - ◆ What item has been stored the longest?
 - ◆ What is the distance between two stored location data points?
 - ◆ Does the path of the two NPCs overlap?
 - ◆ Did student X pass CS135 and CS235 before taking CS335?

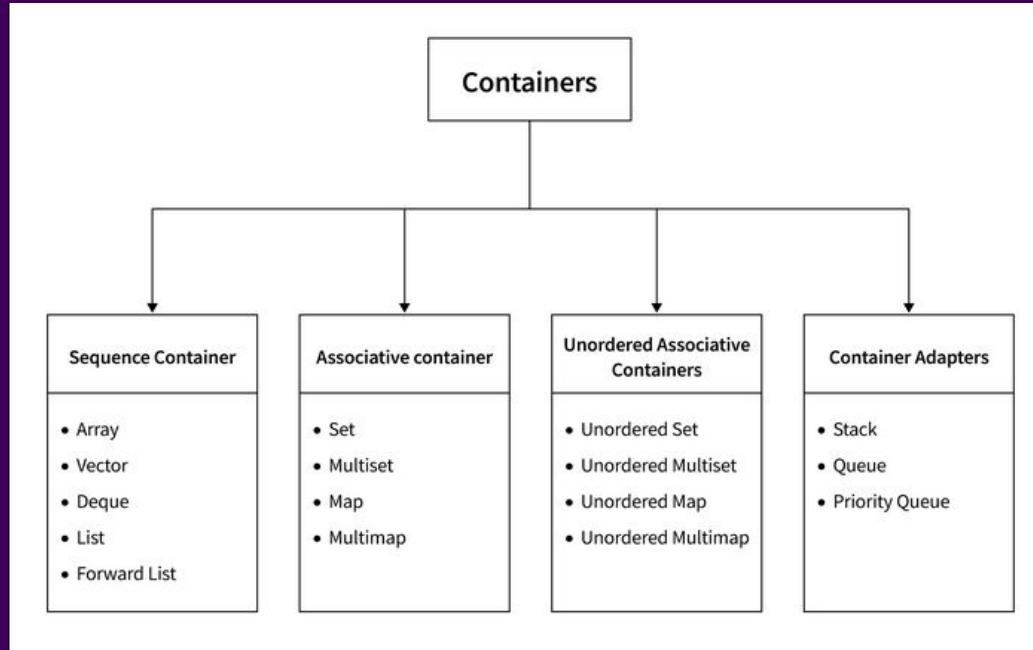
Relationships

Modeling Data



- Relationships are often boolean valued binary operations. For example, given two integers:
- ◆ Are they equal?
 - ◆ Is one less than the other?
 - ◆ Does the first divide the second?

Containers in C++ Standard Template Library (STL)



Note that container adapters are not really a category of containers. Find out more about the STL here:

<https://www.geeksforgeeks.org/cpp/the-c-standard-template-library-stl/>

Container Types

C++ STL Containers

- Sequence Containers:
 - ◆ Containers that store individual elements in linear data structures allowing the elements to be accessed sequentially
- Container Adapters
 - ◆ Provides an interface for other containers by adapting the behavior of the existing containers to fit specific use case
 - ◆ For example: stacks using vectors
- Associative containers
 - ◆ Containers that store keys but also store records associated with those keys
 - ◆ STL provides both ordered and unordered associative containers with different implementations
 - ◆ We will return to this later!
- Note that C++ STL now has even more containers. Find out [here](#).

Container Types

C++ STL Containers

- Sequence Containers:
 - ◆ Containers that store individual elements in linear data structures allowing the elements to be accessed sequentially
- Container Adapters
 - ◆ Provides an interface for other containers by adapting the behavior of the existing containers to fit specific use case
 - ◆ For example: stacks using vectors
- Associative containers
 - ◆ Containers that store keys but also store records associated with those keys
 - ◆ STL provides both ordered and unordered associative containers with different implementations
 - ◆ We will return to this later!
- Note that C++ STL now has even more containers. Find out [here](#).

Array

Sequence Containers

memory locations						
1004	1005	1006	1007	1008	1009	1010
...	2	1	5	3	4	...
	0	1	2	3	4	
index						

- Container that wraps over fixed sized static C++ arrays
- Better alternative to C-style arrays
 - ◆ The **Array** class knows its own size. No need to pass the size separately to a function like static C++ arrays.
 - ◆ No array decay in **Array** class
 - The array may lose its type and dimension when used like a pointer
 - Learn more: [Array Decay](#)
 - ◆ More efficient, light-weight and reliable
- Properties:
 - ◆ Individual elements are accessed by their position in the sequence
 - ◆ Contiguous storage allowing constant time random access (pointer offset)
 - ◆ Fixed size during compile time
 - Allocates only the required space
- [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/array-class-c/>

Vector

Sequence Containers

memory locations						
1004	1005	1006	1007	1008	1009	1010
...	2	1	5	3	4	...
	0	1	2	3	4	
index						

- Dynamic arrays where the size automatically grows when elements are added beyond current capacity
- Things to know about **vector**
 - ◆ No need to worry about limited size
 - ◆ Some insertions can be costly
 - ◆ Requires extra memory
- Properties:
 - ◆ Individual elements are accessed by their position in the sequence
 - ◆ Contiguous storage allowing constant time random access (pointer offset)
 - ◆ Allocates more space as the container grows in size (doubles capacity)
- [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/vector-in-cpp-stl/>

Forward List

Sequence Containers



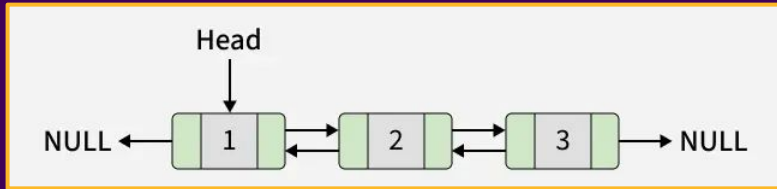
- Implements a singly-linked list
- Things to know about `forward_list`:
 - ◆ Index access is not supported
 - Must be accessed sequentially
 - ◆ Methods to efficiently insert or remove
 - Need to know the location first
 - `insert_after()`
 - `erase_after()`
 - ◆ Can only iterate forward
- Properties:
 - ◆ Individual elements are accessed by their position in the sequence
 - ◆ Not contiguous memory, so no direct random access to each element
 - ◆ Allocates more space as the container grows in size (one element each time)
- [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/forward-list-c-set-1-introduction-important-functions/>

List

Sequence Containers



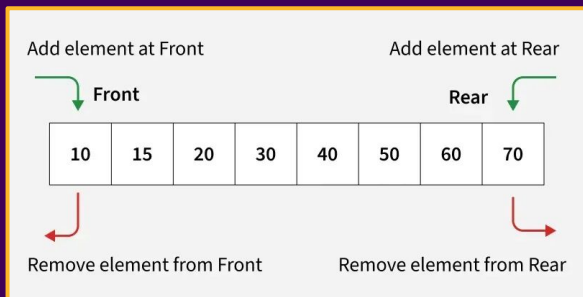
- Implements a doubly-linked list
- Things to know about `list`:
 - ◆ Index access is not supported
 - Must be accessed sequentially
 - ◆ Methods to efficiently insert or remove
 - Need to know the location first
 - ◆ Can be iterated both forward and reverse
 - ◆ Requires more storage per element compared to `forward_list`
- Properties:
 - ◆ Individual elements are accessed by their position in the sequence
 - ◆ Not contiguous memory, so no direct random access to each element
 - ◆ Allocates more space as the container grows in size (one element each time)
- [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/list-cpp-stl/>

Deque

Sequence Containers



- Double-Ended Queue, or a **deque**, allows insertion and removal of elements from both front and back
- Things to know about **deque**
 - ◆ Efficient operations at front and back
 - ◆ Used to adapt **Stack** and **Queue**
 - ◆ Implementation combines properties of **vector** and **list**
 - Allocates elements across multiple fixed size independent chunks which are managed by a map or directory of pointers
- Properties:
 - ◆ Individual elements are accessed by their position in this sequence
 - ◆ Somewhat contiguous storage allowing constant time random access (slightly slower than **vector**)
 - ◆ Allocates more space as the container grows in size (double the map when full)
- [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/deque-cpp-stl/>

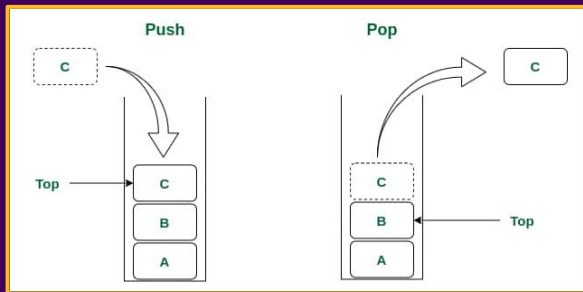
Container Types

C++ STL Containers

- Sequence Containers:
 - ◆ Containers that store individual elements in linear data structures allowing the elements to be accessed sequentially
- Container Adapters
 - ◆ Provides an interface for other containers by adapting the behavior of the existing containers to fit specific use case
 - ◆ For example: stacks using vectors
- Associative containers
 - ◆ Containers that store keys but also store records associated with those keys
 - ◆ STL provides both ordered and unordered associative containers with different implementations
 - ◆ We will return to this later!
- Note that C++ STL now has even more containers. Find out [here](#).

Stack

Container Adapters



- Stack follows Last In First Out (LIFO) order of insertion and deletion
 - ◆ Inserting and deleting in one end
 - ◆ Efficient **push** and **pop**
- Things to know about **stack**
 - ◆ Container adapter means you can change the underlying data structure
 - ◆ Container may be one of the standard containers, or any specifically designed container class with the following:
 - **empty()**
 - **size()**
 - **back()**
 - **push_back()**
 - **pop_back()**
 - ◆ Standard container classes **vector**, **deque**, and **list** can be used
 - ◆ Default container is **deque**
- [Find out more in the documentation](#)

Stack Review

The following slides are provided by Professor Tiziana Ligorio

Stack ADT

```
#ifndef STACK_H_
#define STACK_H_

template<class T>
class Stack
{
public:
    Stack();
    void push(const T& new_entry); // adds an element to top of stack
    void pop(); // removes element from top of stack
    T top() const; // returns a copy of element at top of stack
    int size() const; // returns the number of elements in the stack
    bool isEmpty() const; // returns true if no elements on stack false otherwise

private:
    //implementation details here

}; //end Stack

#include "Stack.cpp"
#endif // STACK_H_`
```

Choose a Data Structure

Array?

Vector?

Linked chain?

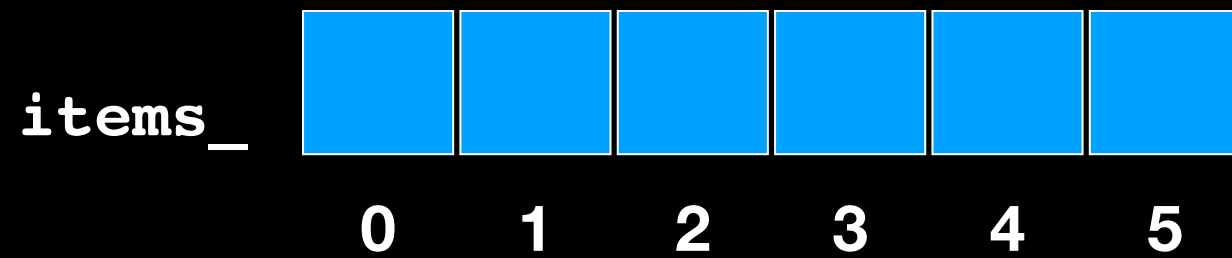
Choose a Data Structure

Inserting and removing from same end (**LIFO**)

Goal: minimize work - Ideally $O(1)$

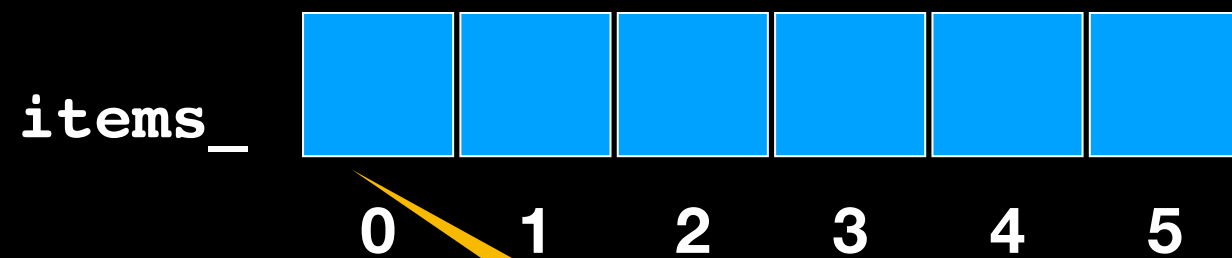
What would you suggest?

Array



Where is the top
of the stack?

Array



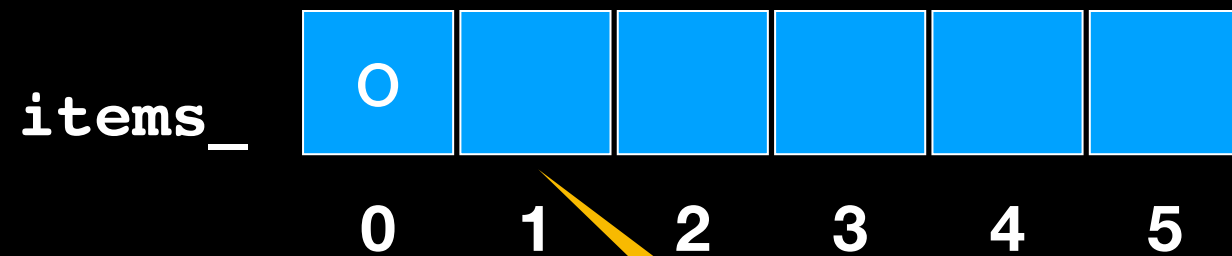
```
item_count_ = 0
```

```
max_items_ = 6
```

Top of the stack:
`items_[item_count_]`

Array

```
push( '0' )
```



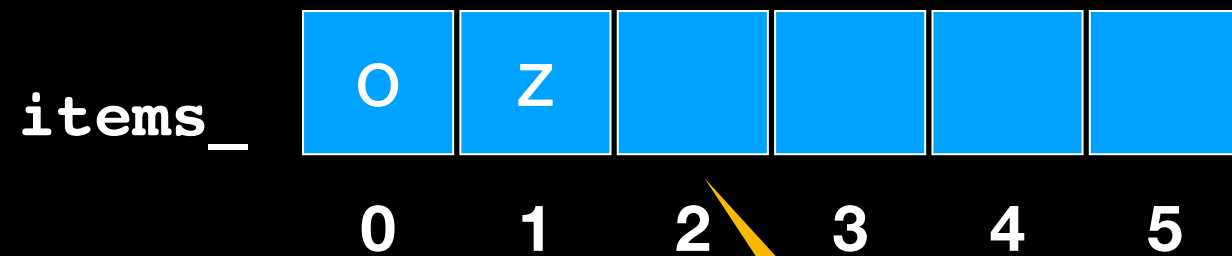
```
item_count_ = 1
```

```
max_items_ = 6
```

Top of the stack:
`items_[item_count_]`

Array

`push( 'Z' )`



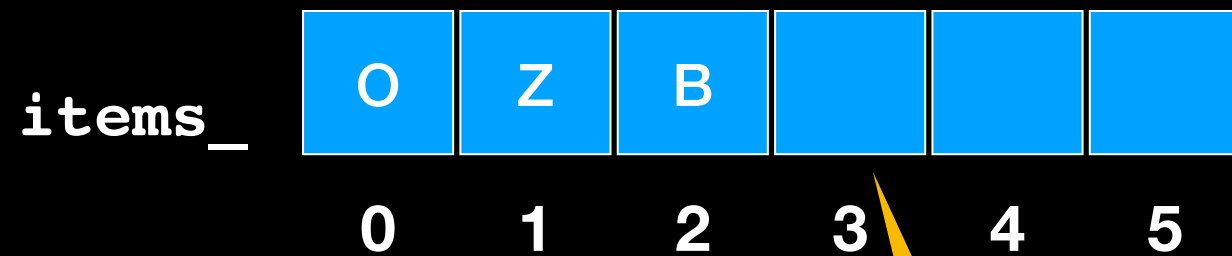
`item_count_ = 2`

`max_items_ = 6`

Top of the stack:
`items_[item_count_]`

Array

`push( 'B' )`

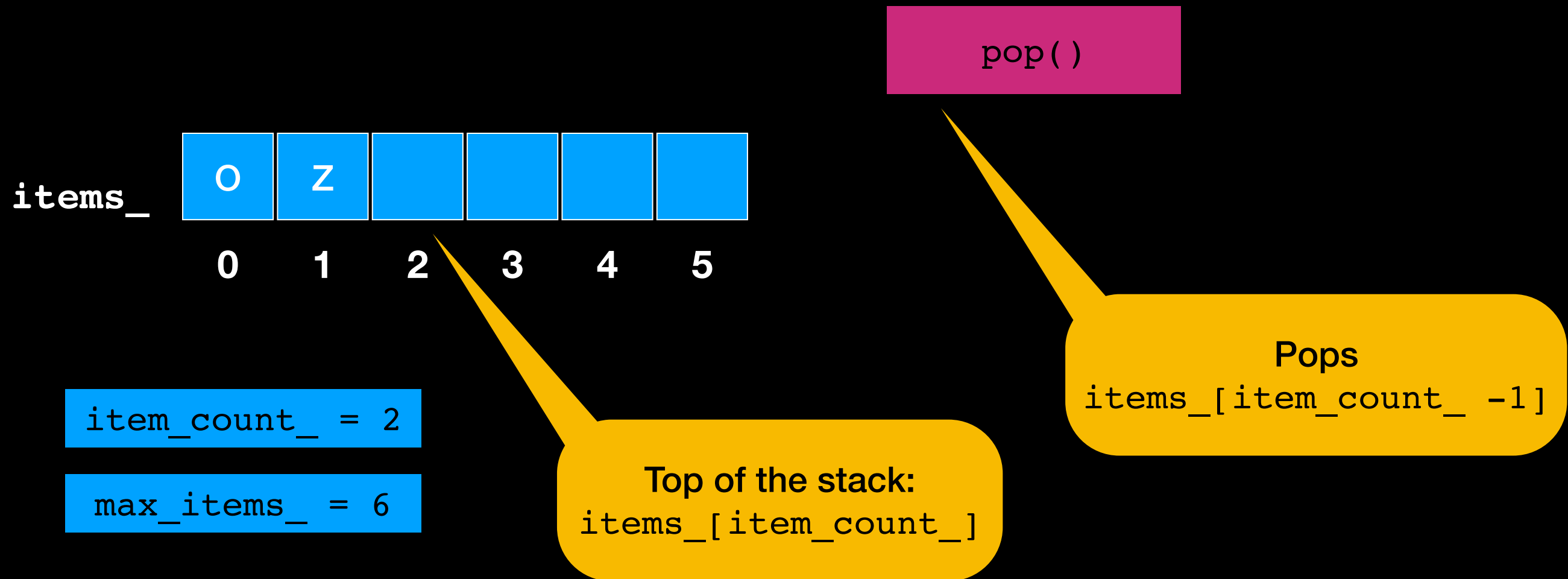


`item_count_ = 3`

`max_items_ = 6`

Top of the stack:
`items_[item_count_]`

Array



Array Analysis

1 assignment + 1 increment/decrement = $O(1)$

size : $O(1)$

isEmpty: $O(1)$

push: $O(1)$

pop : $O(1)$

top : $O(1)$

GREAT!!!!

Array Analysis

1 assignment + 1 increment/decrement = $O(1)$

size : $O(1)$

isEmpty: $O(1)$

push: $O(1)$

pop : $O(1)$

top : $O(1)$

GREAT???

Array

```
push( 'T' )
```

items_	O	Z	B	Y	L	P
	0	1	2	3	4	5



Sorry Stack is Full!!!

```
item_count_ = 6
```

```
max_items_ = 6
```

Top of the stack:
`items_[item_count_]`

Vector

```
std::vector<T> some_vector;
```

So what is a vector really?

Vector

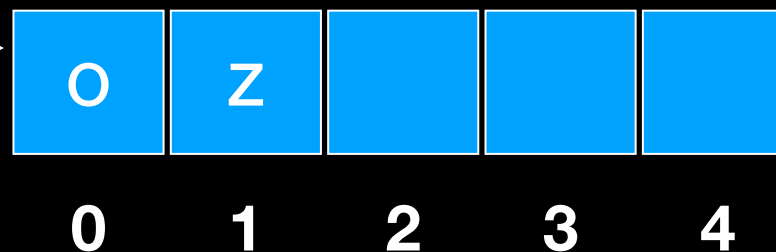
```
std::vector<T> some_vector;
```

So what is a vector really?

Push and pop same as
with arrays

Vector (simplified)

buffer_ =
len_ = 2
capacity_ = 5



Vector

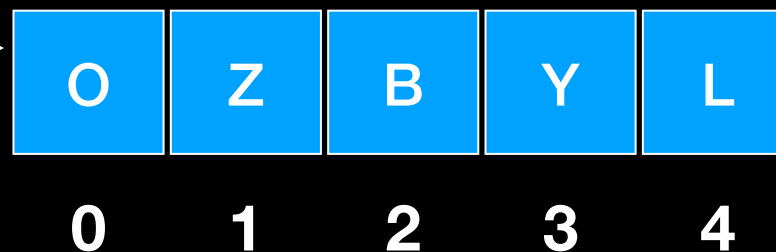
```
std::vector<T> some_vector;
```

So what is a vector really?

Stack is Full?

Vector (simplified)

buffer_ =
len_ = 5
capacity_ = 5



Vector

```
std::vector<T> some_vector;
```

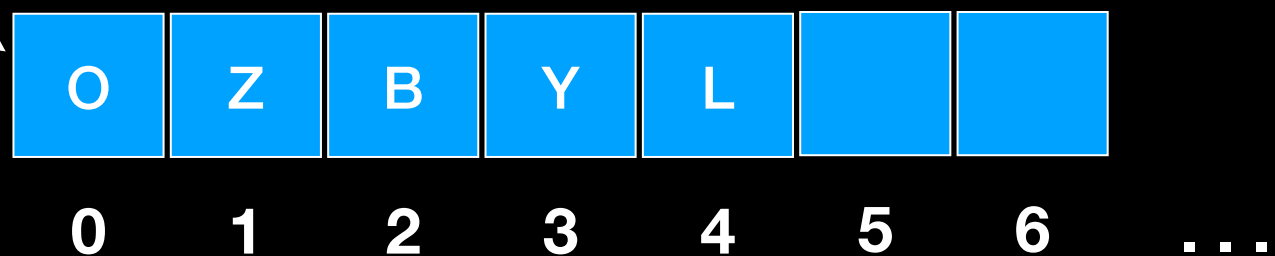
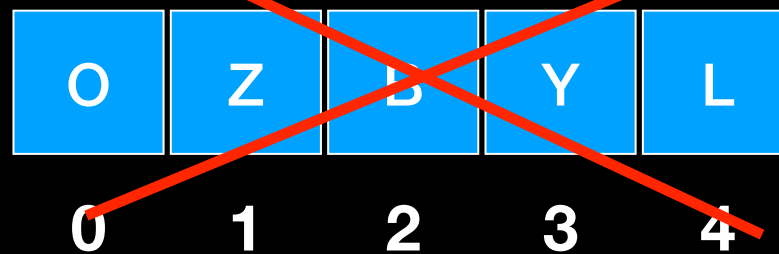
So what is a vector really?



No, I'll Grow!!!

Vector (simplified)

buffer_ =
len_ = 5
capacity_ = ?



Vector Analysis

1 assignment + 1 increment/decrement = $O(1)$

size : $O(1)$

isEmpty: $O(1)$

push: $O(1)$

pop : $O(1)$

top : $O(1)$

GREAT!!!!

Vector Analysis

1 assignment + 1 increment/decrement = $O(1)$

size : $O(1)$

isEmpty: $O(1)$

push: $O(1)$

pop : $O(1)$

top : $O(1)$

GREAT???

Except when stack is full must:

- allocate new array
- copy elements in new array
- delete old array

Vector Analysis

1 assignment + 1 increment/decrement = $O(1)$

size : $O(1)$

isEmpty: $O(1)$

push: $O(1)$

pop : $O(1)$

top : $O(1)$

Except when stack is full must:

- allocate new array $O(1)$
- copy elements in new array $O(n)$
- delete old array $O(1)$

GREAT??? 

How should Vector grow?

Sometimes $O(1)$

Sometimes $O(n)$

Consider behavior over several pushes

Vector Growth: a naive approach

```
std::vector<T> some_vector;
```

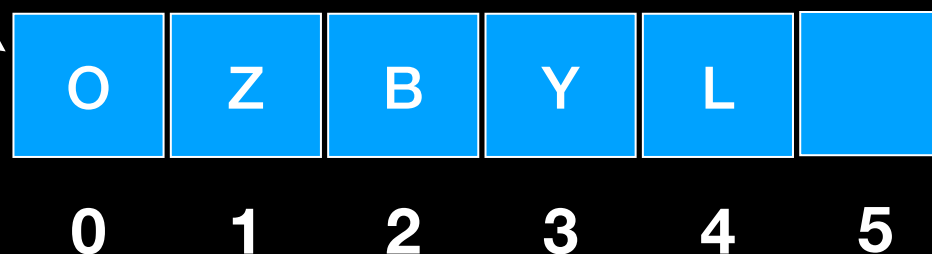
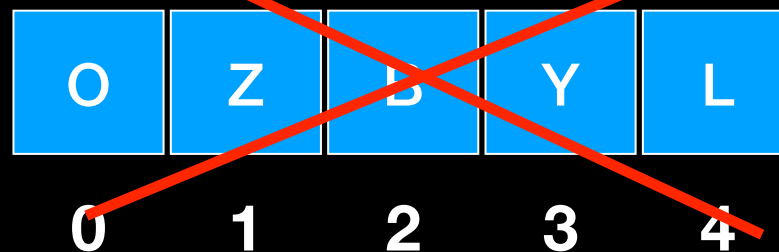
So what is a vector really?



I'll Grow!!!
I will add space for the
item to be added

Vector (simplified)

buffer_ =
len_ = 5
capacity_ = 6



Vector Growth: a naive approach

If vector **grows by 1 each time**, every push costs n “steps”

Cost of pushes:

$$1 + 2 + 3 + 4 + 5 + \dots + n$$
$$= n(n+1)/2$$

Vector Growth: a naive approach

If vector **grows by 1 each time**, every push costs n “steps”

Cost of n pushes:

$$1 + 2 + 3 + 4 + 5 + \dots + n$$

$$= n(n+1)/2$$

$$= n^2 / 2 + n / 2 \quad O(n^2)$$

Vector Growth: a better approach

```
std::vector<T> some_vector;
```

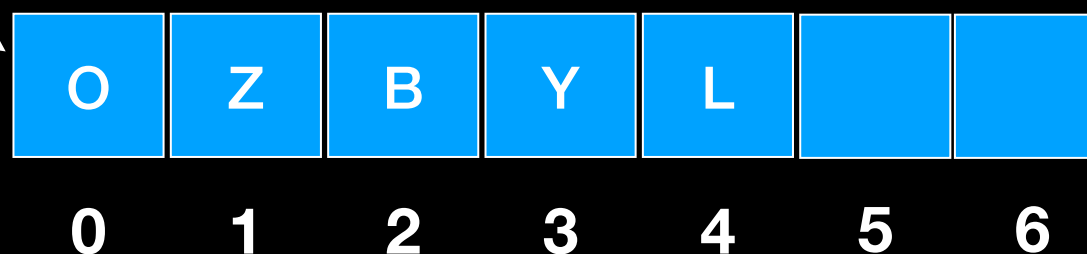
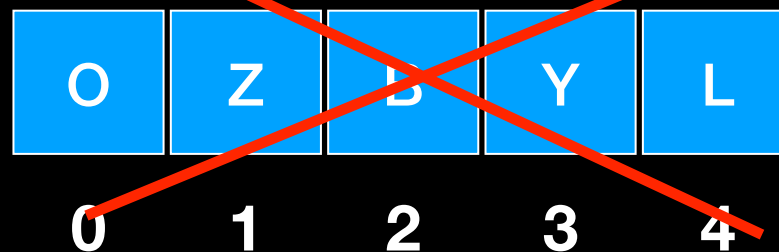
So what is a vector really?



I'll Grow!!!
I will add two more slots!

Vector (simplified)

buffer_ =
len_ = 5
capacity_ = 7



Vector Growth: a better approach

If vector **grows by 2 each time**,

Let a “**hard push**” be one where the whole vector needs to be copied

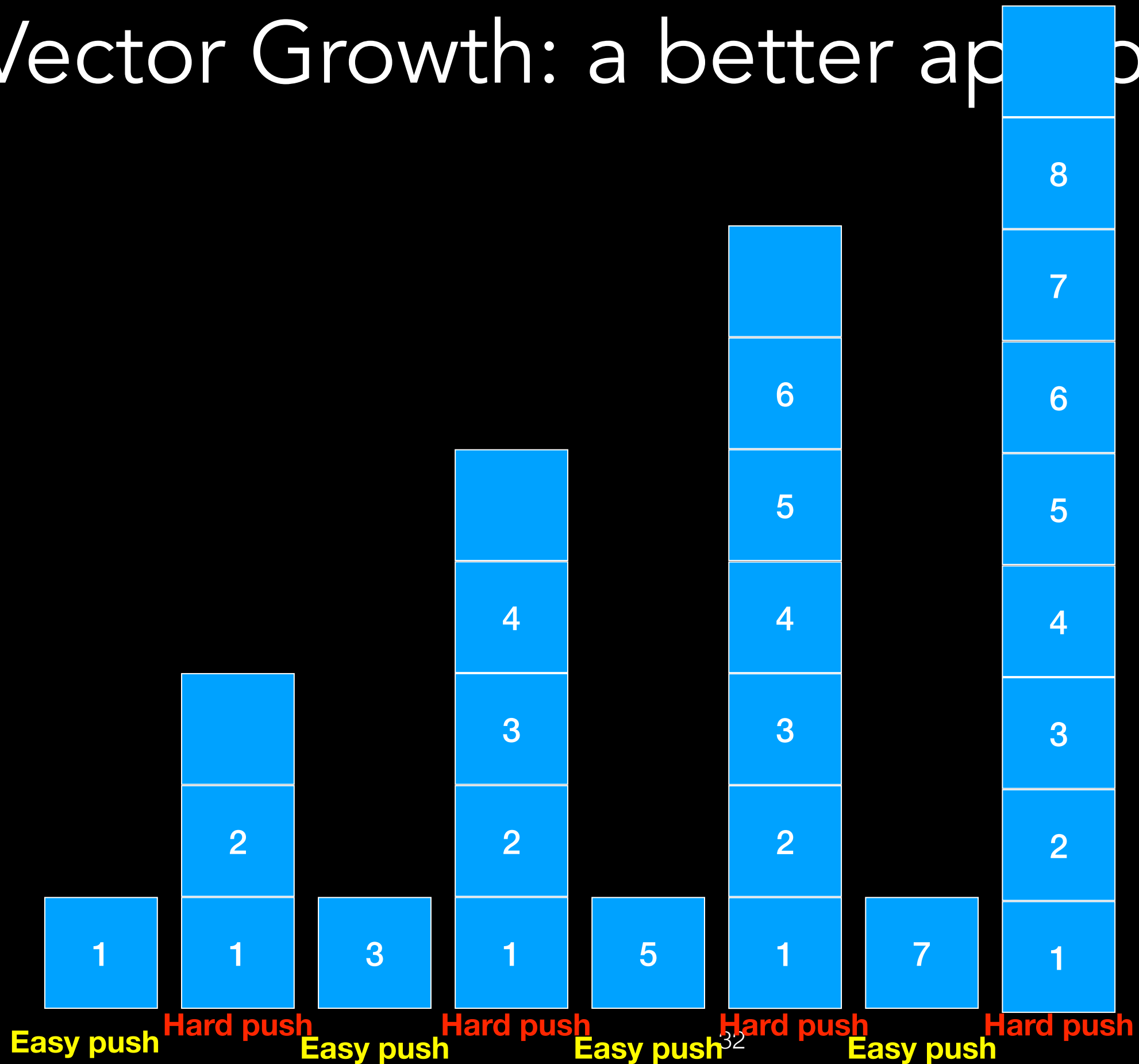
When vector is not copied we have an “**easy push**”

Now **half** our pushes will be **easy (1 step)** and **half** will be **hard (n steps)**

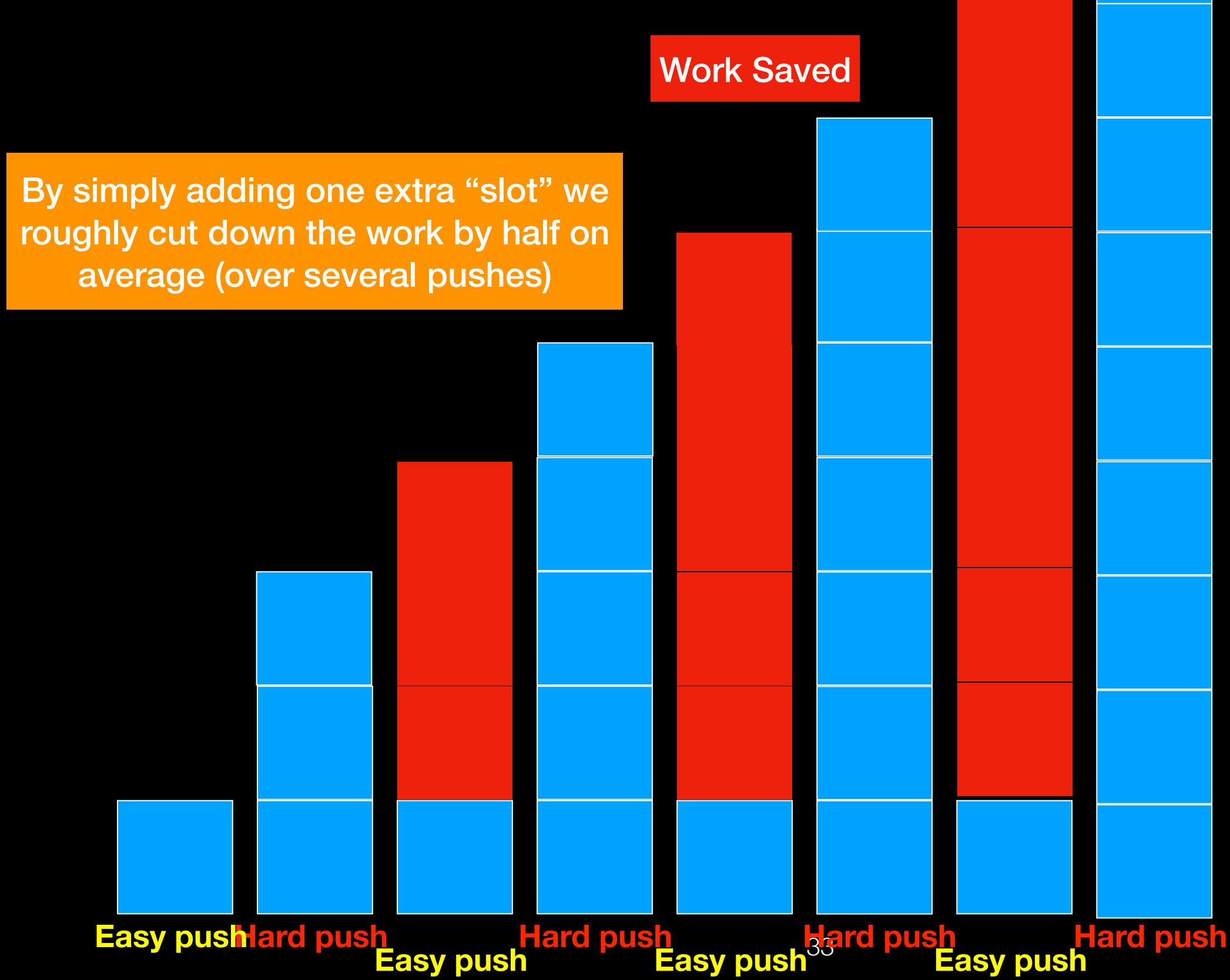
So if reconsider the work **over several pushes**?

Analysis visualization adapted from Keith Schwarz

Vector Growth: a better approach

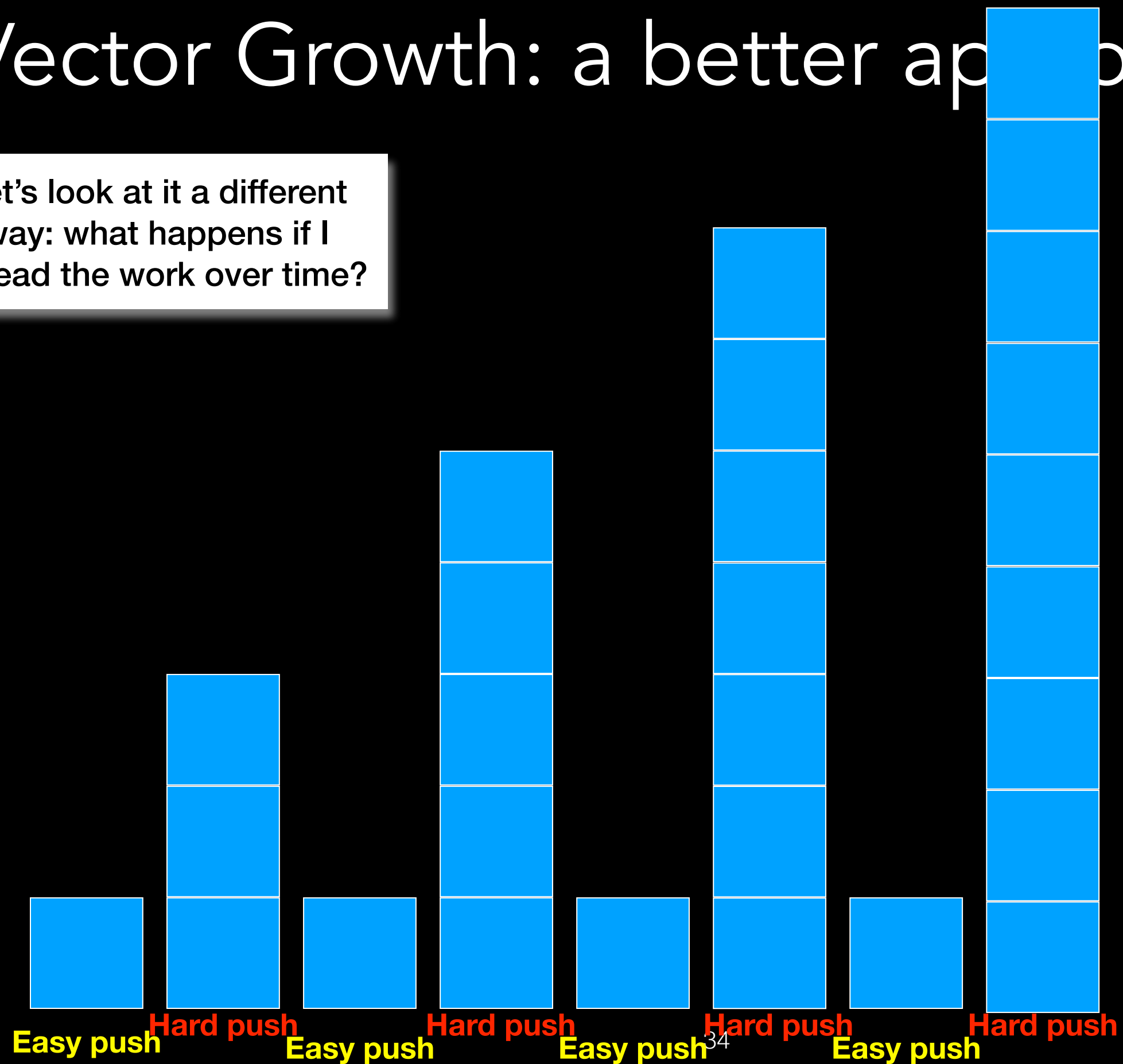


Vector Growth: a better approach



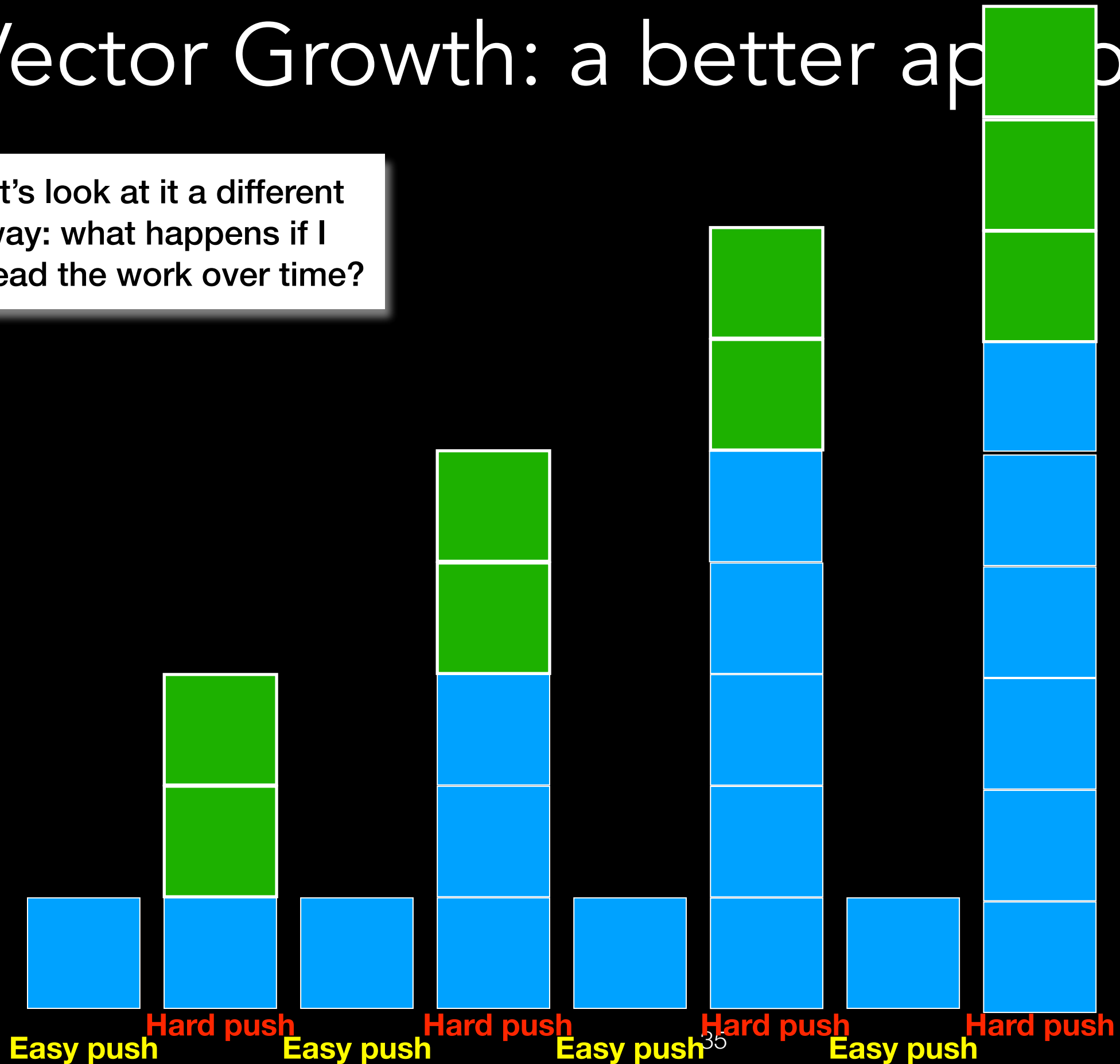
Vector Growth: a better approach

Let's look at it a different way: what happens if I spread the work over time?



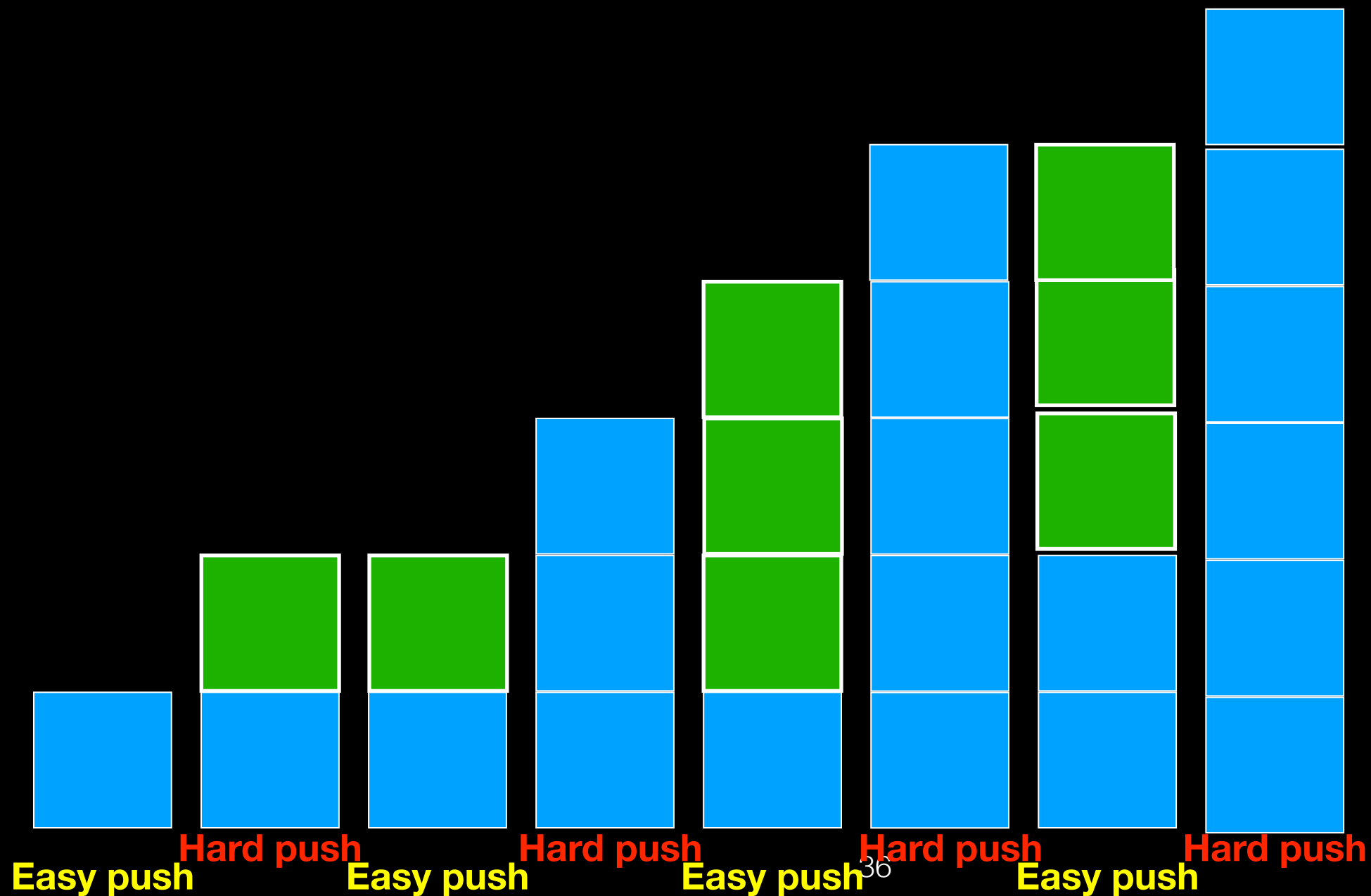
Vector Growth: a better approach

Let's look at it a different way: what happens if I spread the work over time?



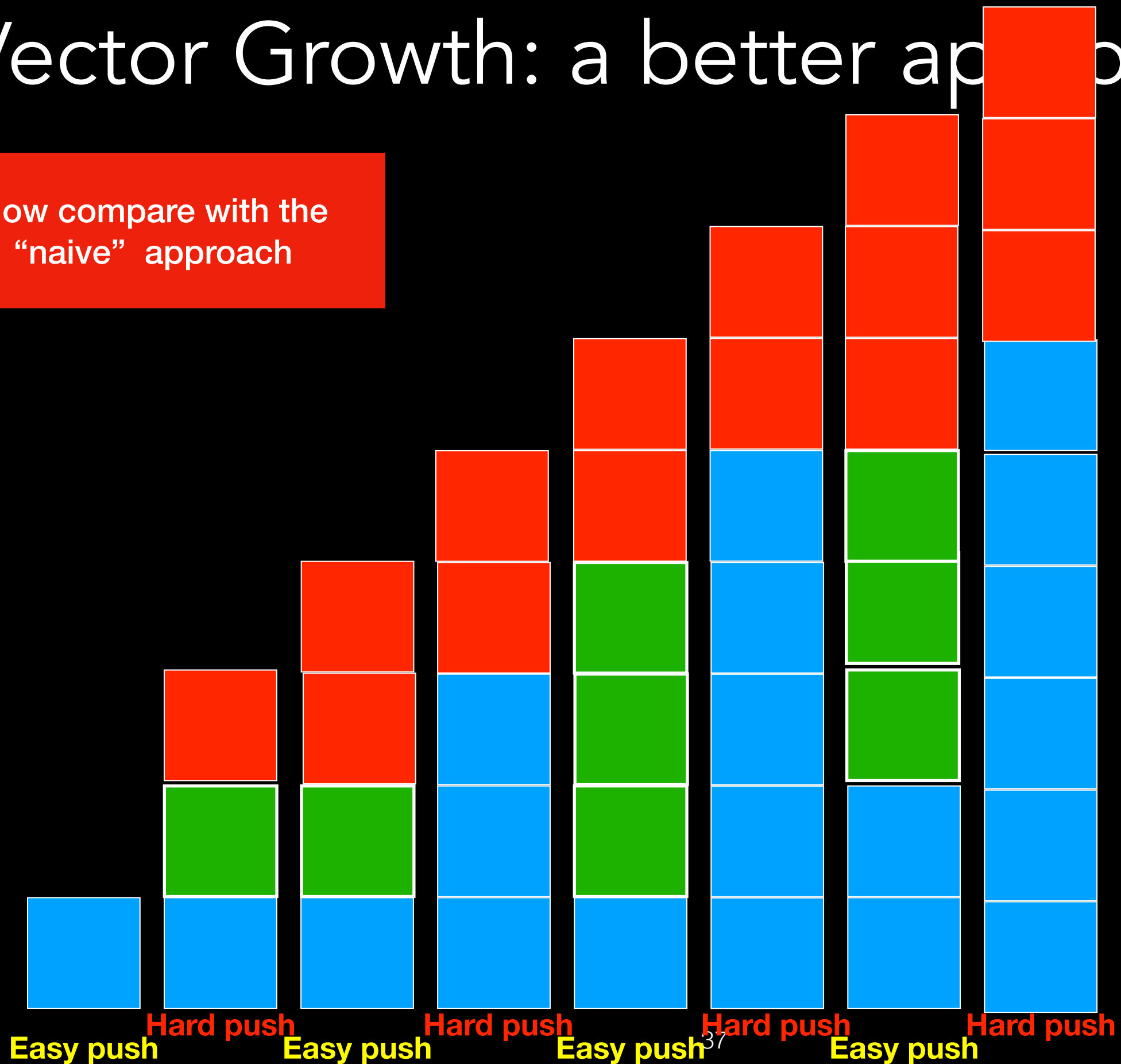
Vector Growth: a better approach

Let's look at it a different way: what happens if I spread the work over time?



Vector Growth: a better approach

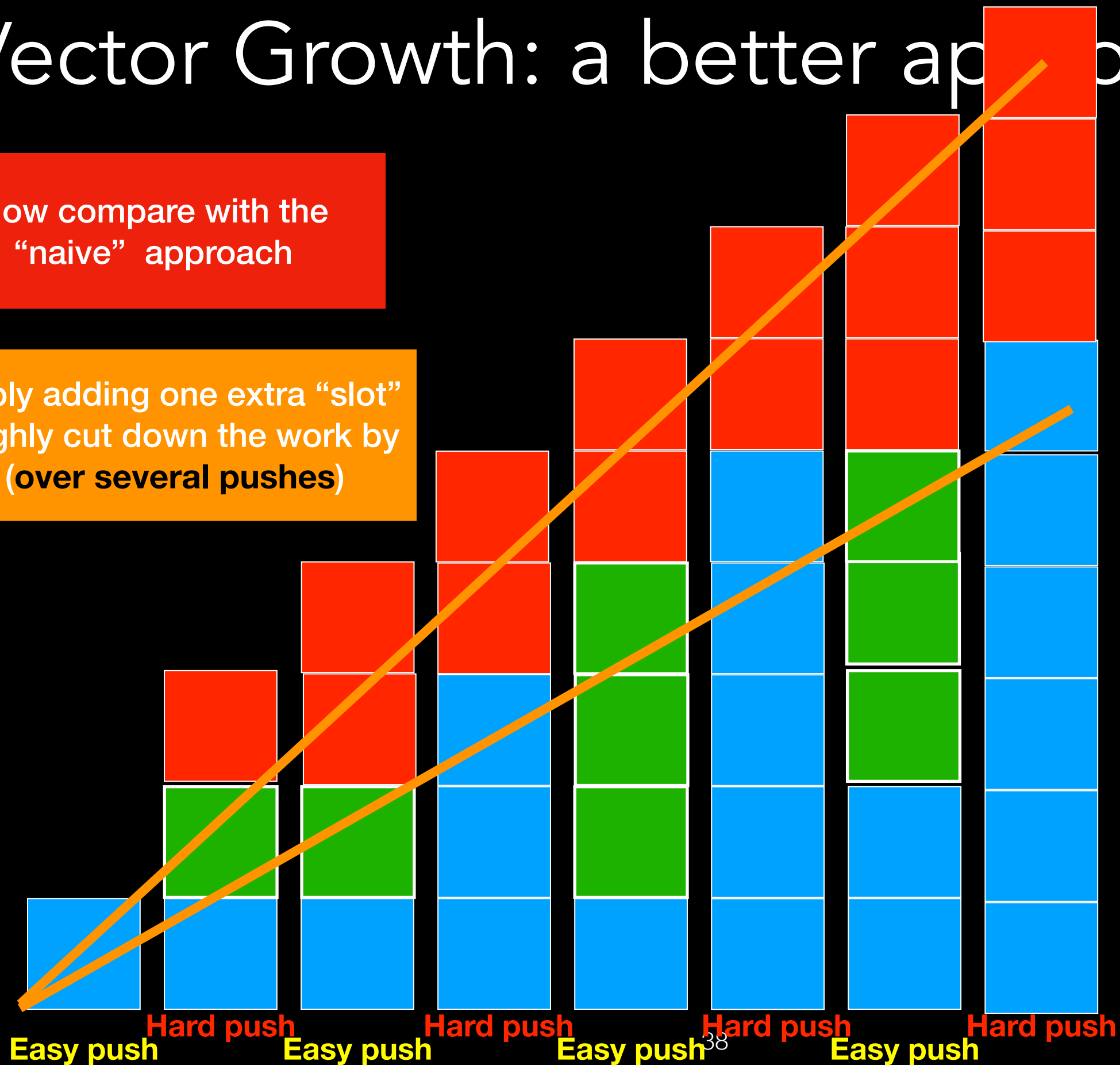
Now compare with the
“naive” approach



Vector Growth: a better approach

Now compare with the
“naive” approach

By simply adding one extra “slot”
we roughly cut down the work by
half (**over several pushes**)



Can we do better?

Vector Growth: a much better approach

```
std::vector<T> some_vector;
```

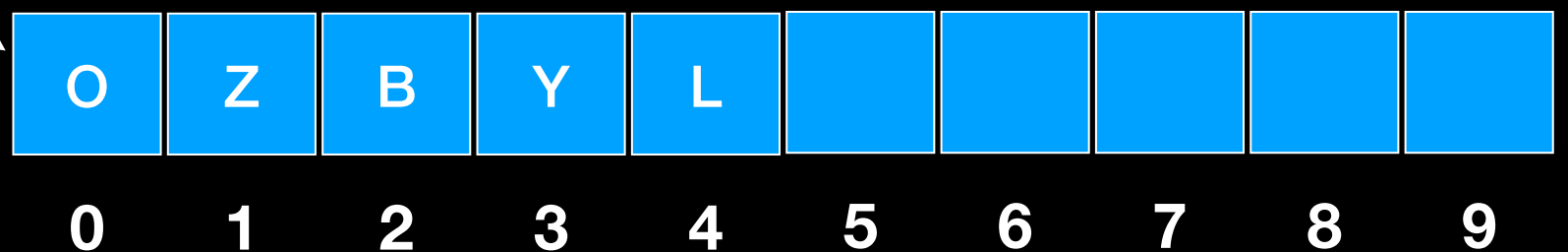
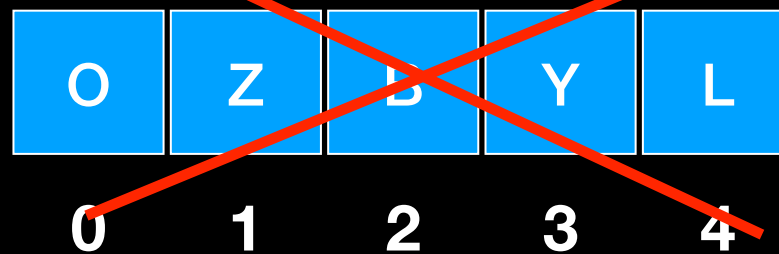
So what is a vector really?



I'll Grow!!!
I'll double my size!

Vector (simplified)

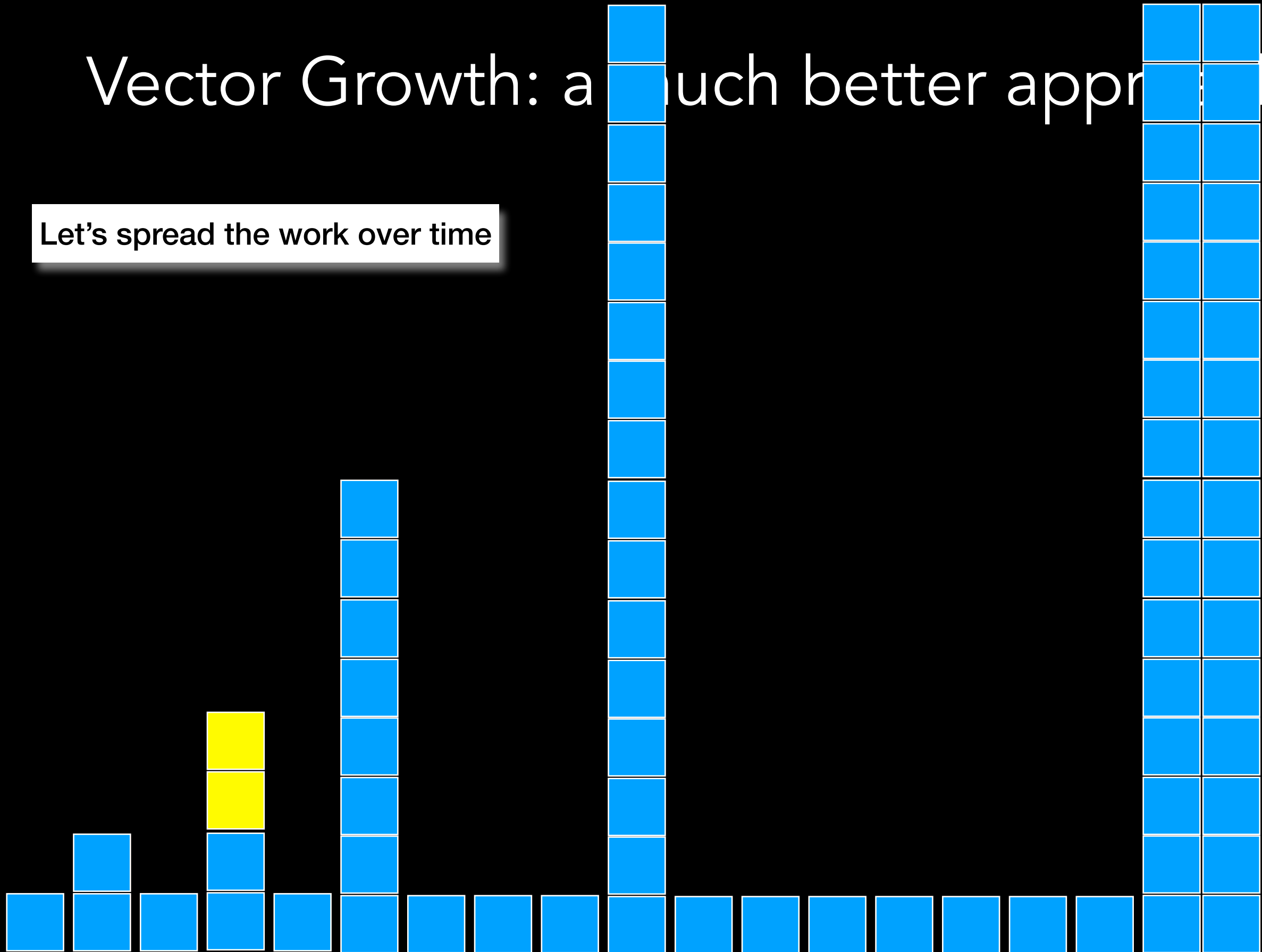
buffer_ =
len_ = 5
capacity_ = 10



[illegible]

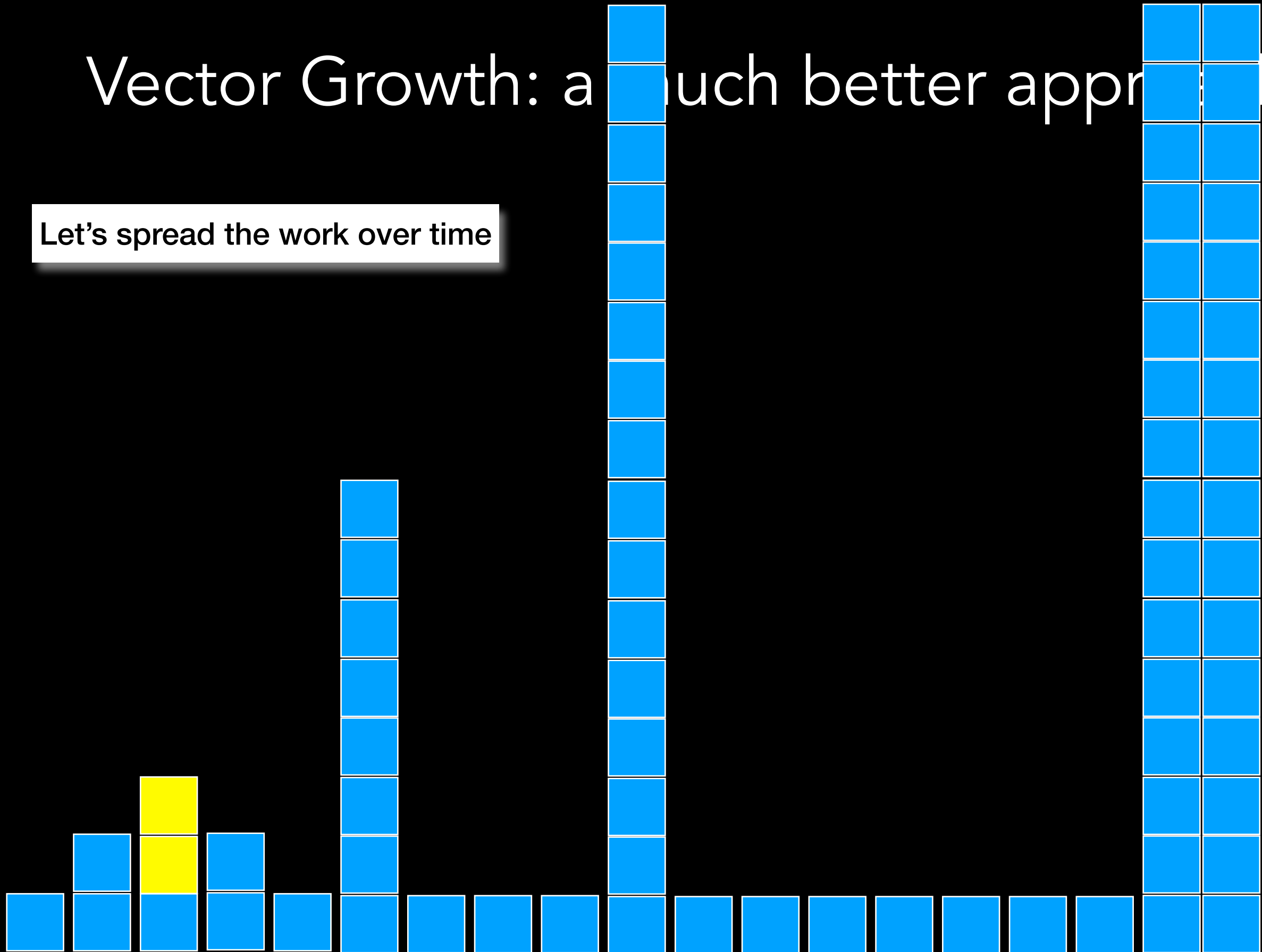
Vector Growth: a much better approach

Let's spread the work over time



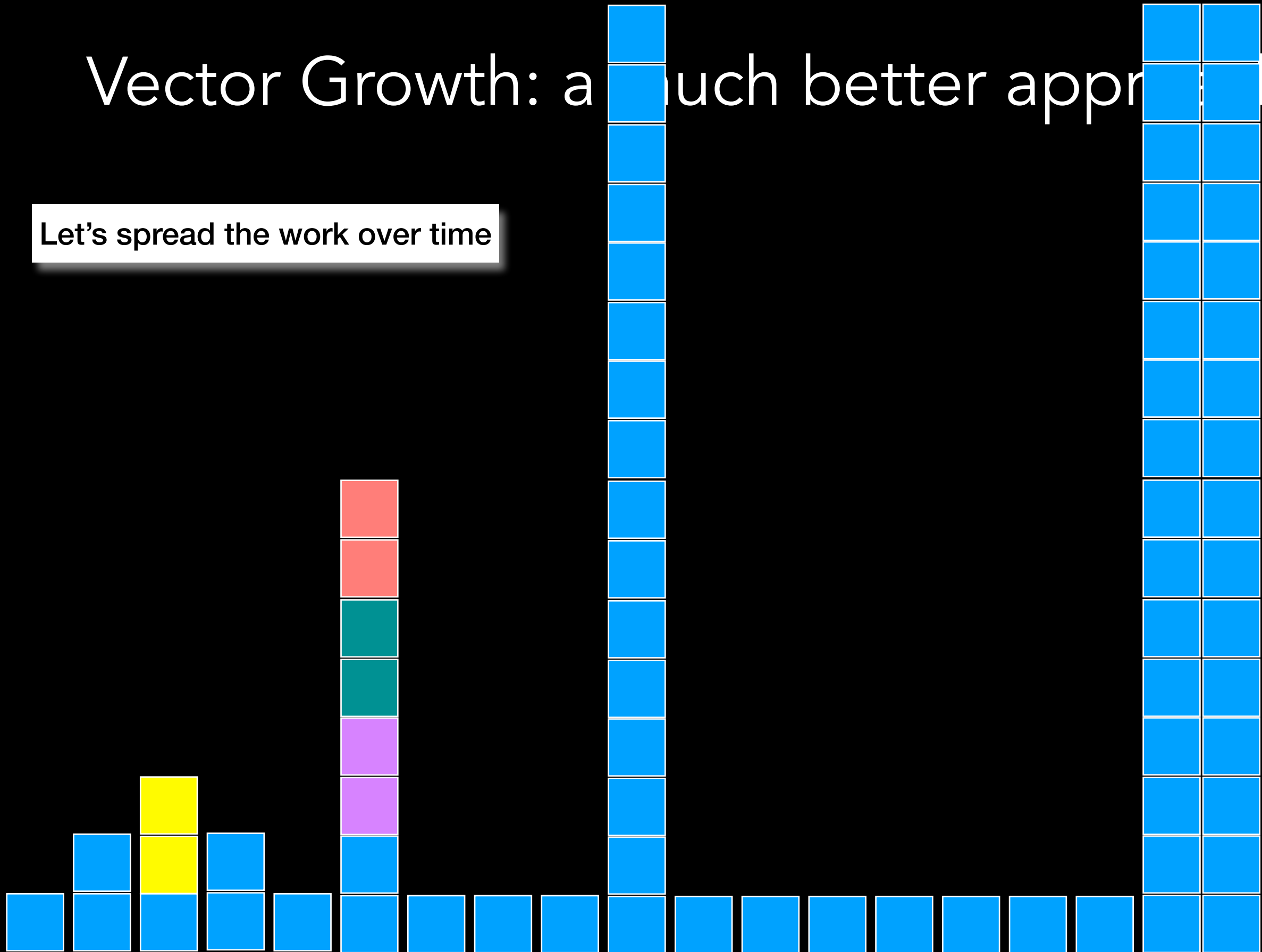
Vector Growth: a much better approach

Let's spread the work over time



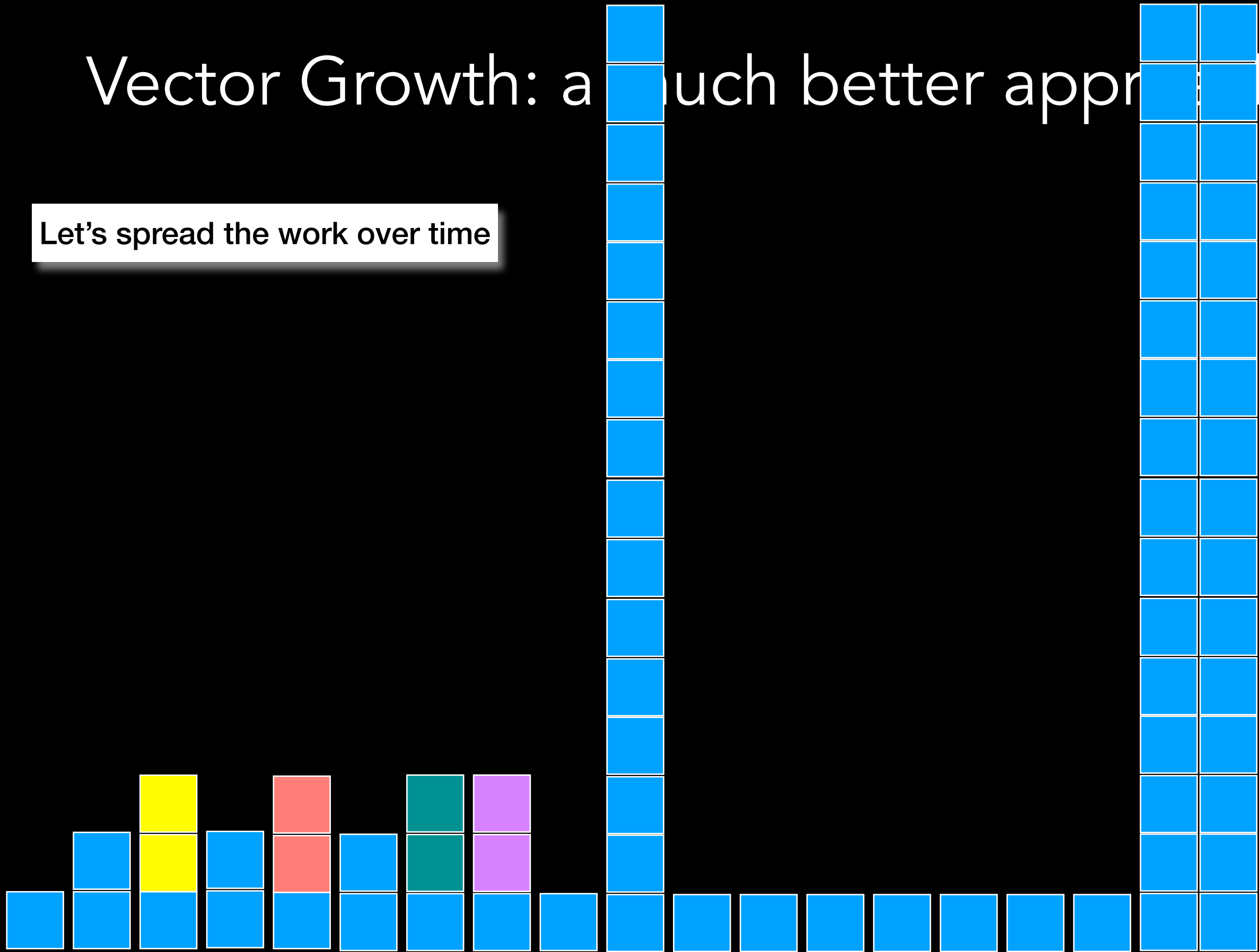
Vector Growth: a much better approach

Let's spread the work over time



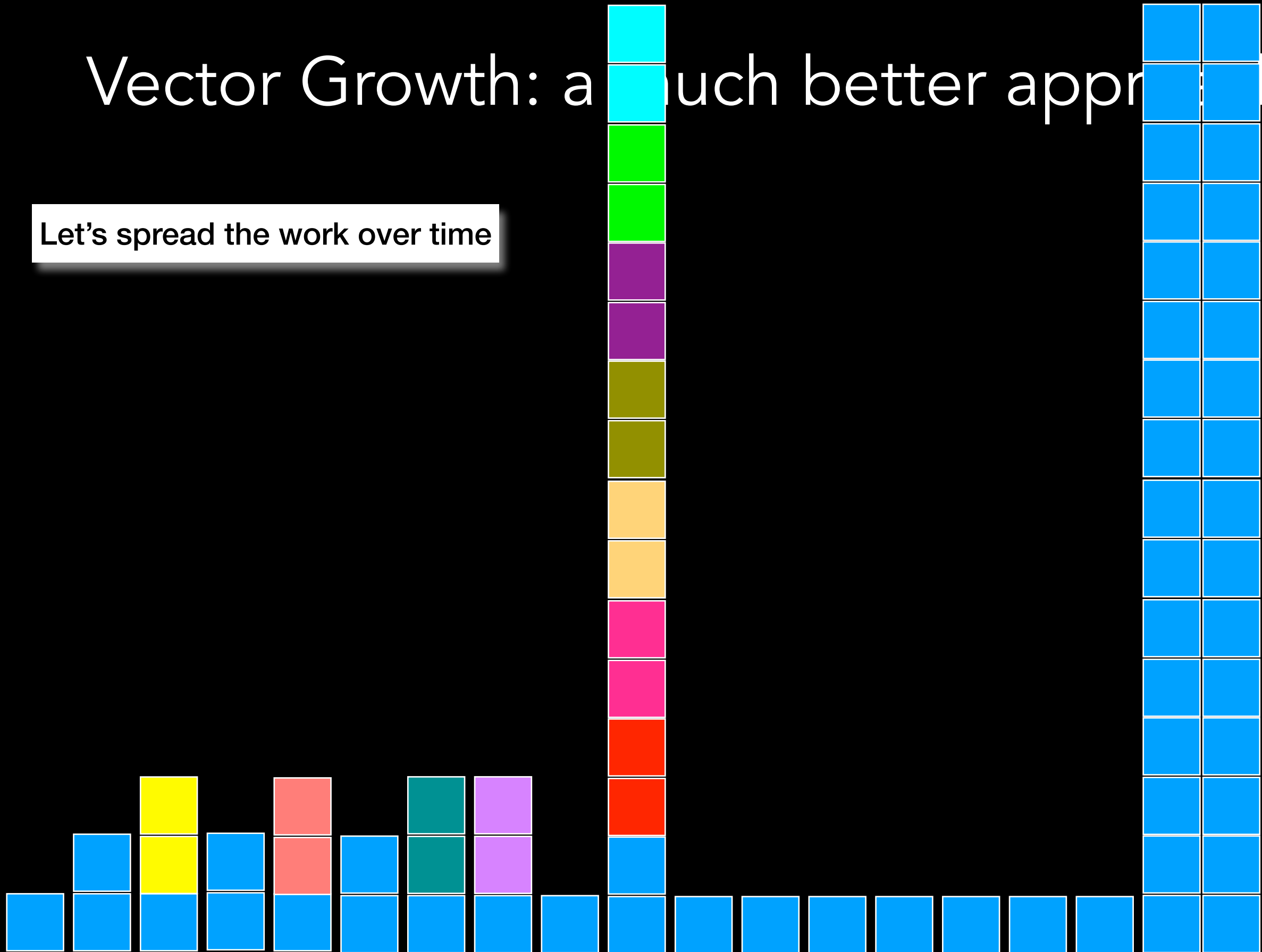
Vector Growth: a much better approach

Let's spread the work over time



Vector Growth: a much better approach

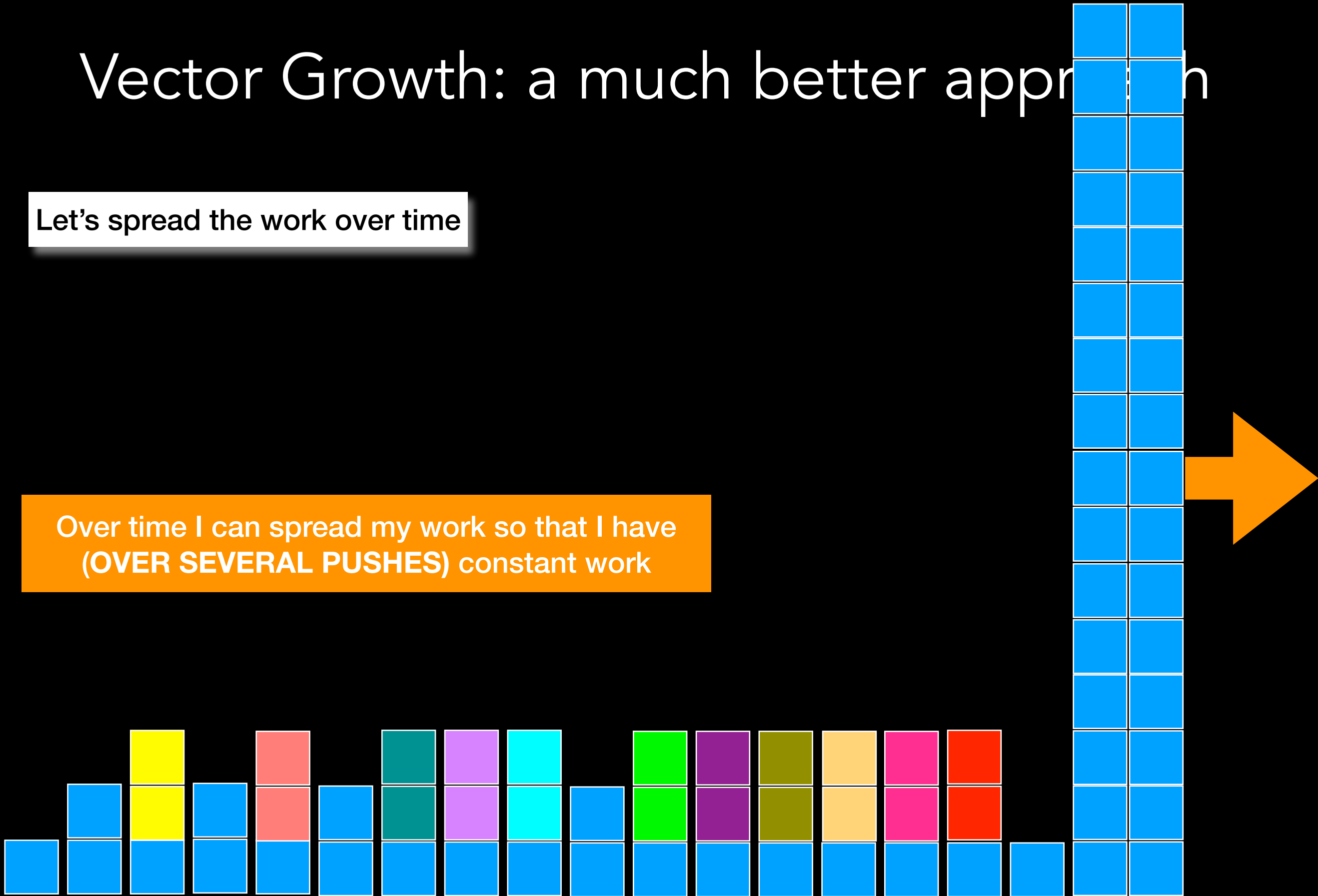
Let's spread the work over time



Vector Growth: a much better approach

Let's spread the work over time

Over time I can spread my work so that I have
(OVER SEVERAL PUSHES) constant work

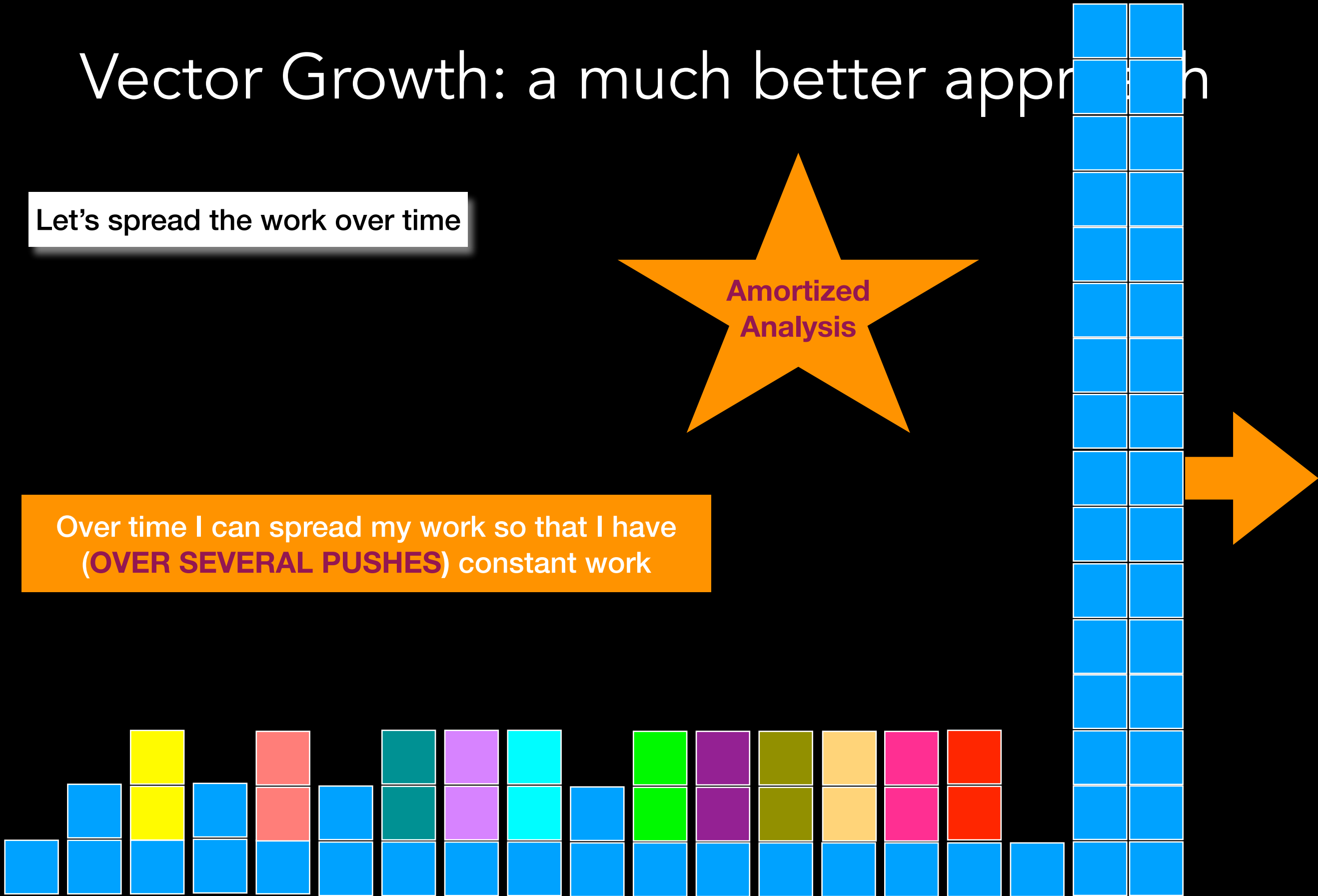


Vector Growth: a much better approach

Let's spread the work over time

Amortized
Analysis

Over time I can spread my work so that I have
(**OVER SEVERAL PUSHES**) constant work



Vector Growth summarized

If it grows by 1, $O(n^2)$ over time (**n pushes** - AMORTIZED ANALYSIS)

If it grows by 2, push takes roughly half the "steps" but still $O(n^2)$ over time (**n pushes** - AMORTIZED ANALYSIS)

If it doubles its size, push takes $O(1)$ over time (**n pushes** - AMORTIZED ANALYSIS)

A steadily shrinking Stack

If a large vector suddenly goes almost empty it will still occupy the same space.



I will not shrink!



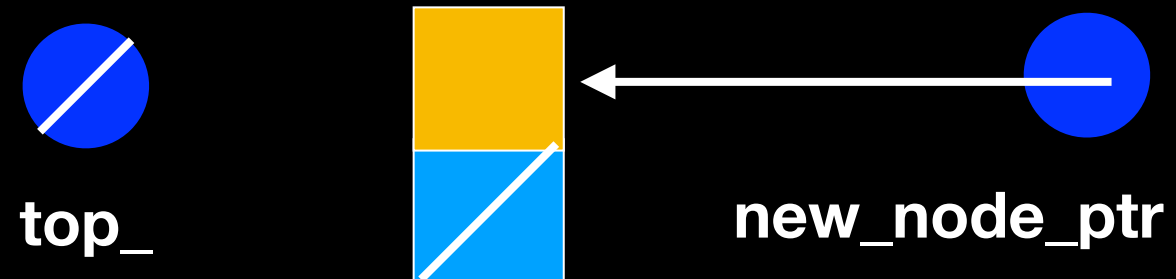
Linked Chain



top_

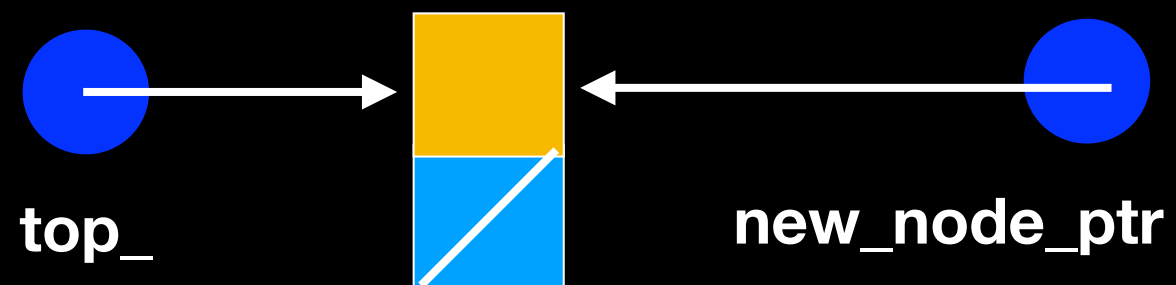
Linked Chain

push



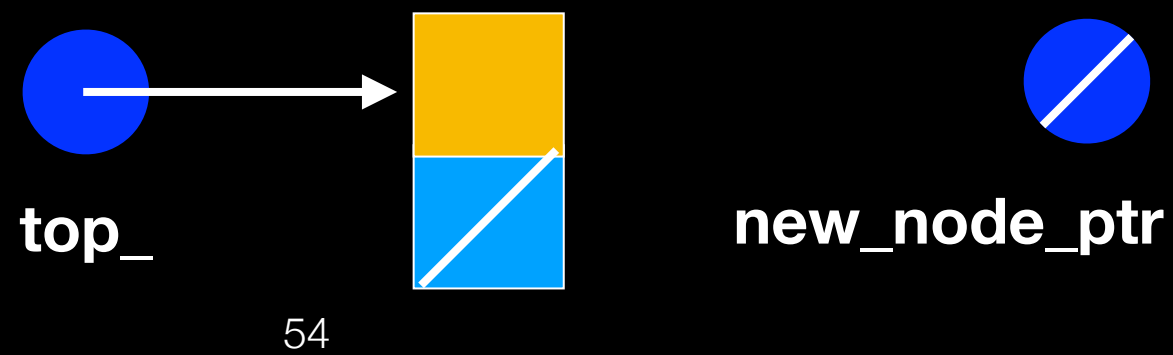
Linked Chain

push

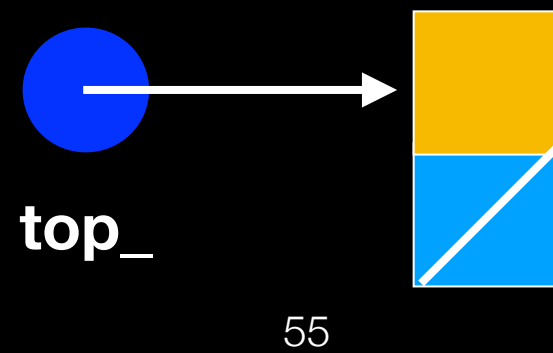


Linked Chain

push

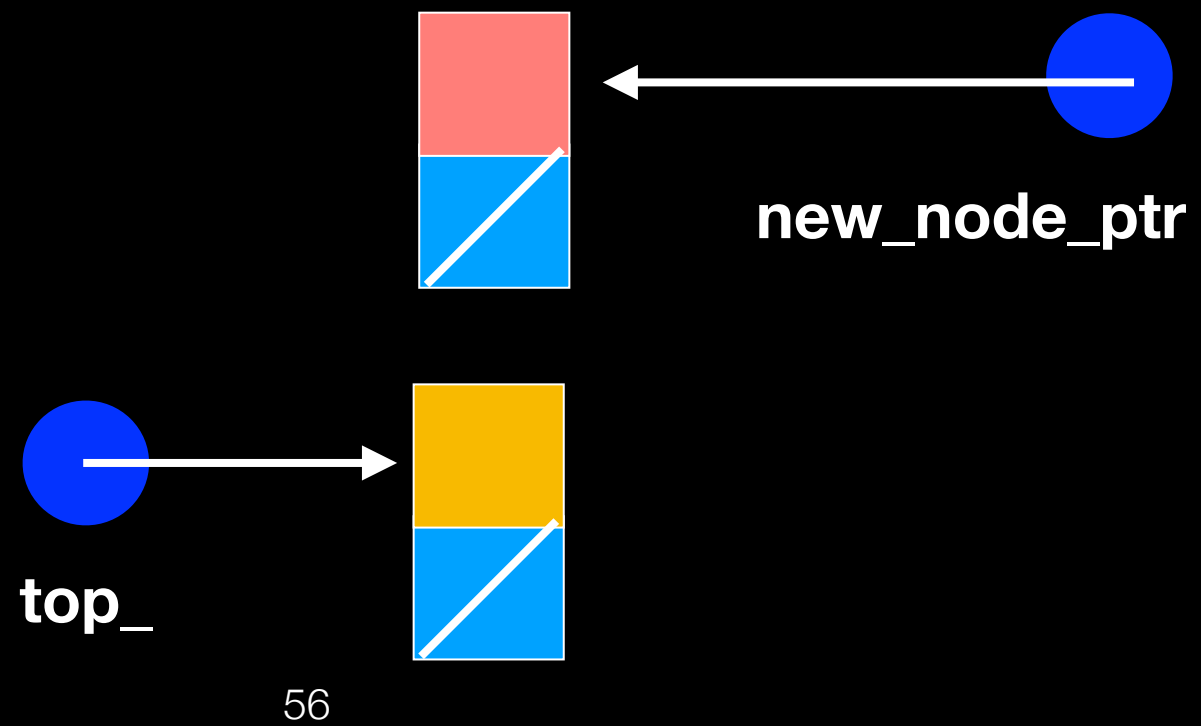


Linked Chain



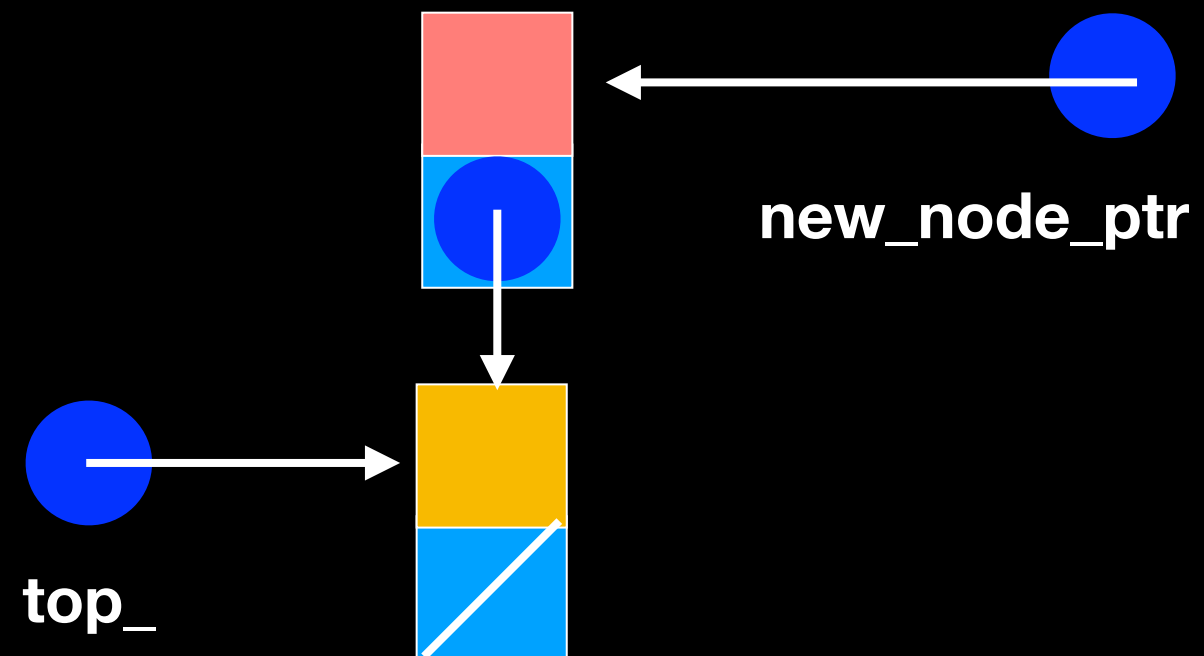
Linked Chain

push



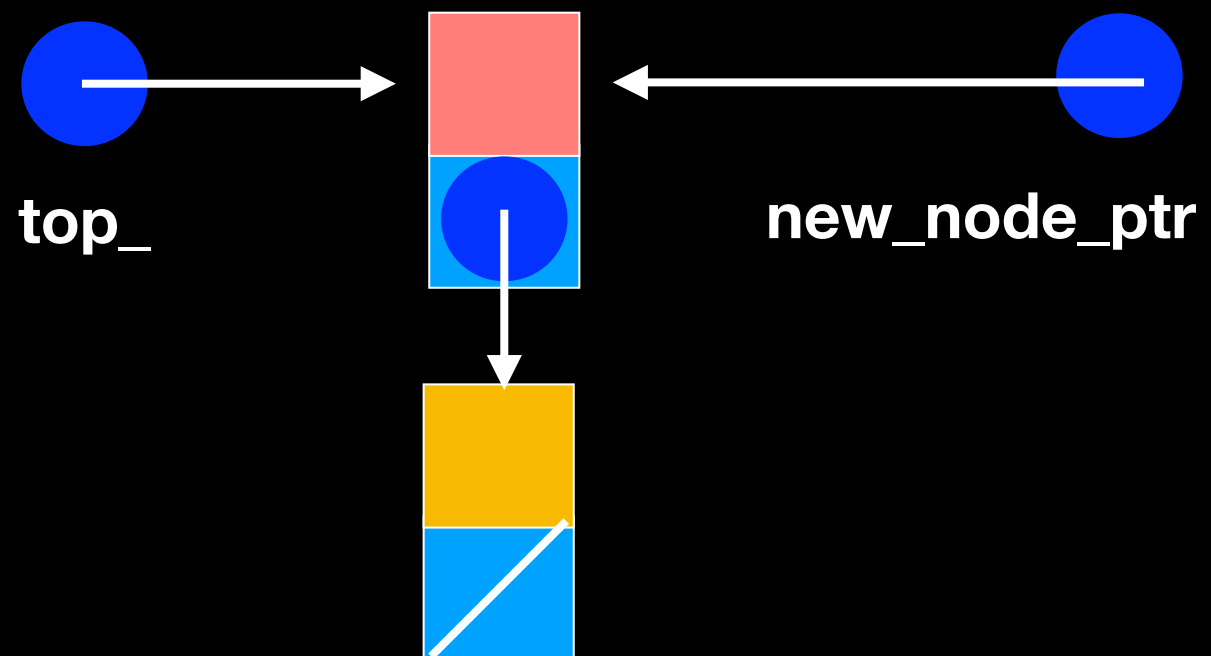
Linked Chain

push



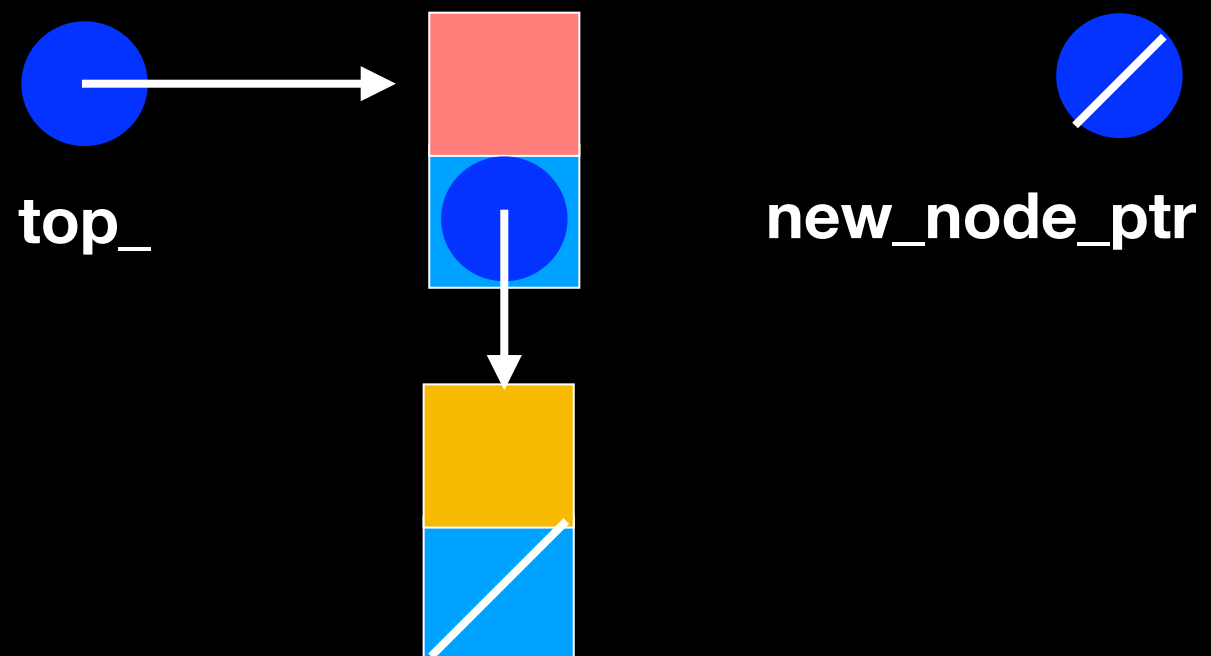
Linked Chain

push

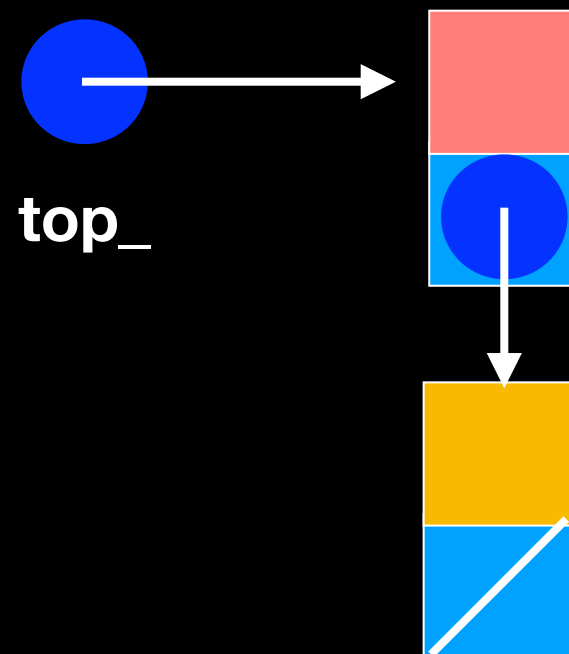


Linked Chain

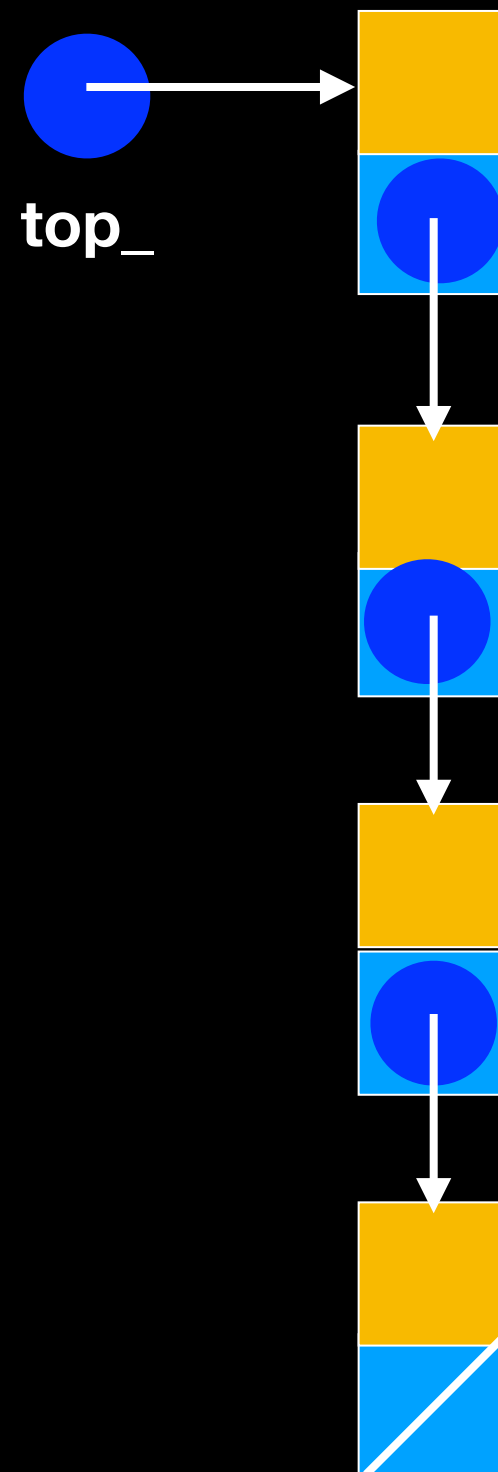
push



Linked Chain

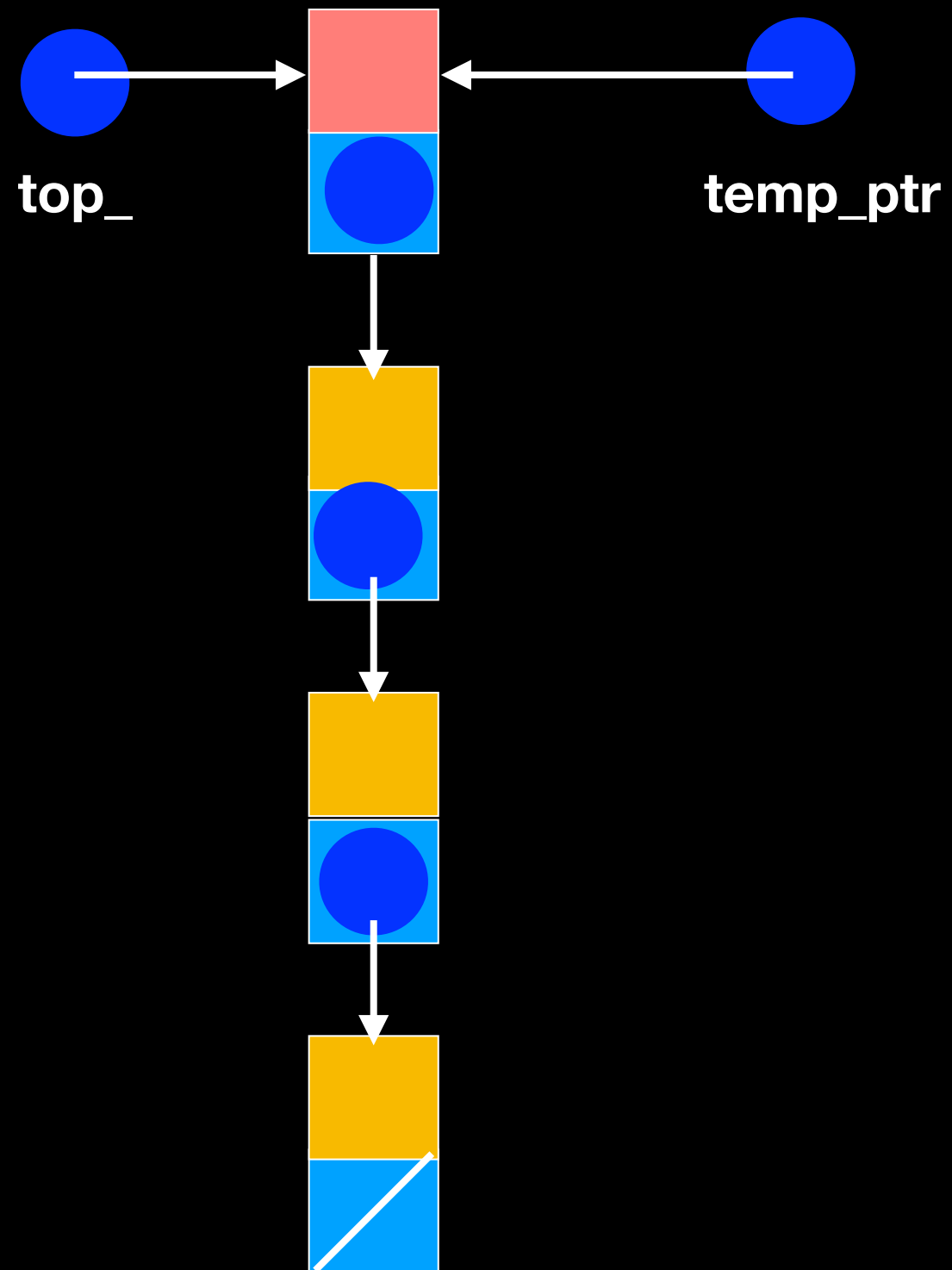


Linked Chain



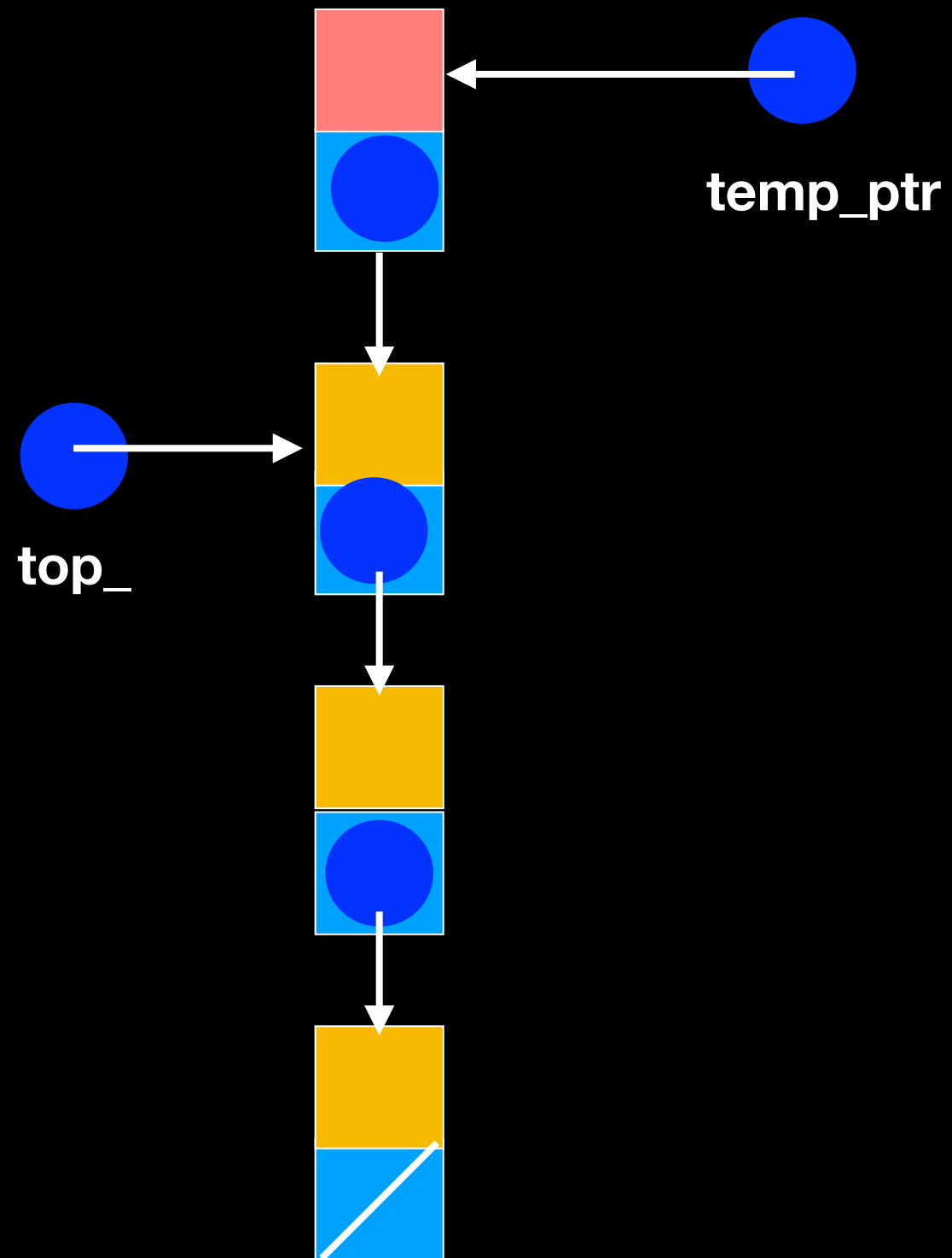
Linked Chain

pop



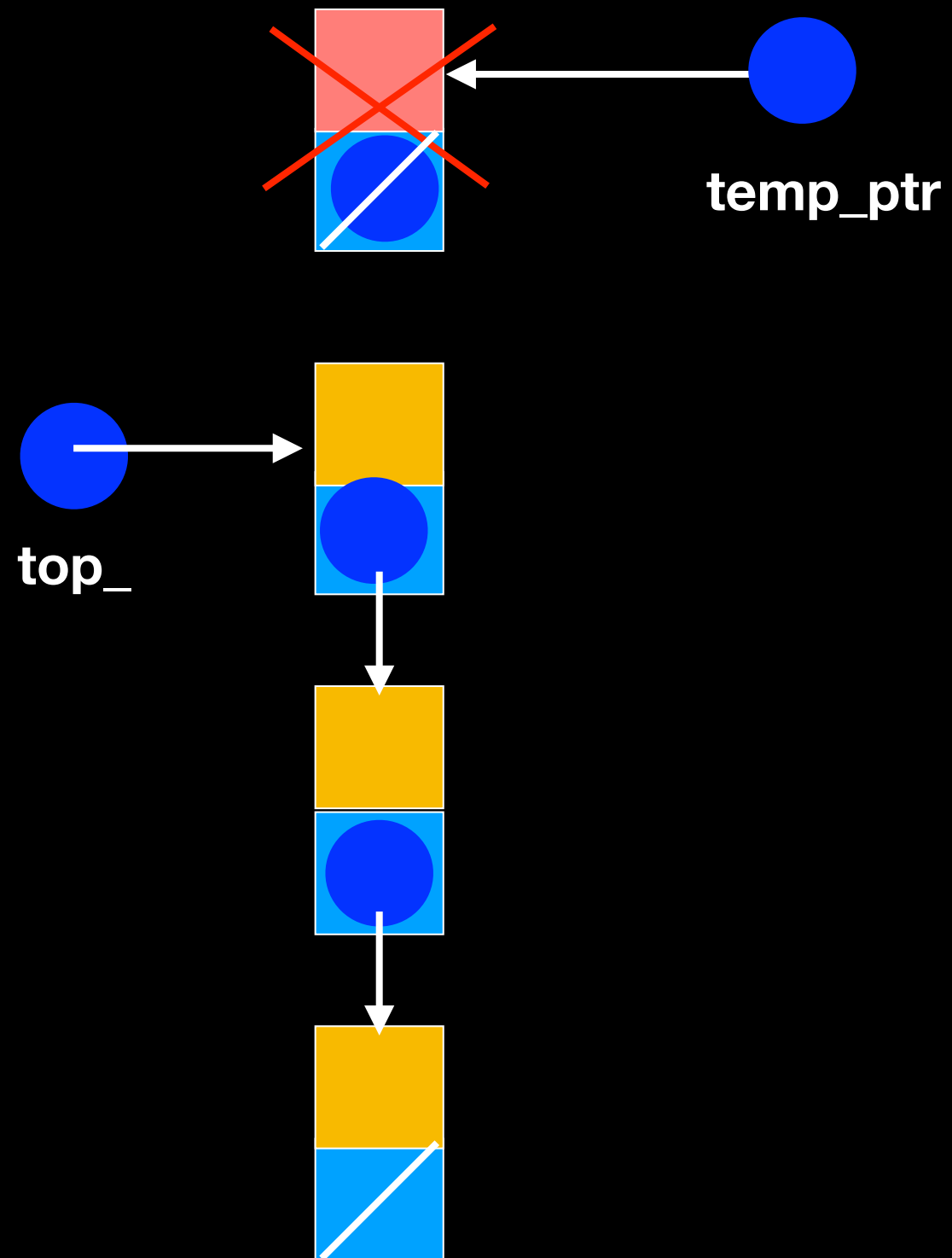
Linked Chain

pop



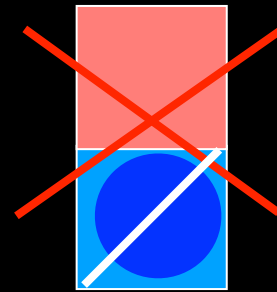
Linked Chain

pop



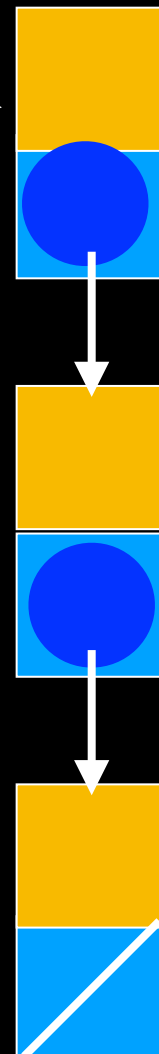
Linked Chain

pop

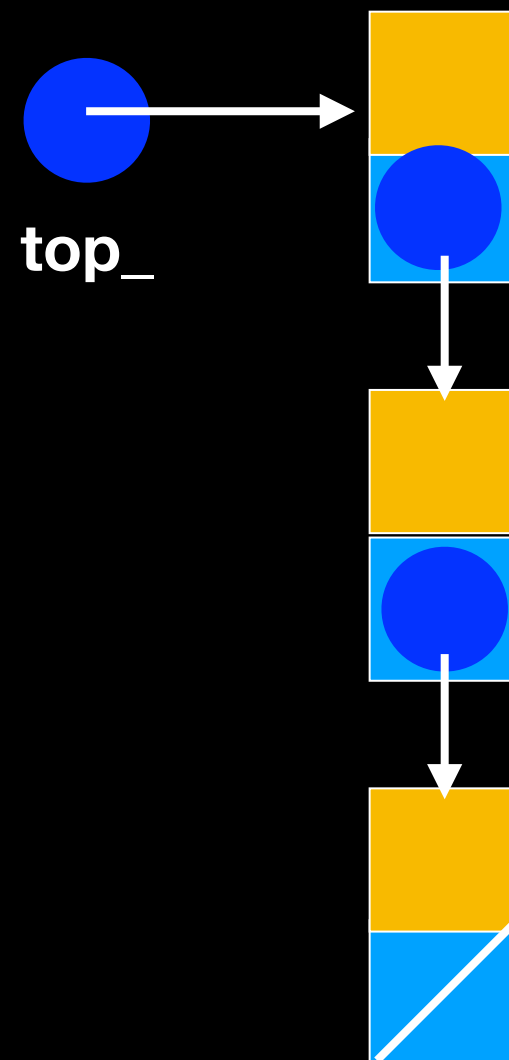


temp_ptr

top_



Linked Chain



Linked-Chain Analysis

1 assignment + 1 increment/decrement = $O(1)$

size : $O(1)$

isEmpty: $O(1)$

push: $O(1)$

pop : $O(1)$

top : $O(1)$

GREAT!!!! And there is no "Except" case here, every operation is $O(1)$!

To summarize

Array: $O(1)$ for push and pop, but size is bounded

Vector: size is unbounded but

- Some push operations take $O(1)$, others take

- $O(n) \rightarrow O(1)$ over time (AMORTIZED ANALYSIS)

Linked-Chain: $O(1)$ for push and pop and size is unbounded

Implement Stack ADT

```
#ifndef STACK_H_
#define STACK_H_

template<class T>
class Stack
{

public:
    Stack();
    void push(const T& newEntry); // adds an element to top of stack
    void pop(); // removes element from top of stack
    T top() const; // returns a copy of element at top of stack
    int size() const; // returns the number of elements in the stack
    bool isEmpty() const; // returns true if no elements on stack false otherwise

private:
    // implementation-specific members here

}; //end Stack

#include "Stack.cpp"
#endif // STACK_H_
```

What should we add here to implement it as a linked chain?

Implement Stack ADT

```
#ifndef STACK_H_
#define STACK_H_

template<class T>
class Stack
{
public:
    Stack();
    ~Stack(); // destructor
    Stack(const Stack<T>& a_stack); //copy constructor
    void push(const T& newEntry); // adds an element to top of stack
    void pop(); // removes element from top of stack
    T top() const; // returns a copy of element at top of stack
    int size() const; // returns the number of elements in the stack
    bool isEmpty() const; // returns true if no elements on stack false otherwise

private:
    Node<T>* top_; // Pointer to top of stack
    int item_count; // number of items currently on the stack

}; //end Stack

#include "Stack.cpp"
#endif // STACK_H_
```

Usage Example

<https://www.geeksforgeeks.org/cpp/stack-in-cpp-stl/>

End Stack Review

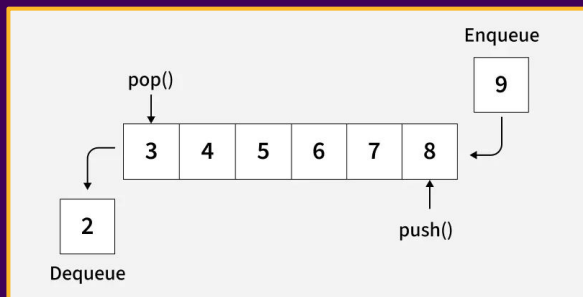
Try solving the following Stack problems for practice:

- [Valid Parenthesis](#)
- [Evaluate Reverse Polish Notation](#)
- [Palindrome Linked List](#)
- [Remove All Adjacent Duplicates in a String](#)
- [Minimum String Length After Removing Substrings](#)
- [More Stack Problems](#)

Best way to get better at solving problems is through practice!

Queue

Container Adapters



- Queue follows First In First Out (FIFO) insertion and deletion
 - ◆ Inserting in one end
 - ◆ Deleting from the other end
 - ◆ Efficient `enqueue` and `dequeue`
- Things to know about `queue`
 - ◆ Container adapter means you can change the underlying data structure
 - ◆ Container may be one of the standard containers, or any specifically designed container class with the following:
 - `empty()`
 - `size()`
 - `front()`
 - `back()`
 - `push_back()`
 - `pop_front()`
 - ◆ Standard container classes `deque`, and `list` can be used
 - ◆ Default container is `deque`
- [Find out more in the documentation](#)

Usage Example

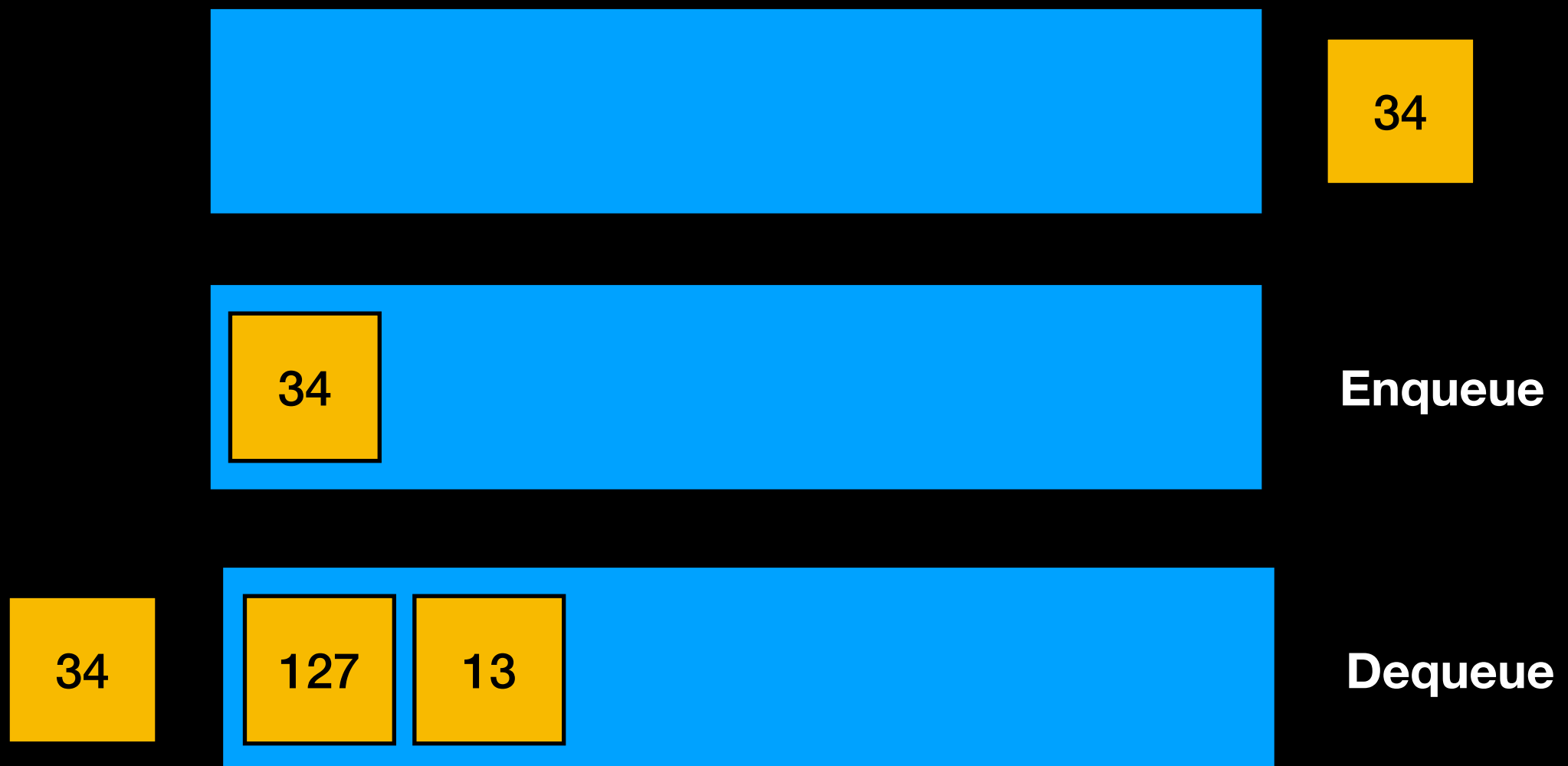
<https://www.geeksforgeeks.org/cpp/queue-cpp-stl/>

Queue Review

The following slides are provided by Professor Tiziana Ligorio

Recap

FIFO structure: First In First Out



Queue ADT

```
#ifndef QUEUE_H_
#define QUEUE_H_

template<class T>
class Queue
{
public:
    Queue();
    void enqueue(const T& new_entry); // adds an element to back queue
    void dequeue(); // removes element from front of queue
    T front() const; // returns a copy of element at the front of queue
    int size() const; // returns the number of elements in the queue
    bool isEmpty() const; // returns true if no elements in queue, false otherwise

private:
    //implementation details here

}; //end Queue

#include "Queue.cpp"
#endif // QUEUE_H_`
```


Choose a Data Structure

Array?

Vector?

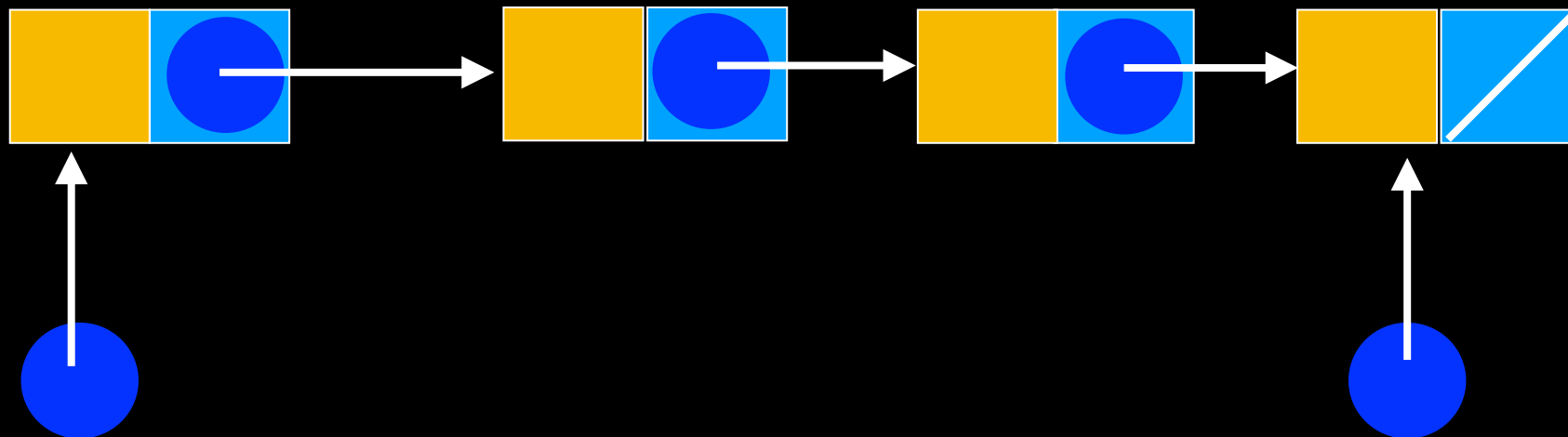
Linked List?

We are looking to enqueue and dequeue in $O(1)$ time

What is the main difference
btw stack and queue?

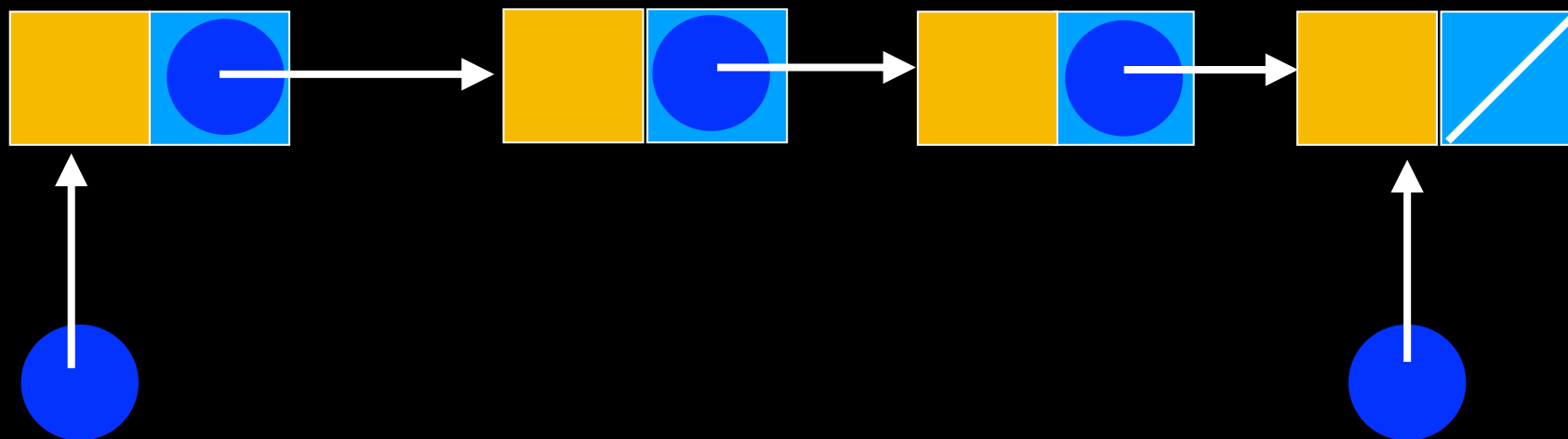
Singly Linked Chain

**Where is front?
Where is back?**

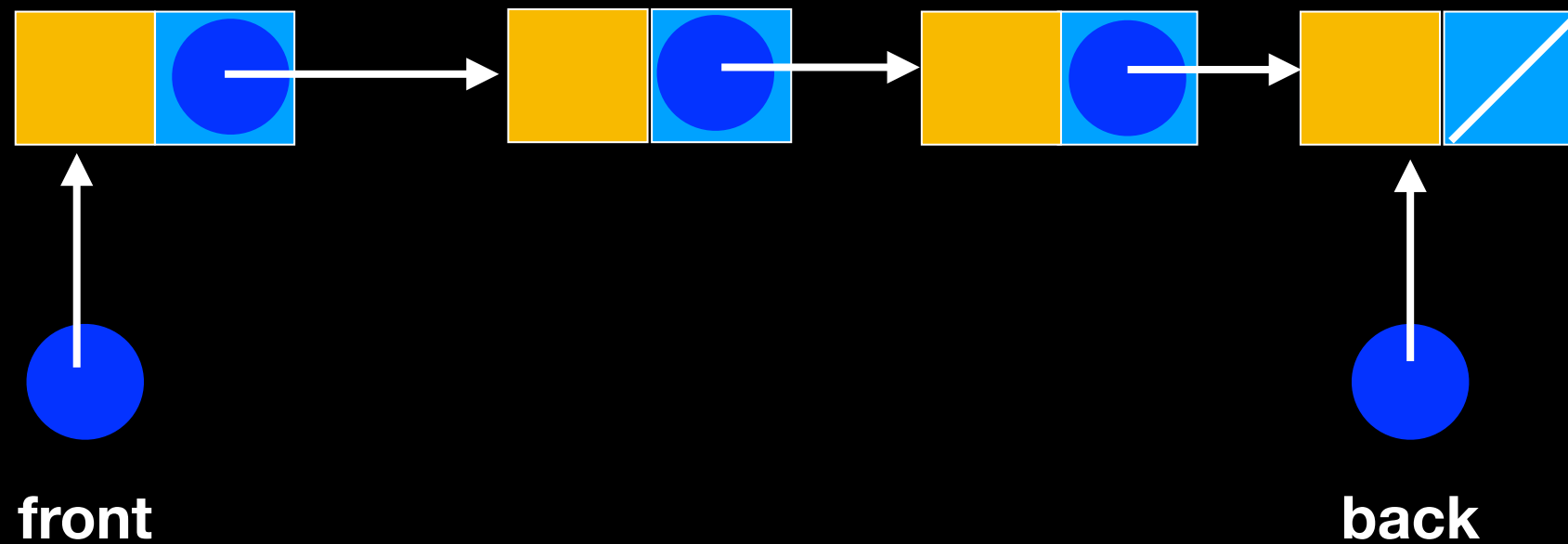


Singly Linked Chain

Deleting here is not $O(1)$
Because we don't have
pointer to previous node

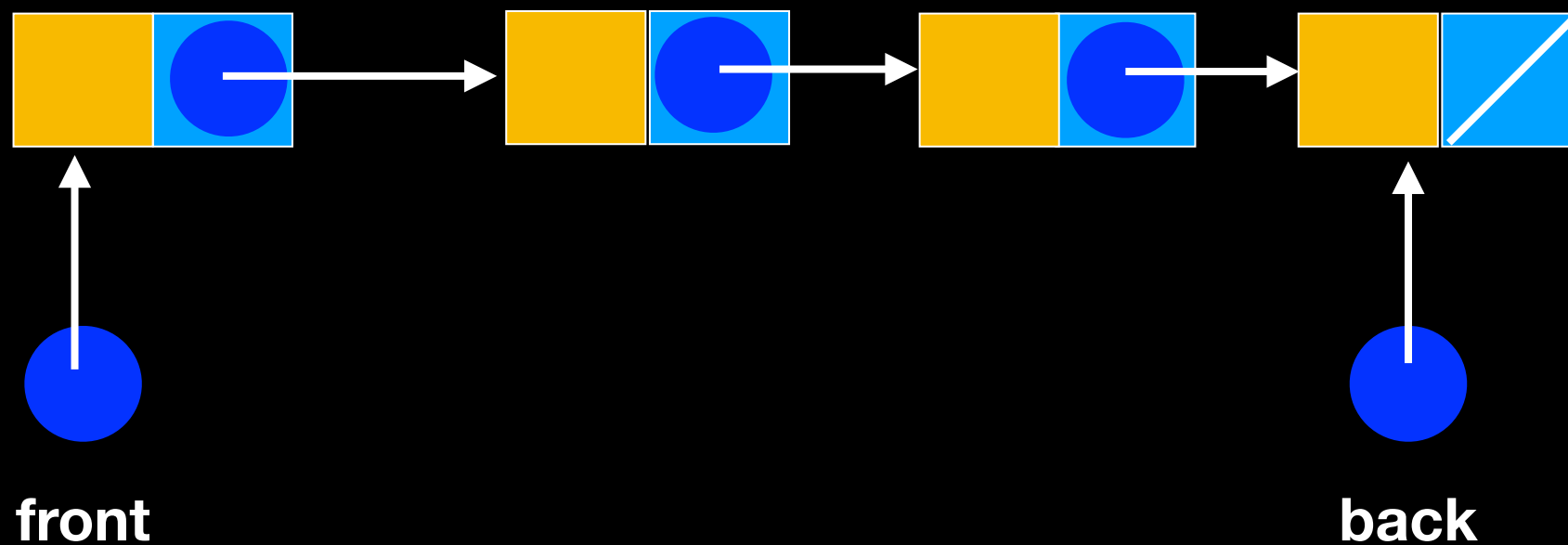


Singly Linked Chain



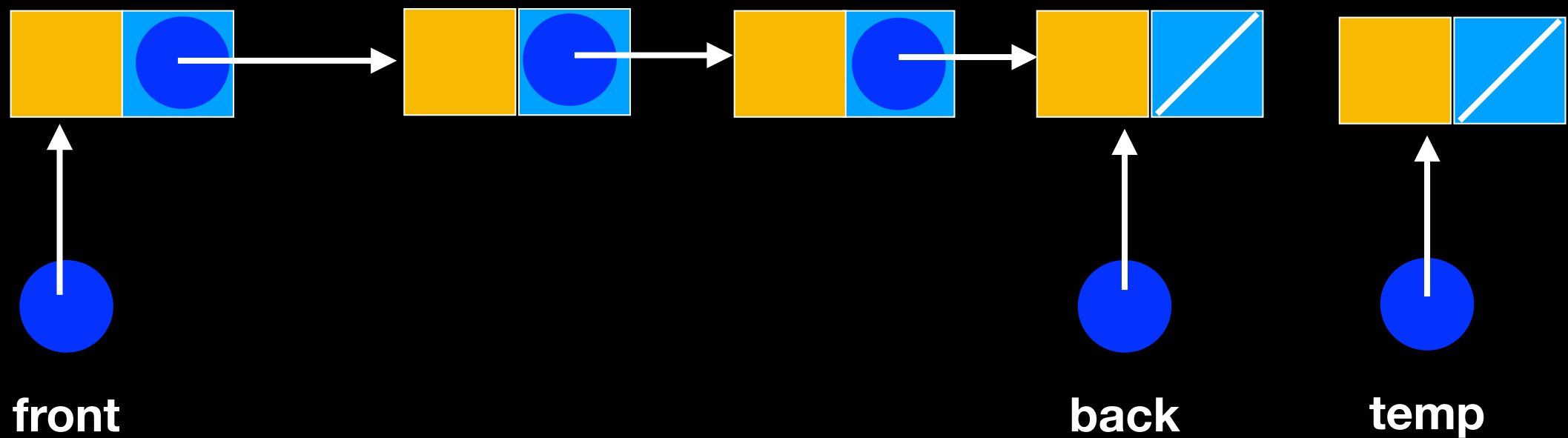
Singly Linked Chain

enqueue



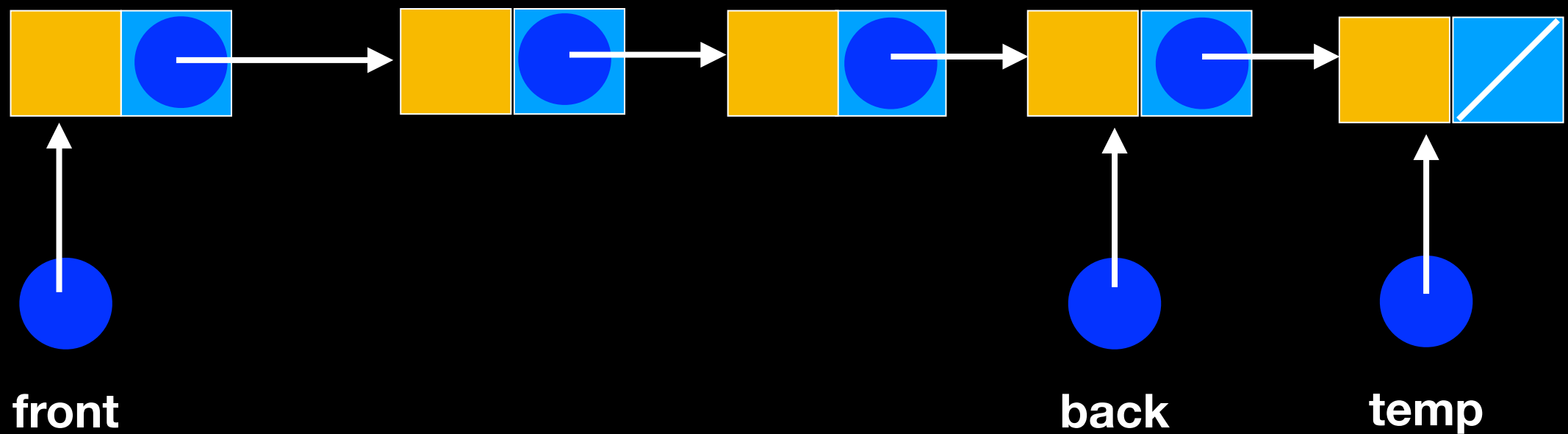
Singly Linked Chain

enqueue



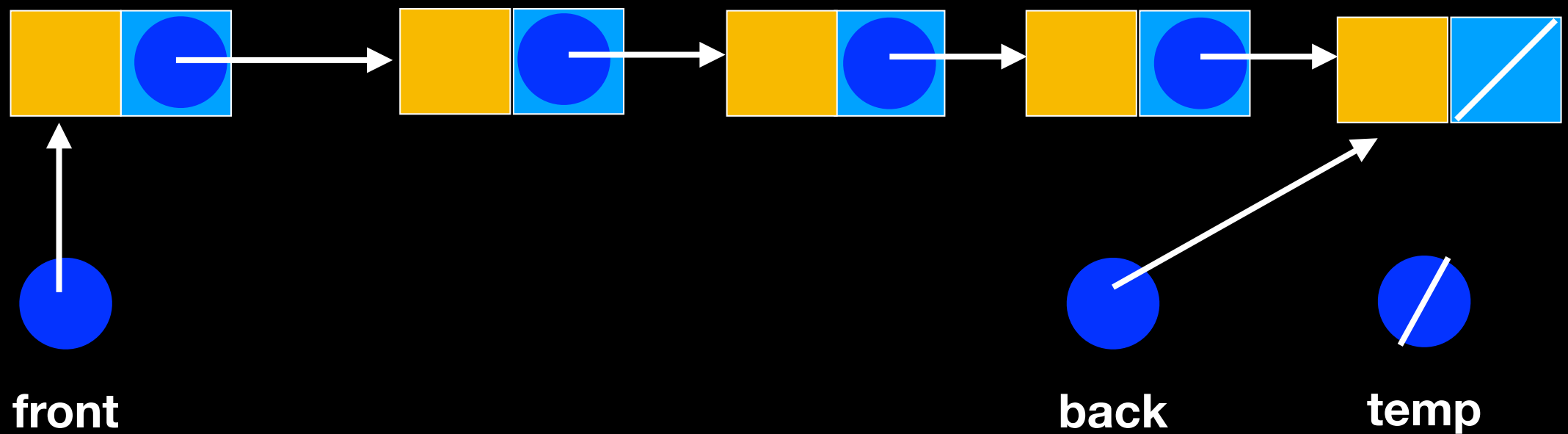
Singly Linked Chain

enqueue



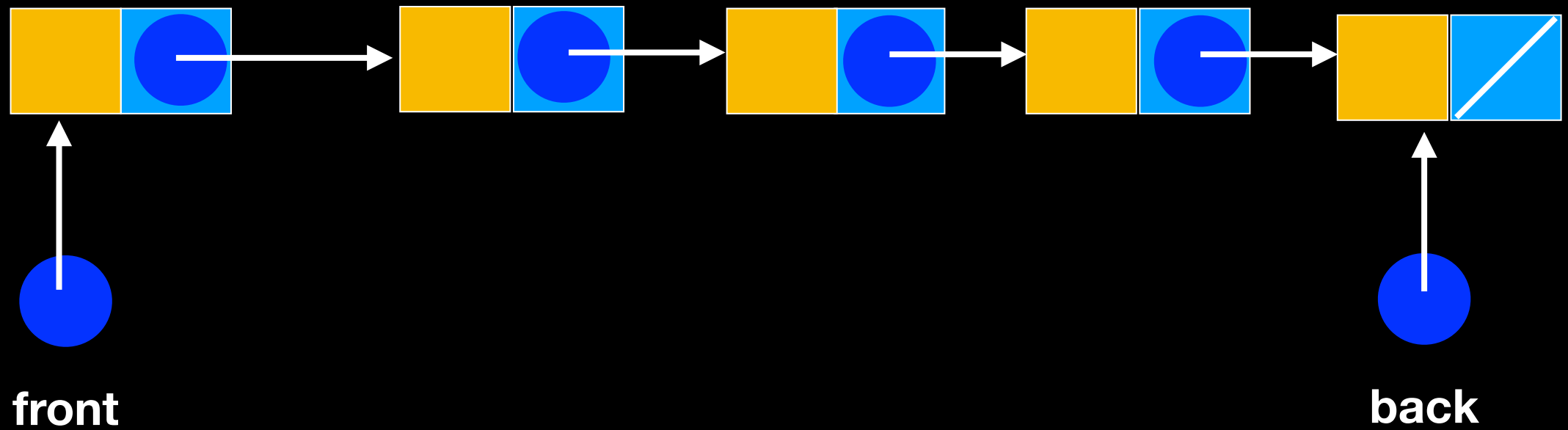
Singly Linked Chain

enqueue



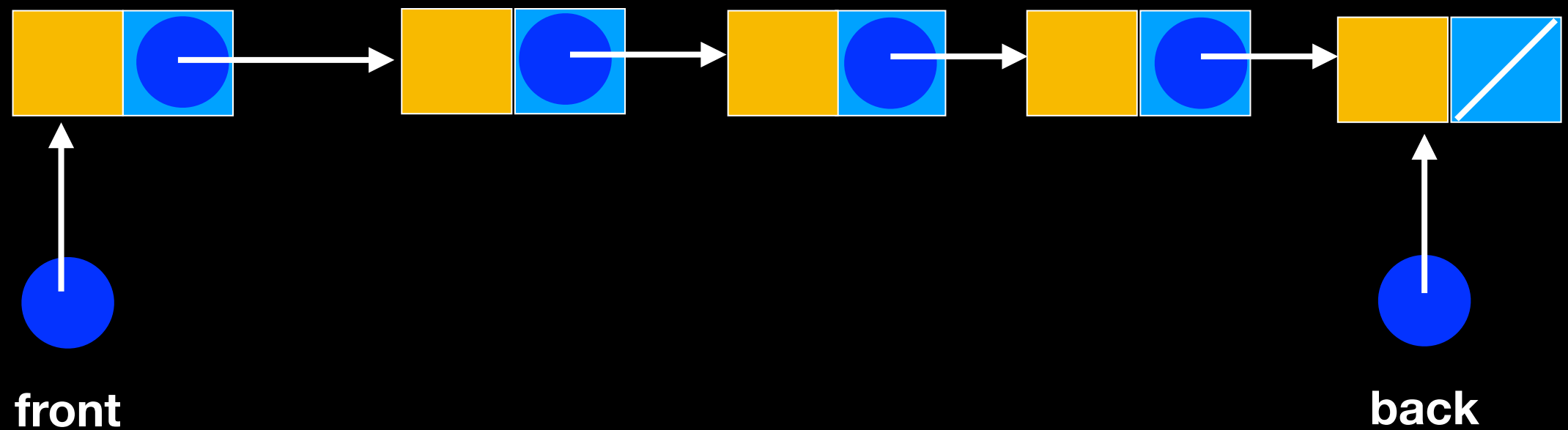
Singly Linked Chain

enqueue



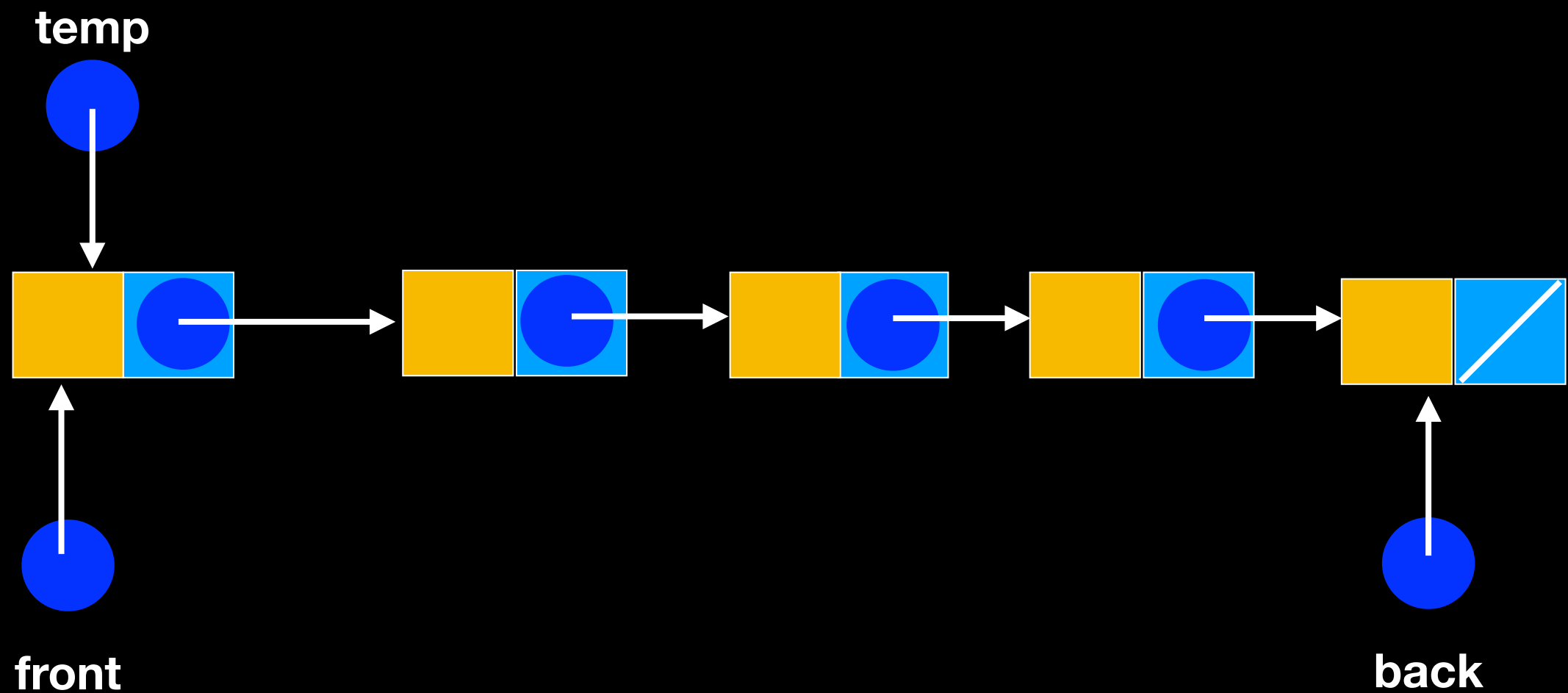
Singly Linked Chain

dequeue



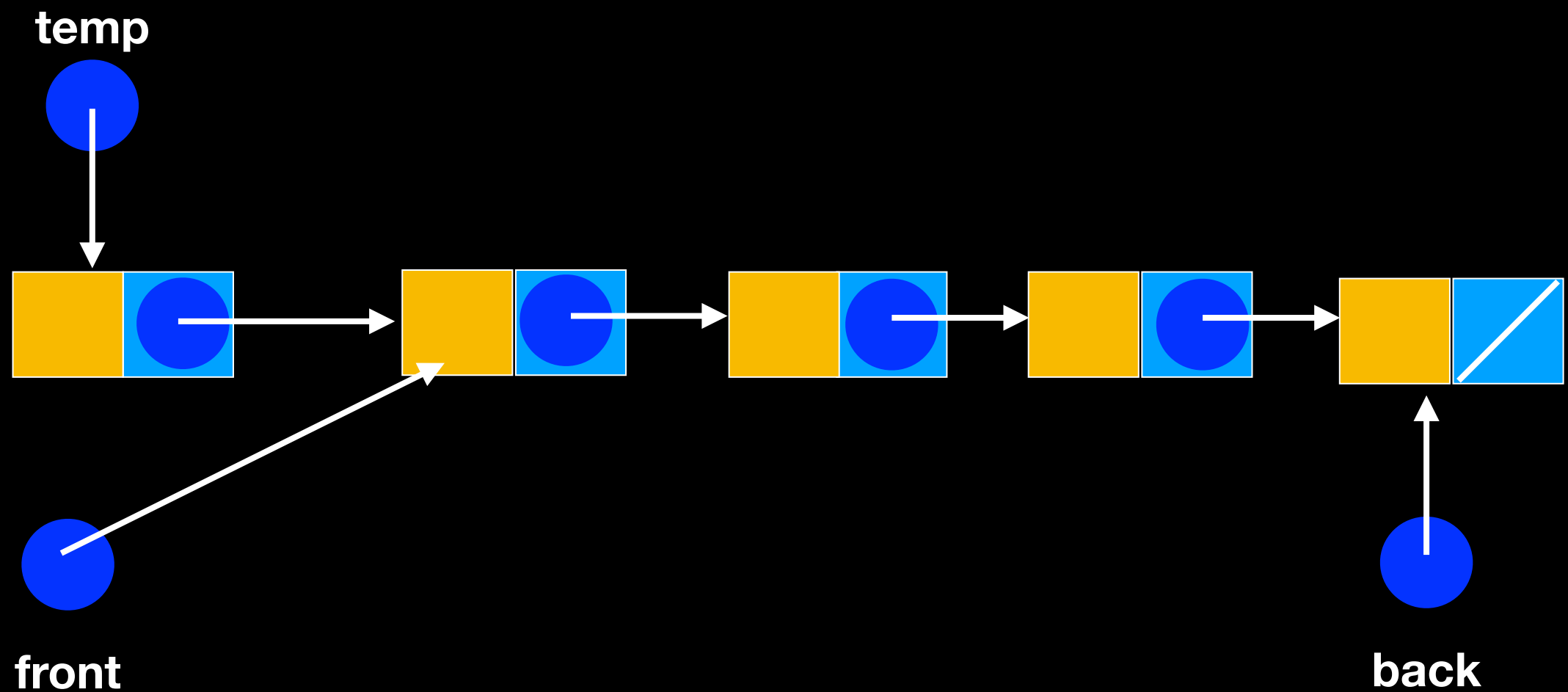
Singly Linked Chain

dequeue



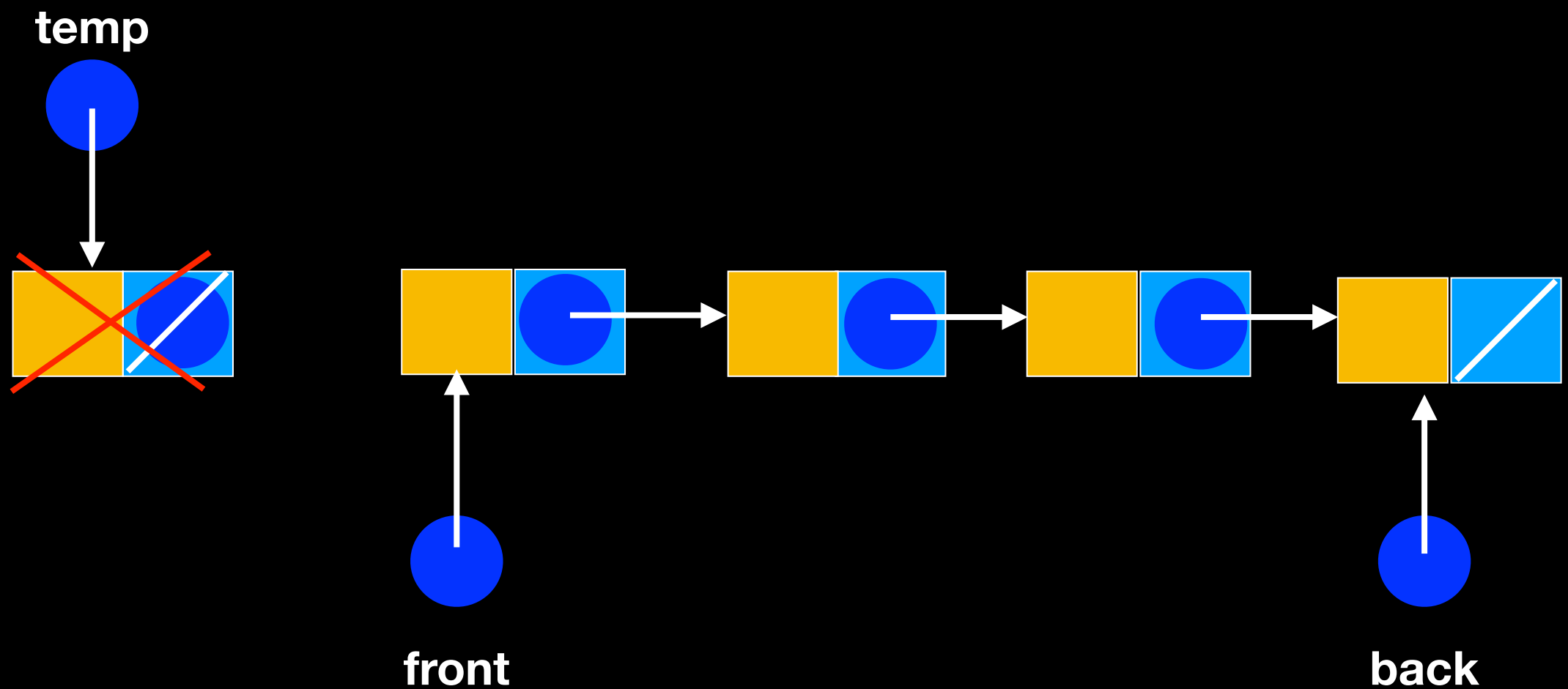
Singly Linked Chain

dequeue



Singly Linked Chain

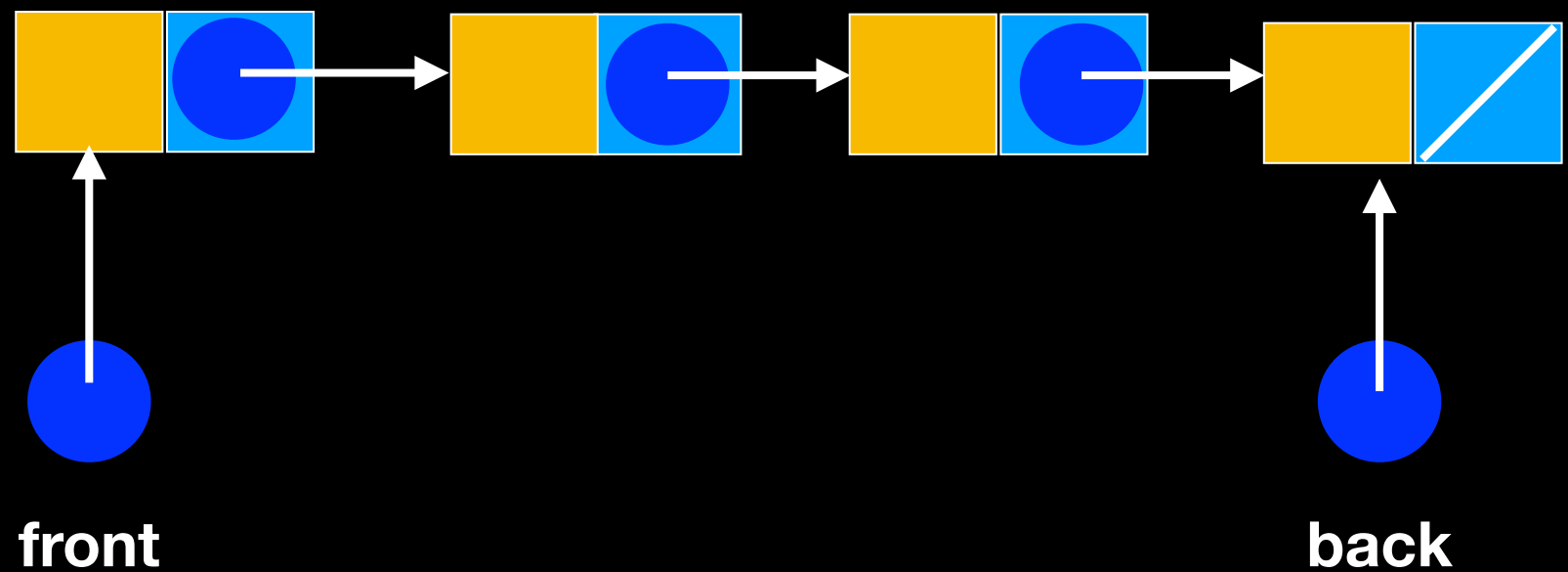
dequeue



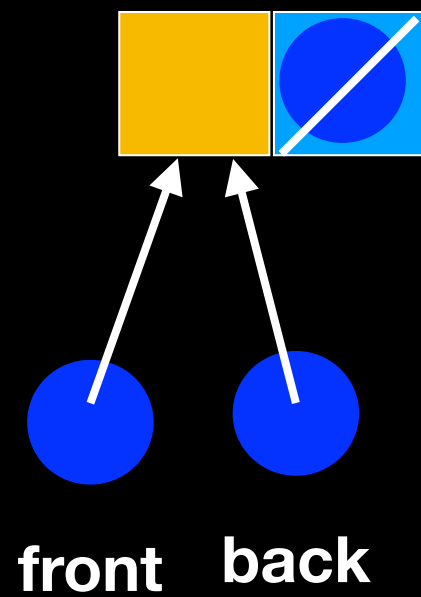
Singly Linked Chain

dequeue

temp



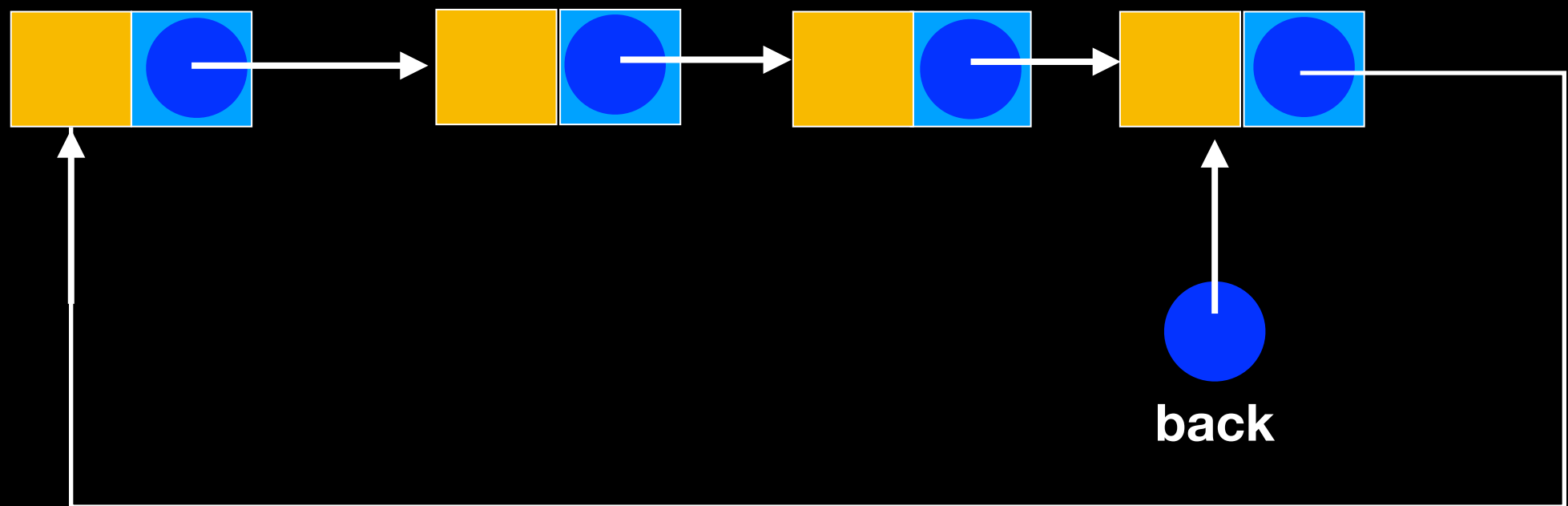
Singly Linked Chain



That's it!

Singly Linked Chain

An Alternative: A Circular Linked Chain

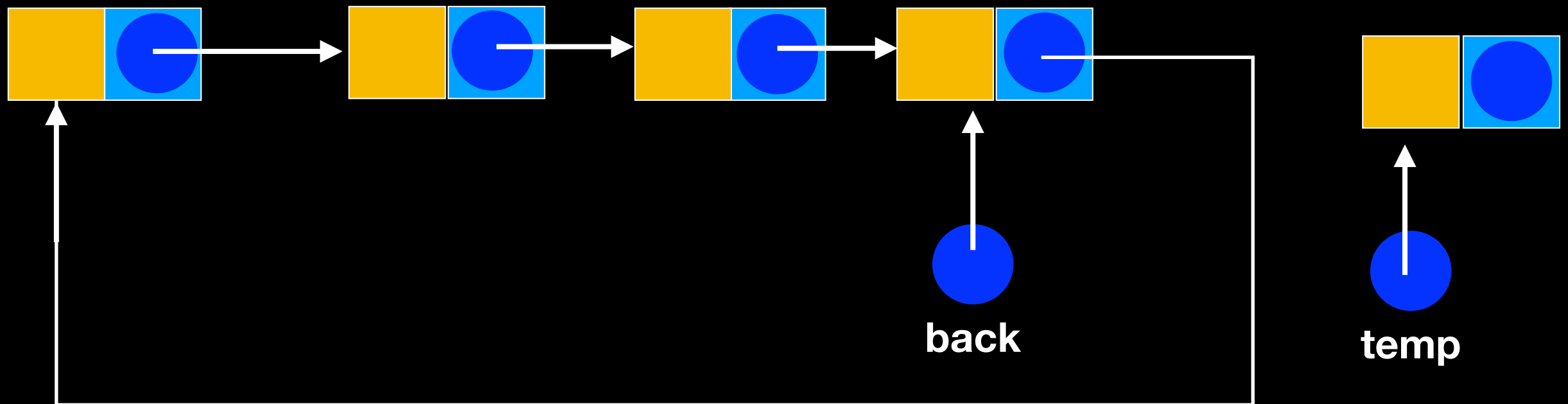


Singly Linked Chain

enqueue

**An Alternative:
A Circular Linked Chain**

Instantiate new node

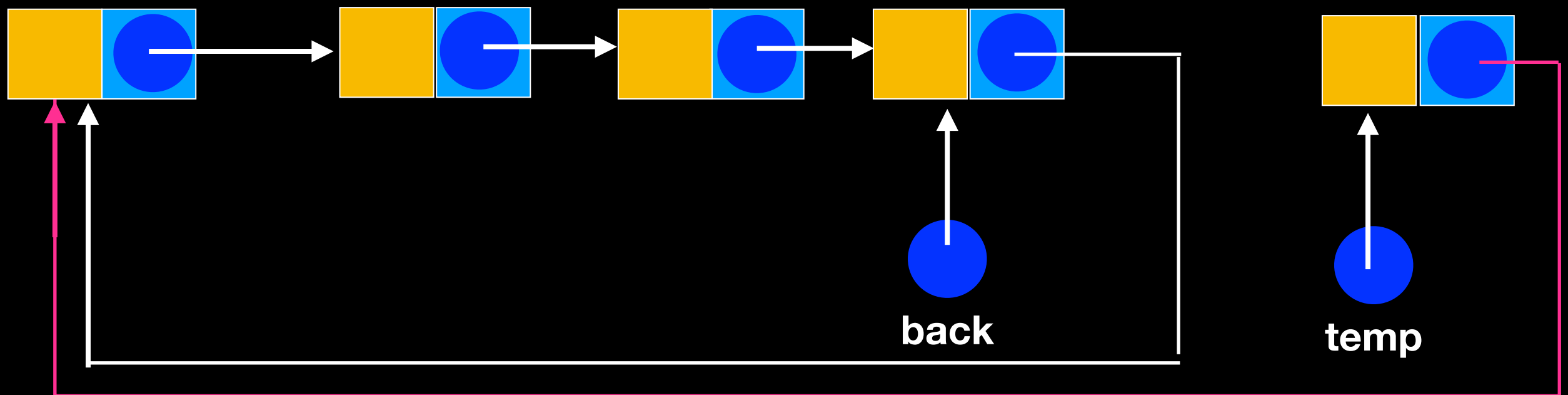


Singly Linked Chain

enqueue

**An Alternative:
A Circular Linked Chain**

```
temp->setNext(back->getNext());
```

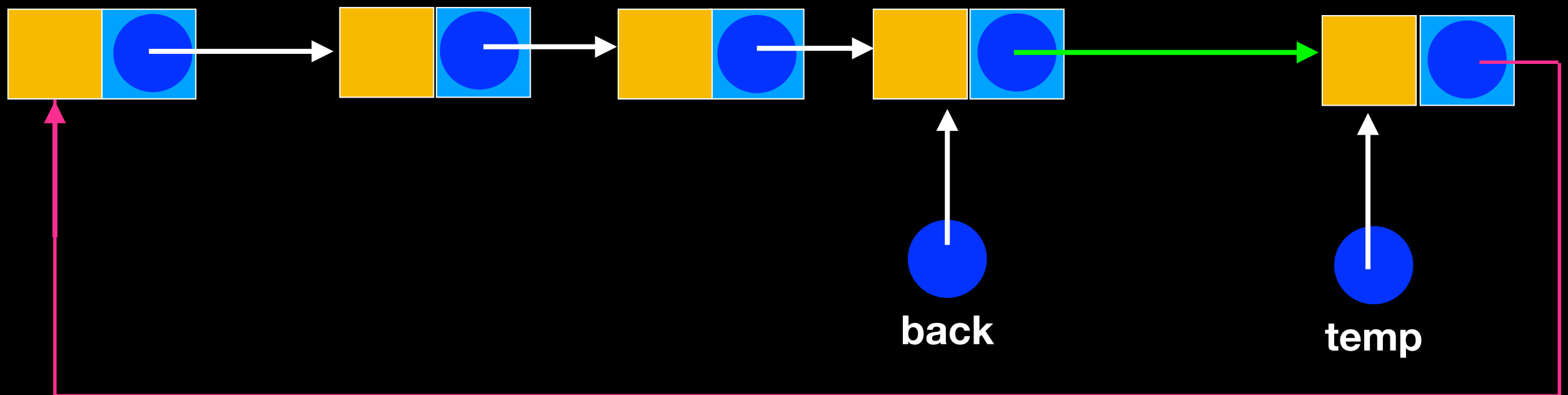


Singly Linked Chain

enqueue

**An Alternative:
A Circular Linked Chain**

```
back->setNext(temp);
```

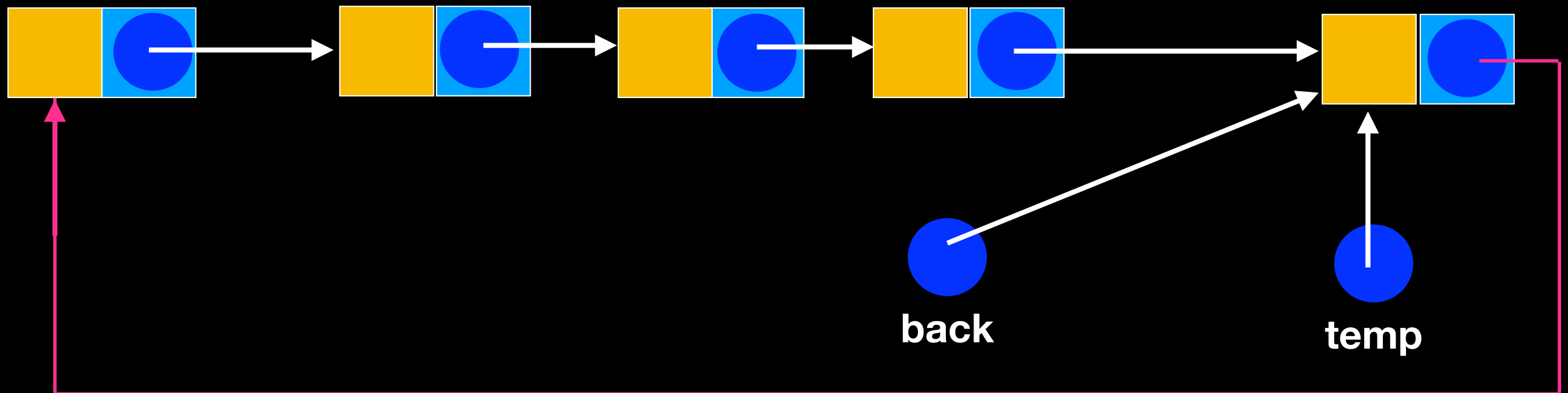


Singly Linked Chain

enqueue

**An Alternative:
A Circular Linked Chain**

```
back = temp;
```

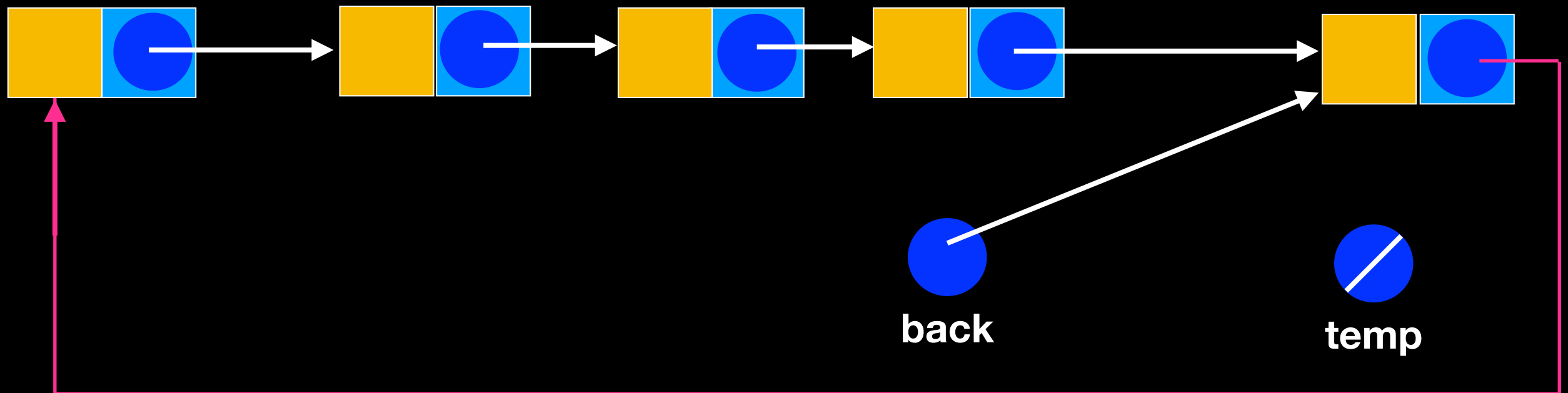


Singly Linked Chain

enqueue

**An Alternative:
A Circular Linked Chain**

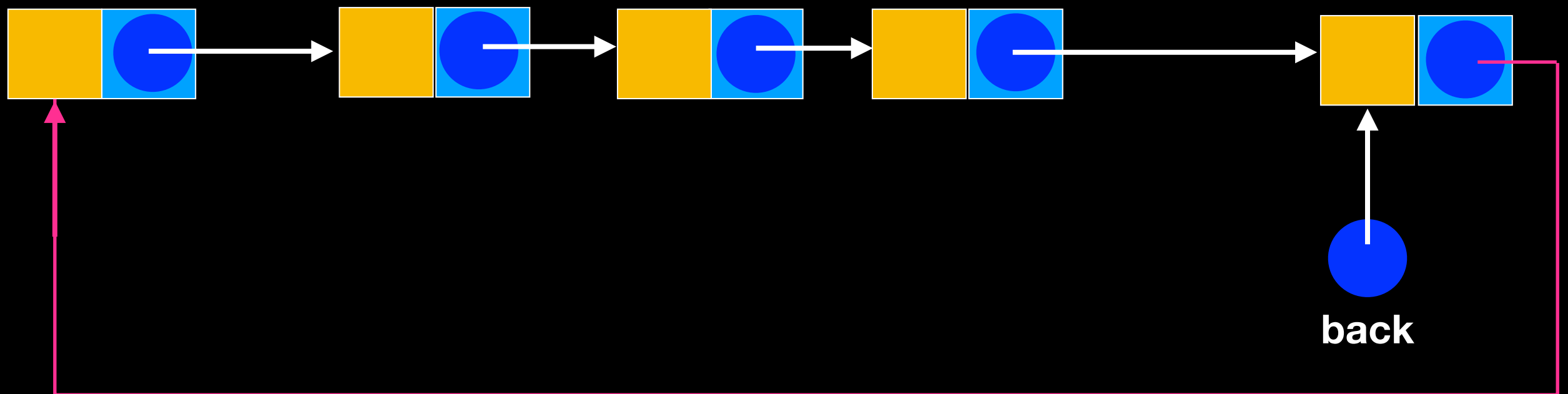
```
temp = nullptr;
```



Singly Linked Chain

enqueue

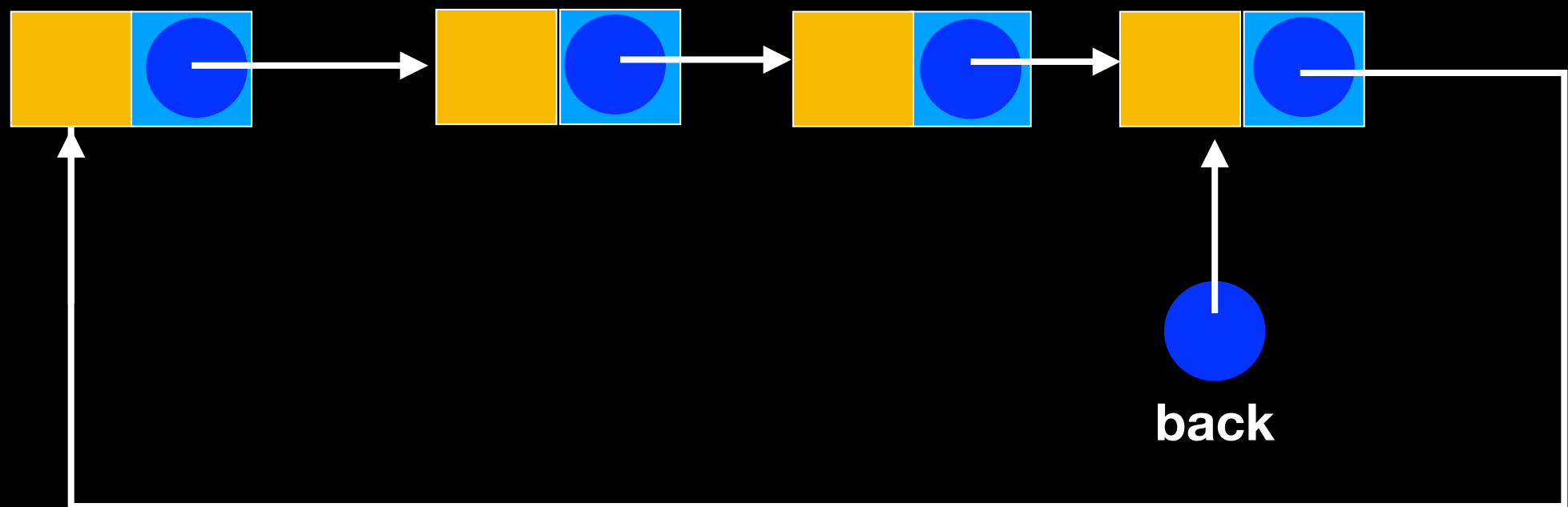
**An Alternative:
A Circular Linked Chain**



Singly Linked Chain

dequeue

**An Alternative:
A Circular Linked Chain**

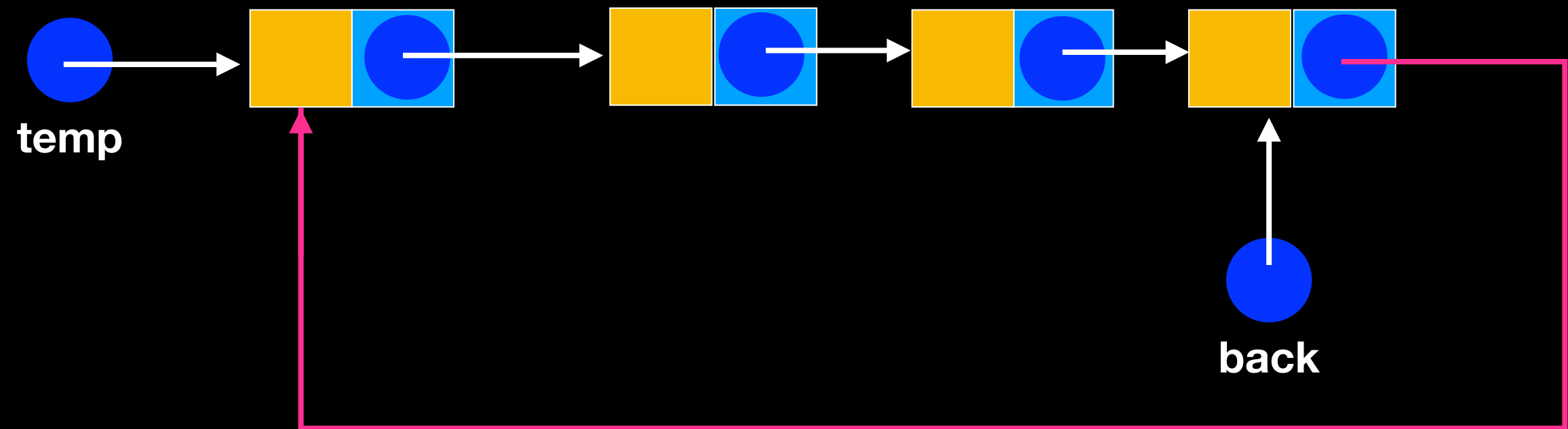


Singly Linked Chain

dequeue

An Alternative:
A Circular Linked Chain

```
temp = back->getNext()
```

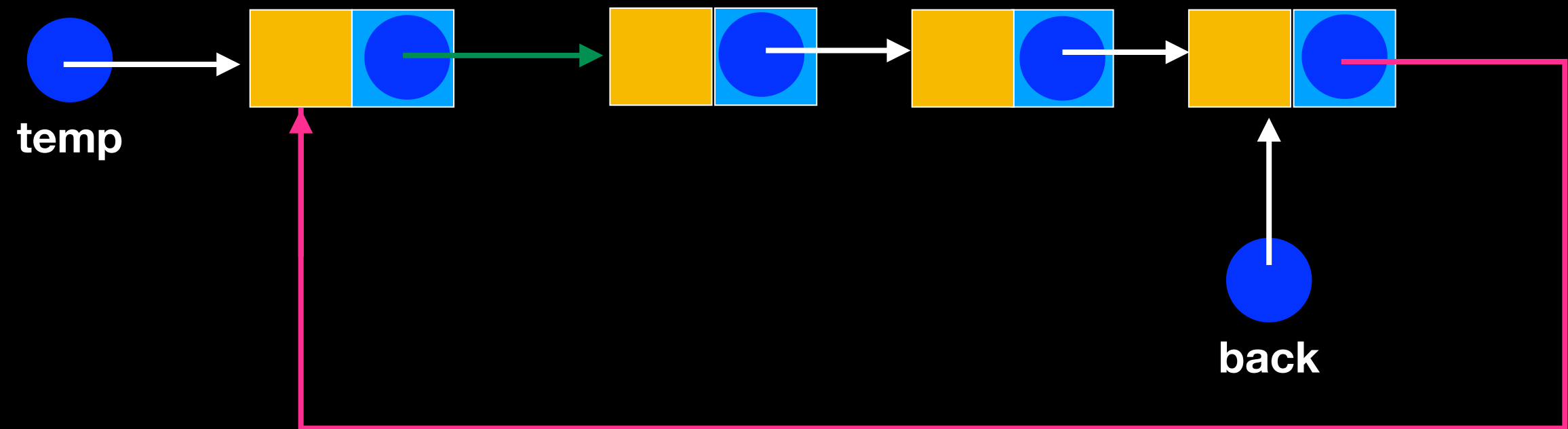


Singly Linked Chain

dequeue

An Alternative:
A Circular Linked Chain

```
back->setNext(back->getNext()->getNext())
```

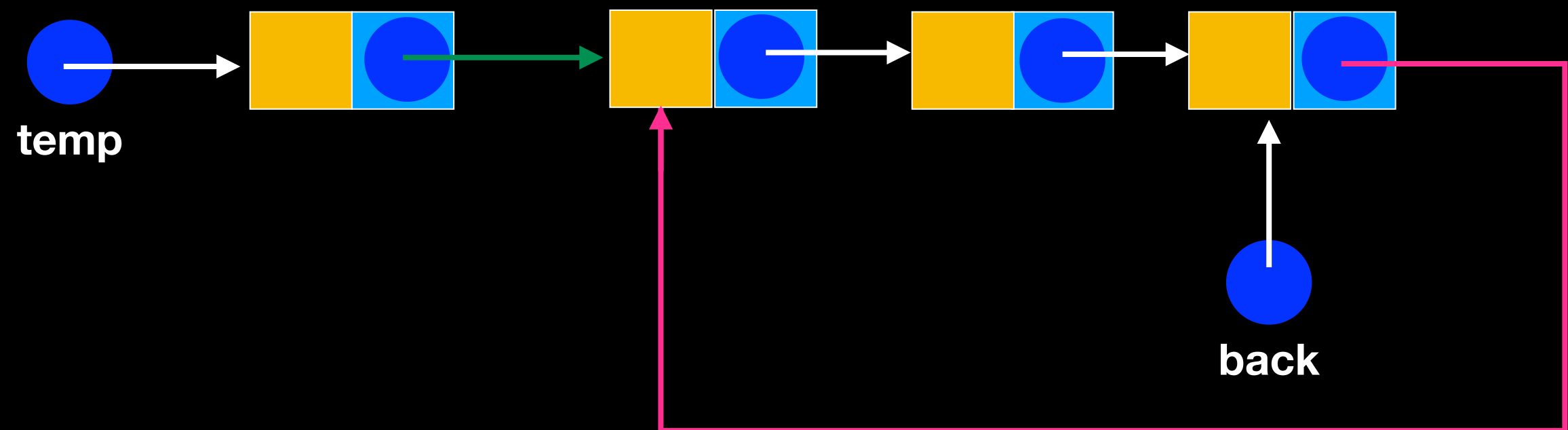


Singly Linked Chain

dequeue

An Alternative:
A Circular Linked Chain

```
back->setNext(back->getNext()->getNext())
```

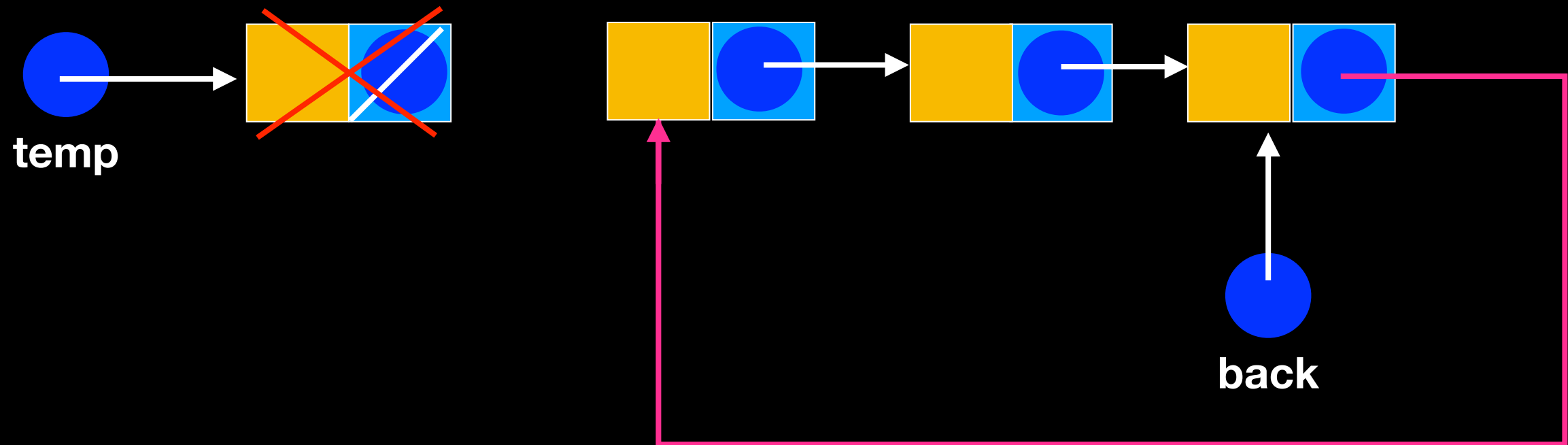


Singly Linked Chain

dequeue

**An Alternative:
A Circular Linked Chain**

```
temp->setNext(nullptr);  
delete temp;
```



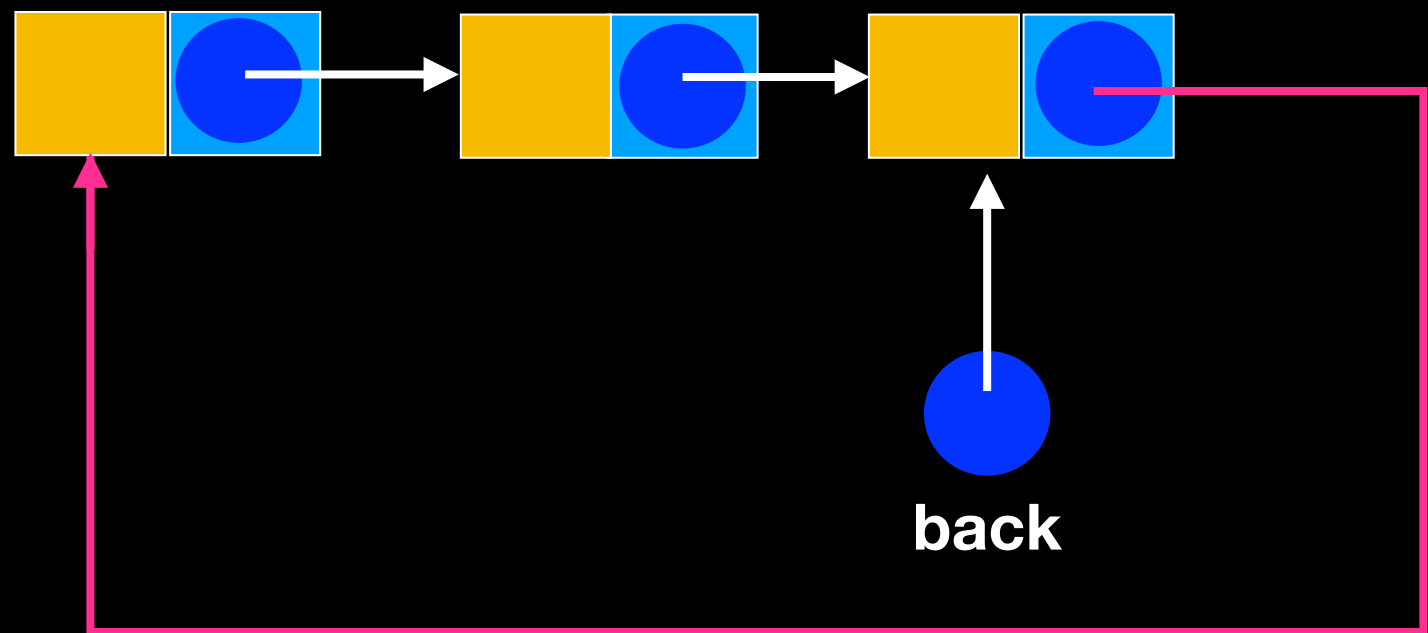
Singly Linked Chain

dequeue

**An Alternative:
A Circular Linked Chain**

back->getNext() is the front pointer!


temp



Queue ADT

(Circular Linked Chain)

```
#ifndef QUEUE_H_
#define QUEUE_H_

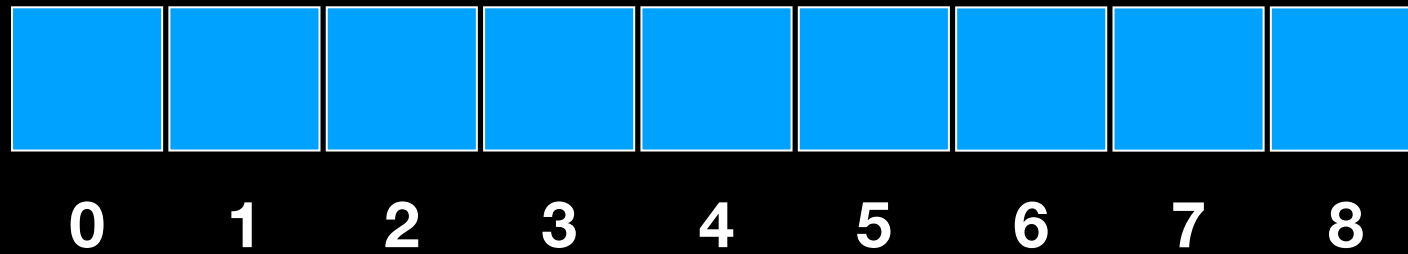
template<class T>
class Queue
{
public:
    Queue();
    Queue(const Queue<T>& a_queue); // Copy constructor
    ~Queue();
    void enqueue(const T& new_entry); // adds an element to back queue
    void dequeue(); // removes element from front of queue
    T front() const; // returns a copy of element at the front of queue
    int size() const; // returns the number of elements in the queue
    bool isEmpty() const; // returns true if no elements in queue, false otherwise

private:
    Node<T>* back_; // Pointer to back of queue
    int item_count; // number of items currently on the stack
}; //end Queue

#include "Queue.cpp"
#endif // QUEUE_H_
```

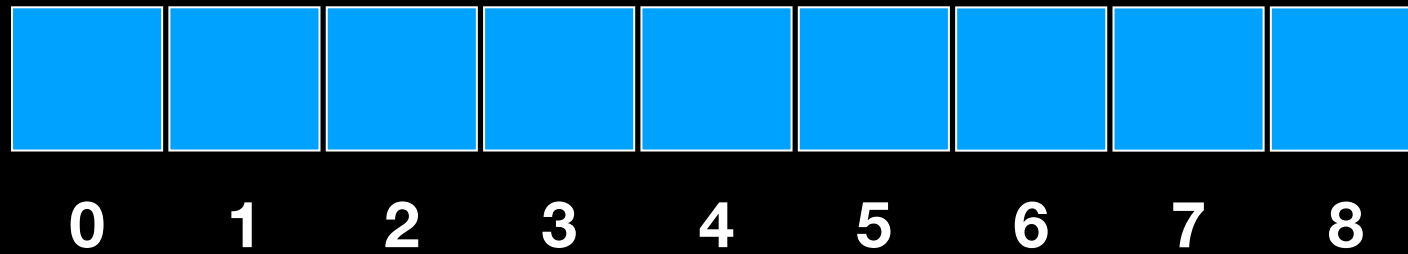

Array Considerations

front = ?
back = ?



Array Considerations

`front = 0`
`back = -1`



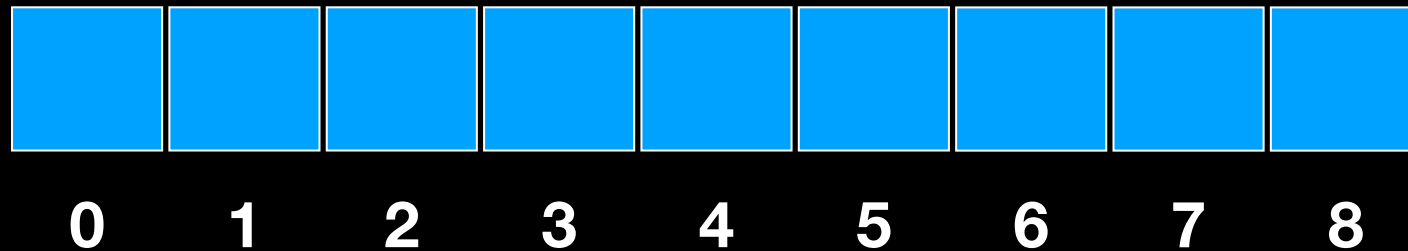
Array Considerations

enqueue

Increment back and add
element to `items_[back]`

`front = 0`

`back = -1`



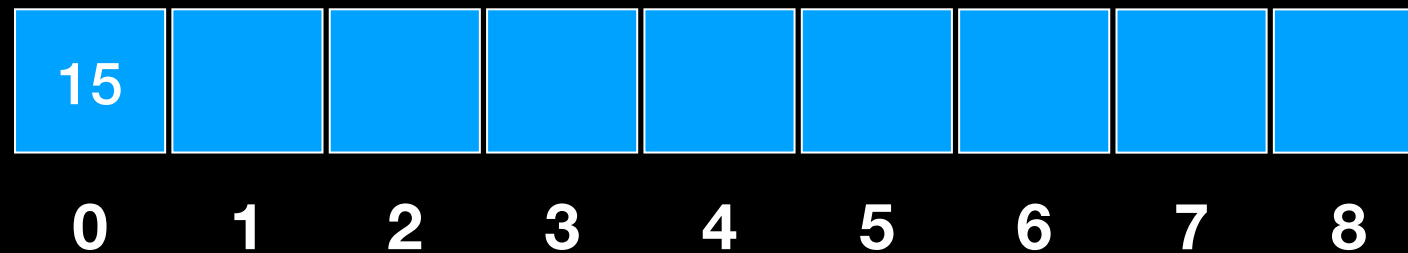
Array Considerations

enqueue

Increment back and add
element to `items_[back]`

`front = 0`

`back = 0`

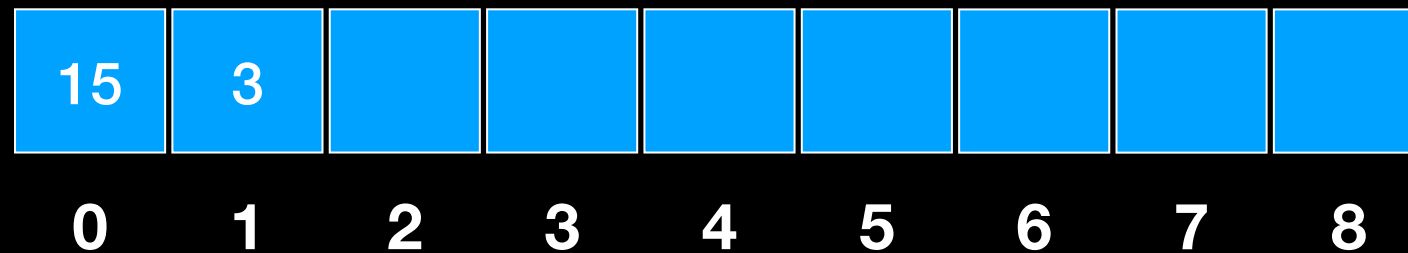


Array Considerations

enqueue

Increment back and add
element to `items_[back]`

`front = 0`
`back = 1`



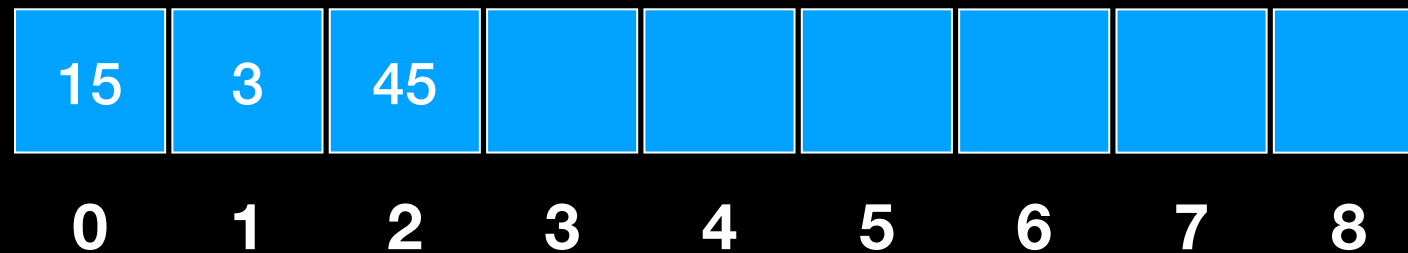
Array Considerations

enqueue

Increment back and add
element to `items_[back]`

`front = 0`

`back = 2`



Array Considerations

enqueue

Increment back and add
element to `items_[back]`

`front = 0`
`back = 5`

15	3	45	13	75	84			
0	1	2	3	4	5	6	7	8

This seems to work, but what happens when we start dequeuing?

Array Considerations

dequeue

Increment front

front = 1
back = 5

15	3	45	13	75	84			
0	1	2	3	4	5	6	7	8

We want $O(1)$ operations, so
simply increment front!

Array Considerations

dequeue

Increment front

front = 2
back = 5

15	3	45	13	75	84			
0	1	2	3	4	5	6	7	8

Array Considerations

`front = 6`
`back = 8`

15	3	45	13	75	84	55	38	97
0	1	2	3	4	5	6	7	8

RIGHTWARD DRIFT!!!

At some point queue will be full even if it contains only a few elements

Array Considerations

`front = 6`
`back = 8`

15	3	45	13	75	84	55	38	97
0	1	2	3	4	5	6	7	8

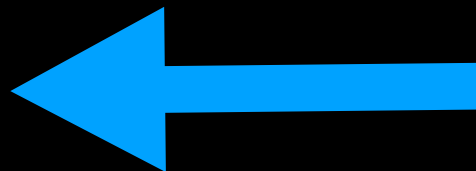
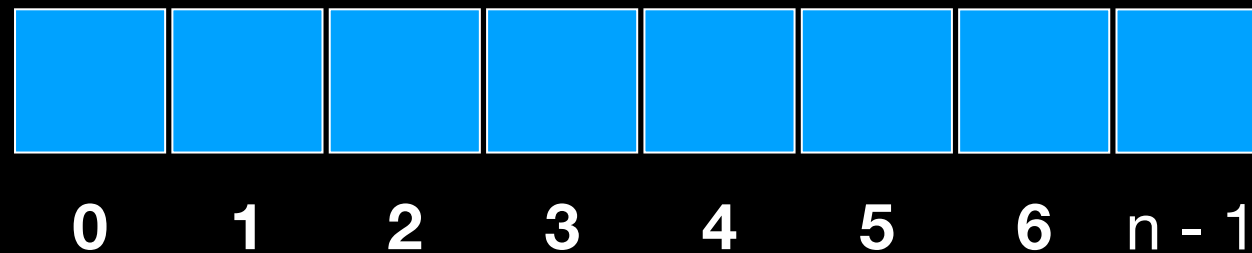
RIGHTWARD DRIFT!!!

At some point queue will be full even if it contains only a few elements

No
Good

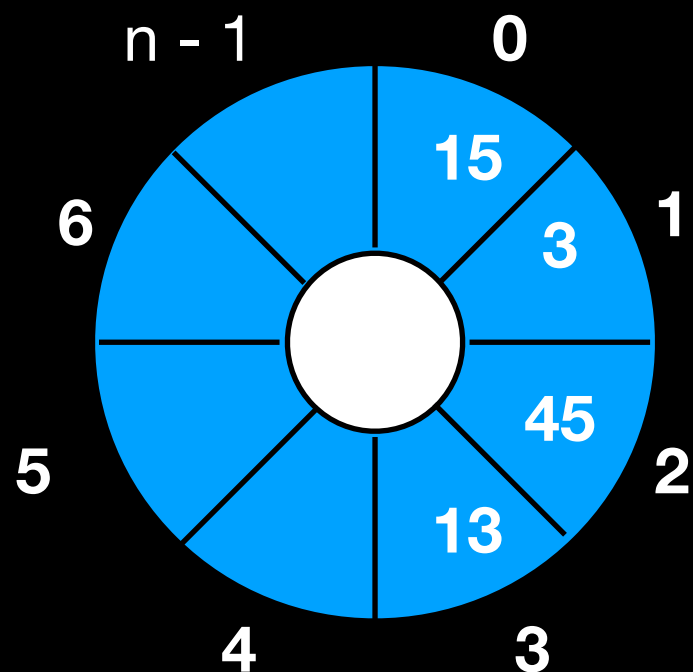
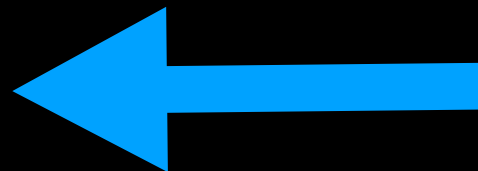
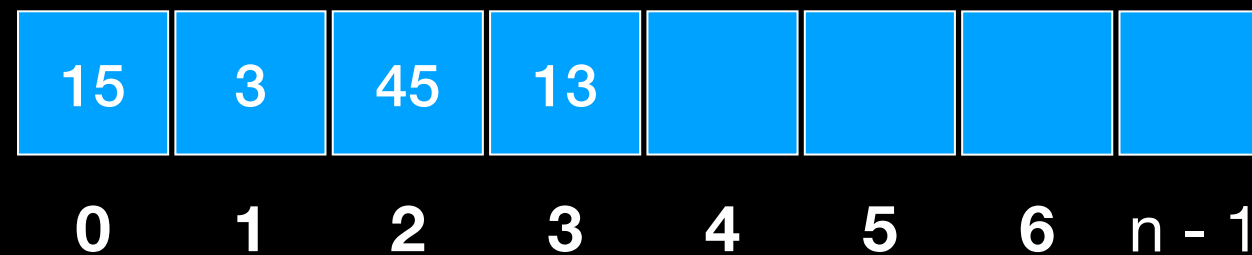
Circular Array Implementation

`front = 0`
`back = -1`



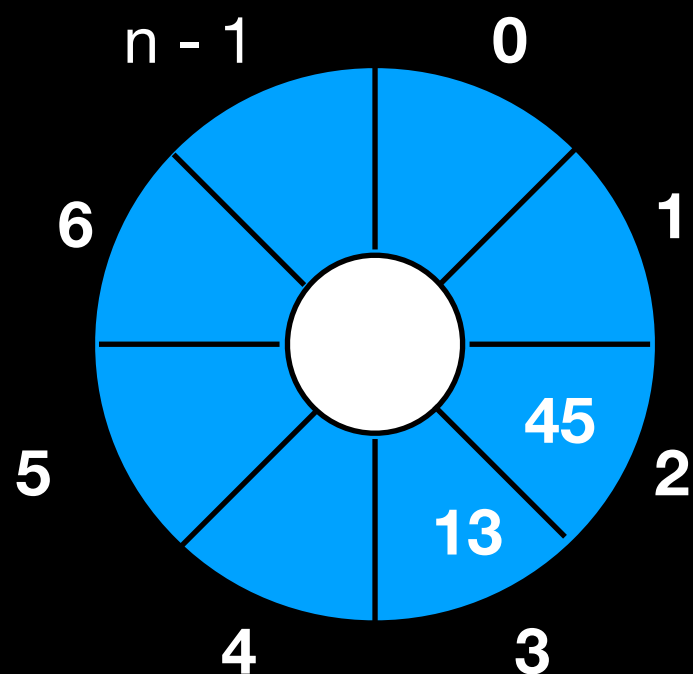
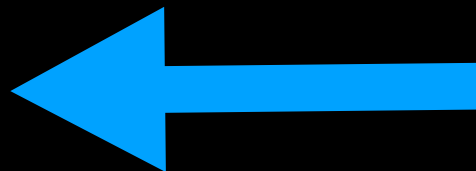
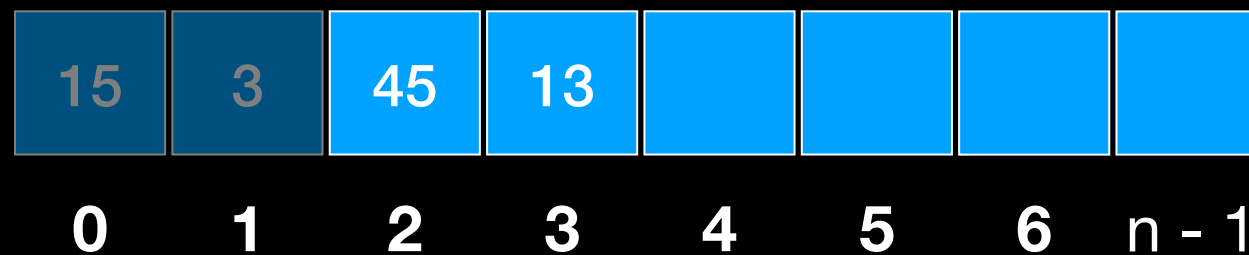
Circular Array Implementation

`front = 0`
`back = 3`



Circular Array Implementation

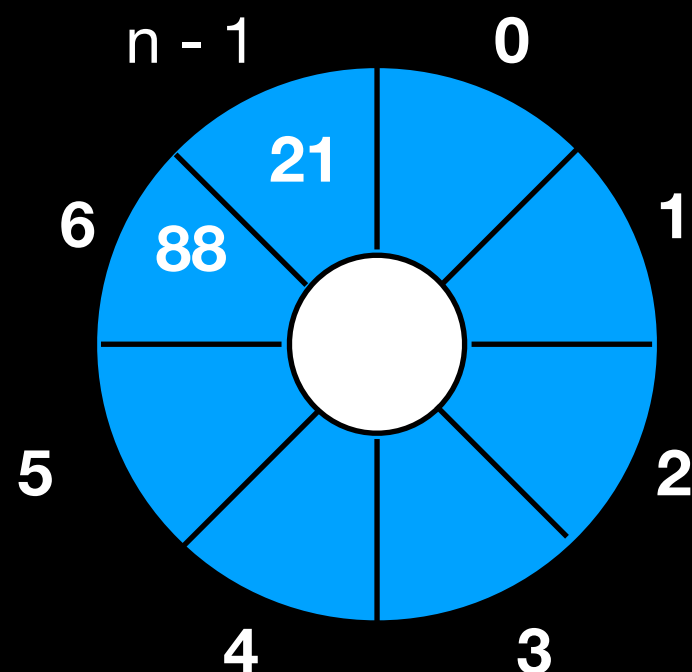
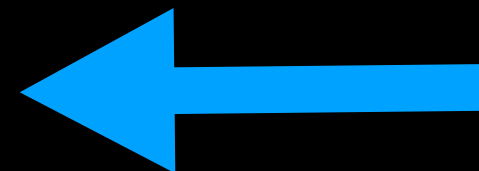
`front = 2`
`back = 3`



Circular Array Implementation

front = 6

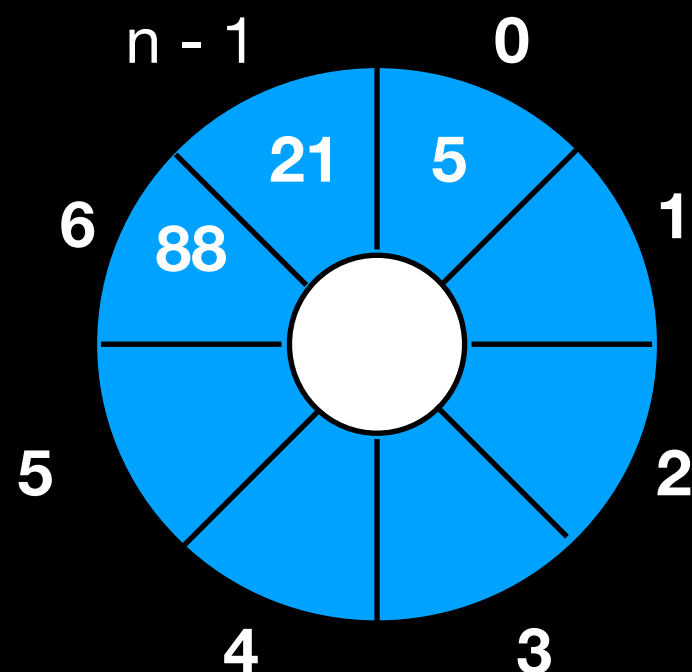
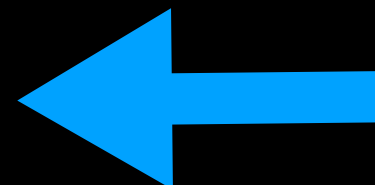
back = $n - 1$



Circular Array Implementation

front = 6

back = 0



WRAP AROUND USING
MODULO ARITHMETIC

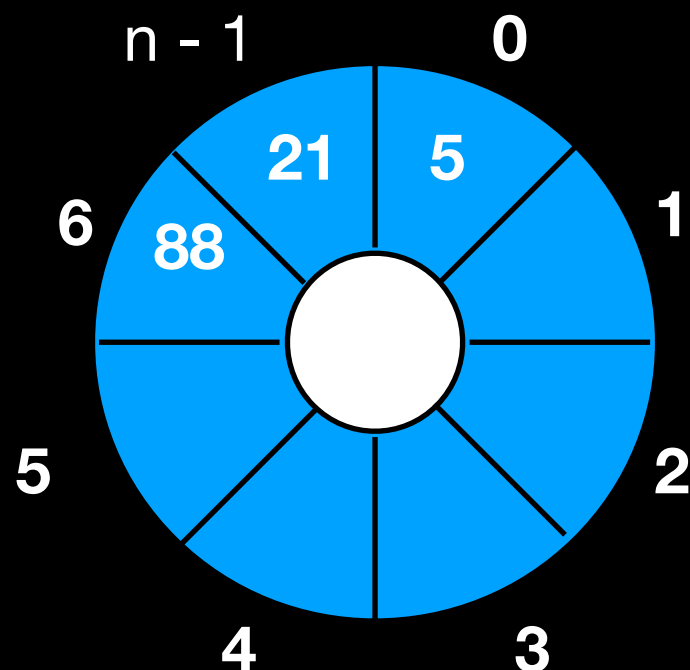
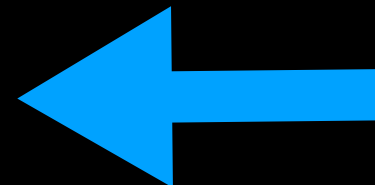
Circular Array Implementation

enqueue

```
back = (back + 1) % n  
add element to items_[back]
```

front = 6

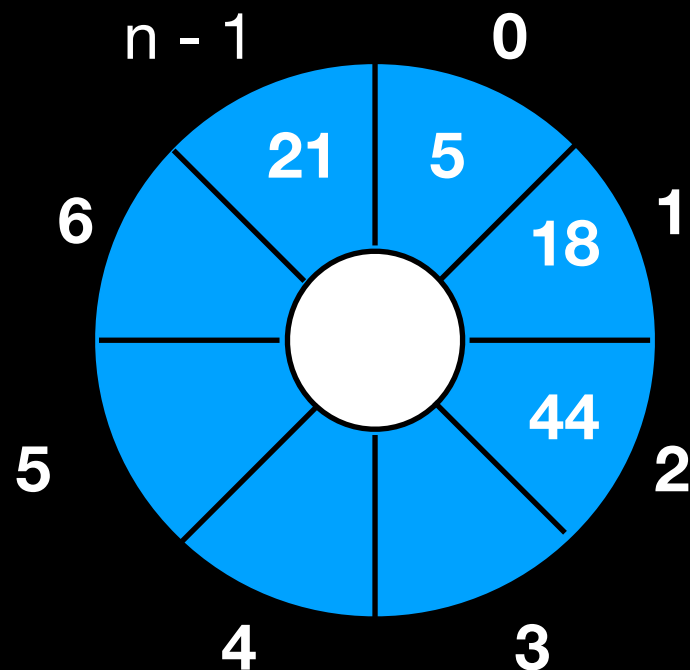
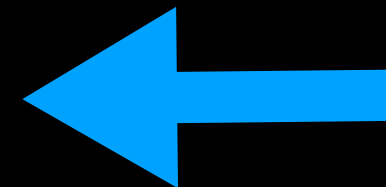
back = 0



Circular Array Implementation

front = **n-1**

back = **2**



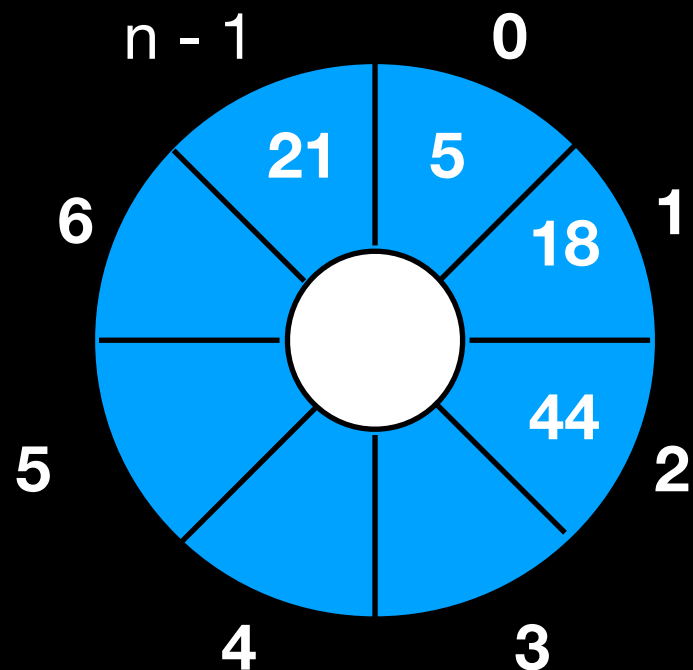
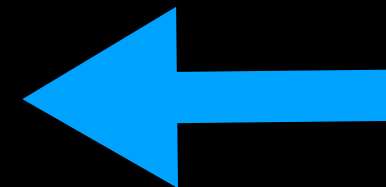
Circular Array Implementation

dequeue

`front = (front + 1) % n`

`front = n-1`

`back = 2`



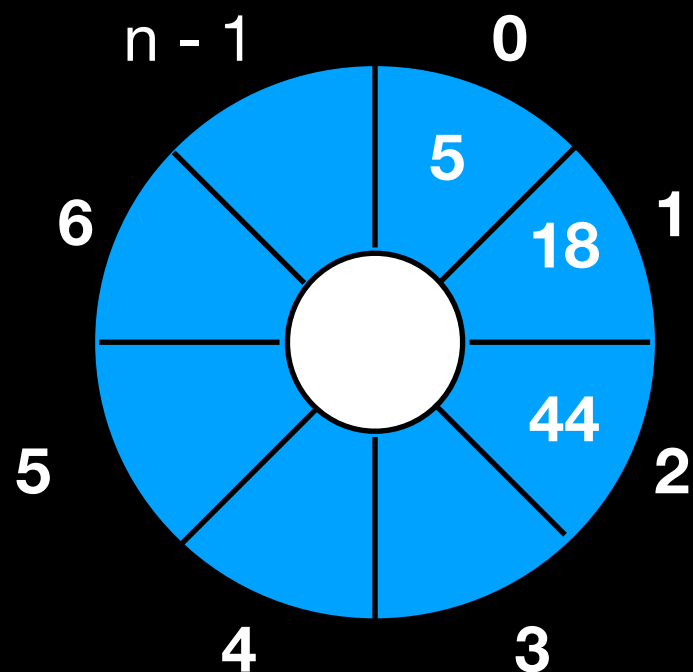
Circular Array Implementation

dequeue

`front = (front + 1) % n`

`front = 0`

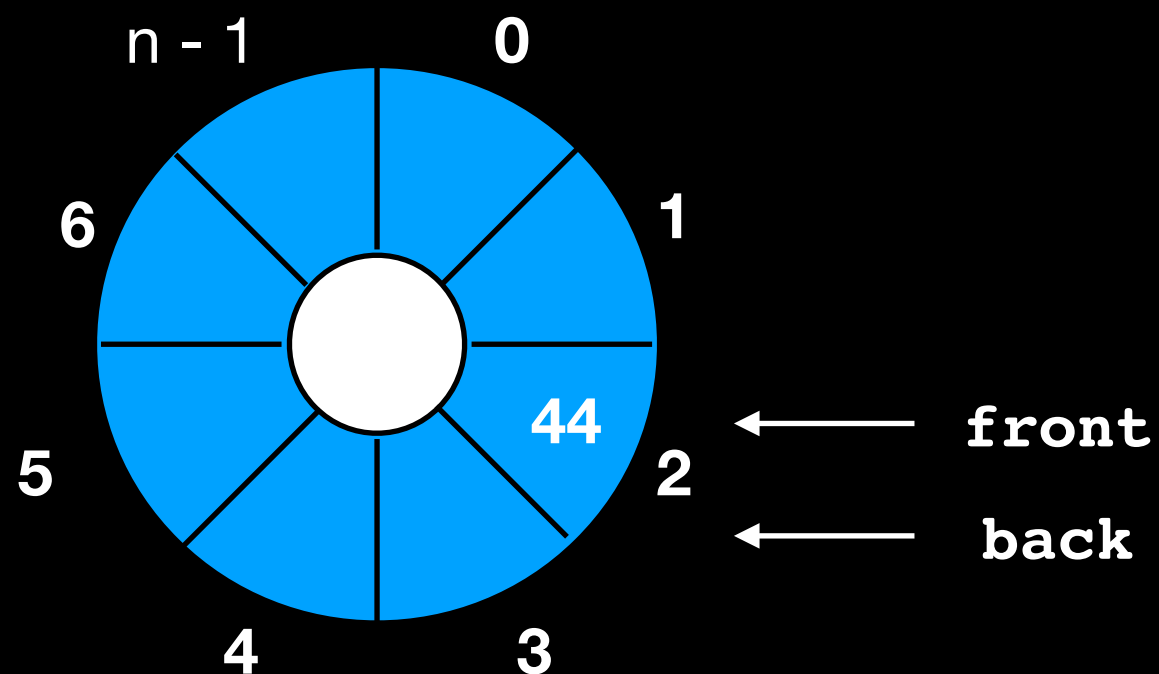
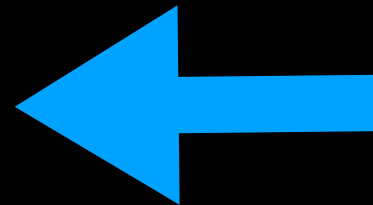
`back = 2`



Circular Array Implementation

front = 2

back = 2



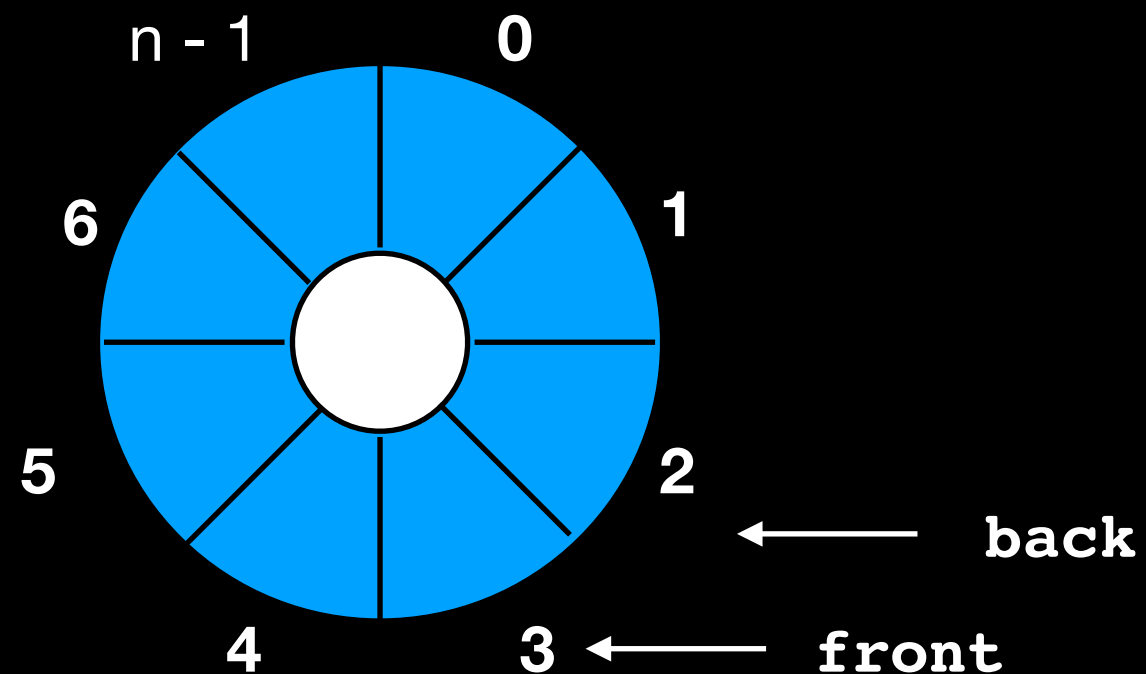
Circular Array Implementation

dequeue

`front = (front + 1) % n`

`front = 3`

`back = 2`



front passes back when
queue is EMPTY

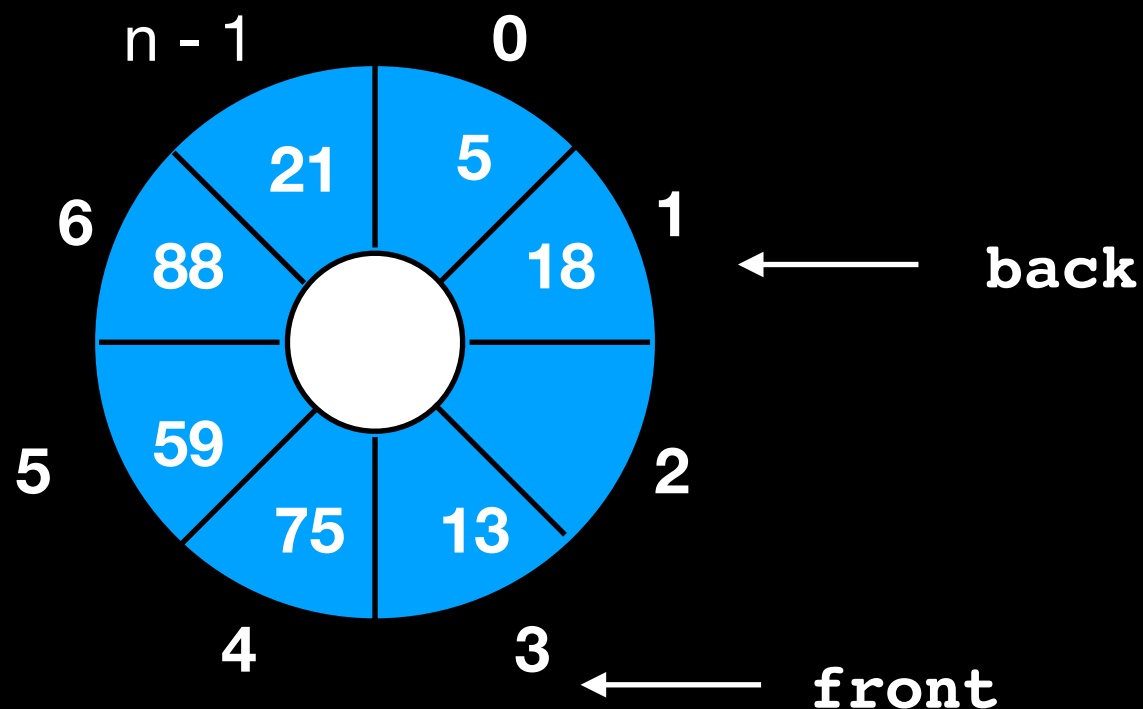
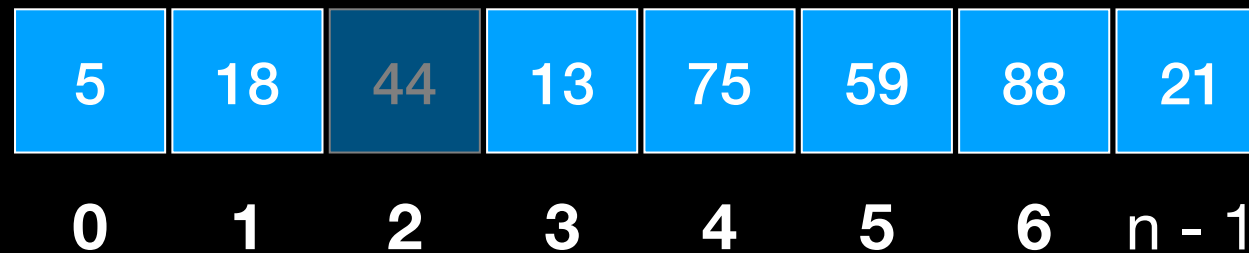
Circular Array Implementation

enqueue

$\text{back} = (\text{back} + 1) \% n$
add element to `items_[back]`

`front = 3`

`back = 1`



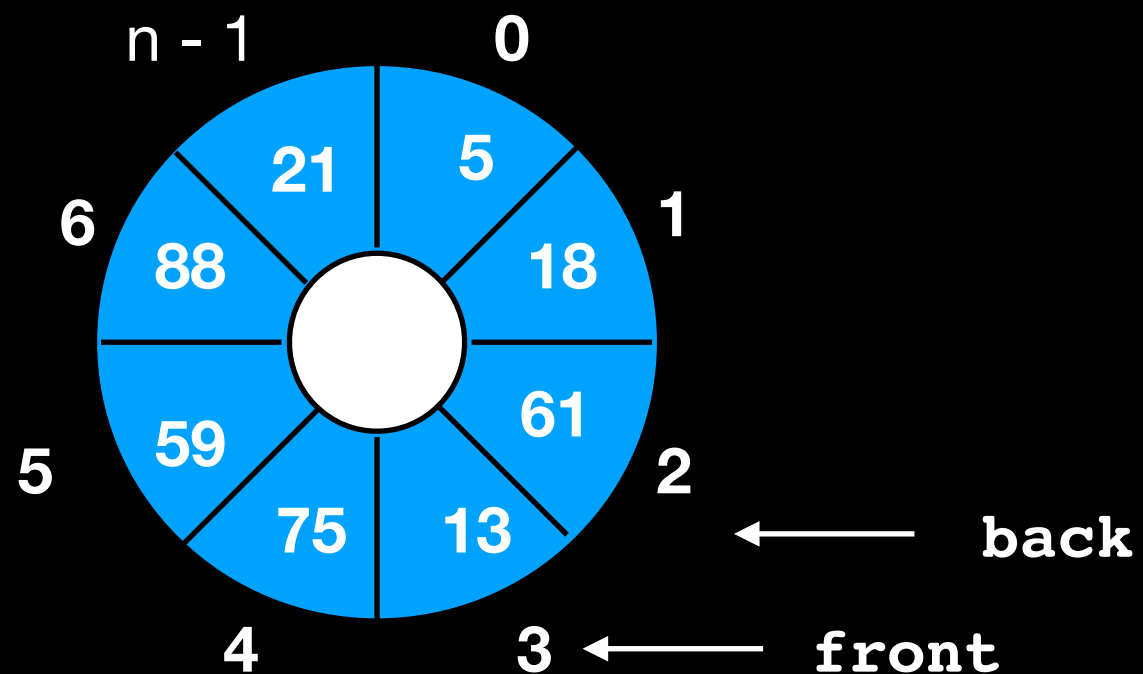
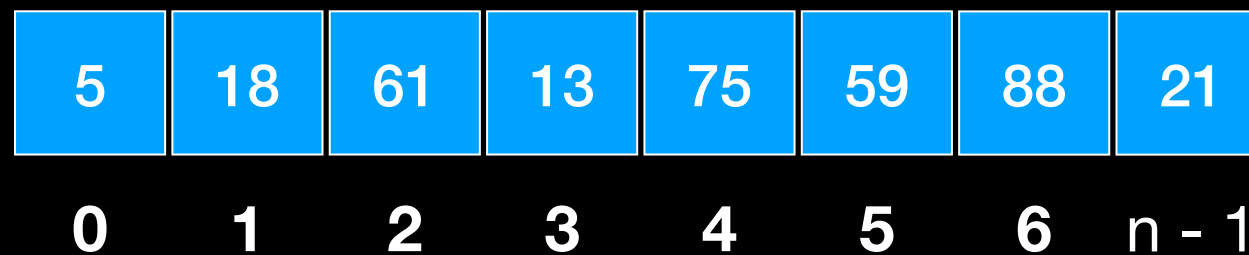
Circular Array Implementation

enqueue

$\text{back} = (\text{back} + 1) \% n$
add element to `items_[back]`

`front = 3`

`back = 2`



front passes back **ALSO**
when queue is **FULL**

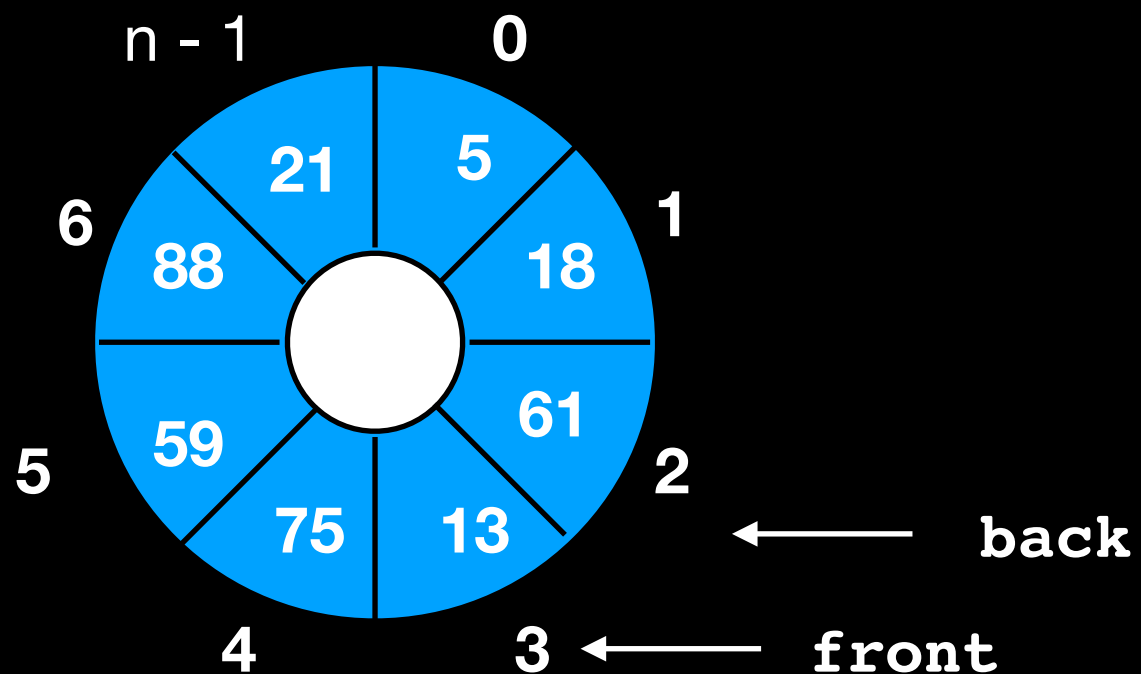
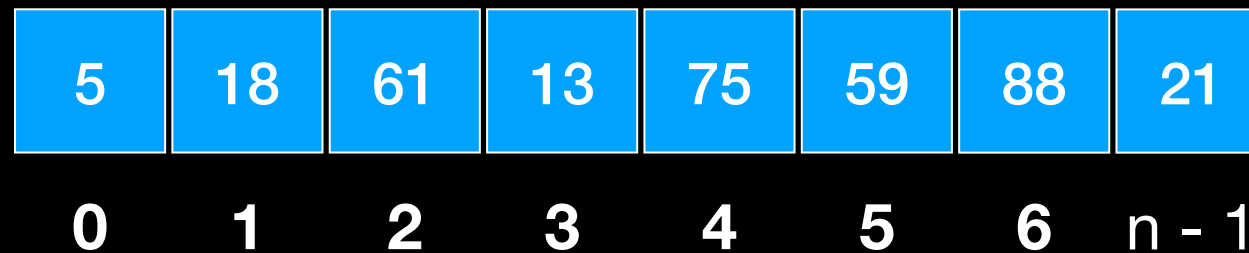
Circular Array Implementation

enqueue

$\text{back} = (\text{back} + 1) \% n$
add element to `items_[back]`

`front = 3`

`back = 2`



To distinguish between **empty** and **full** queue must keep a **COUNTER** for number of items

End Queue Review

Try solving the following Queue problems for practice:

- [Time Needed to Buy Tickets](#)
- [Reveal Cards in Increasing Order](#)
- [Level Order Traversal of BST](#)
- [Number of Recent Calls](#)
- [Rotting Oranges](#)
- [More Queue Problems](#)

Best way to get better at solving problems is through practice!

Common Operations

C++ STL Algorithms

- STL algorithms offer a wide ranges of functions to perform common operations on containers
 - ◆ Sorting
 - E.g., `std::stable_sort()`
 - ◆ Searching
 - E.g., `std::binary_search()`
 - ◆ Transforming
 - E.g., `std::copy()`
 - ◆ Counting and Comparing
 - E.g., `std::count()`
 - ◆ Numeric functions
 - E.g., `std::inner_product()`
 - ◆ And many more ...
- Most efficient versions of the algorithms are provided
- [Find out more in the documentation](#)

After Discussing Trees

- Sequence Containers:
 - ◆ Containers that store individual elements in linear data structures allowing the elements to be accessed sequentially
- Container Adapters
 - ◆ Provides an interface for other containers by adapting the behavior of the existing containers to fit specific use case
 - ◆ For example: stacks using vectors
- Associative containers
 - ◆ Containers that store keys but also store records associated with those keys
 - ◆ STL provides both ordered and unordered associative containers with different implementations
 - ◆ We will return to this later!
- Note that C++ STL now has even more containers. Find out [here](#).

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access							
Copy							
Destructor							
Insert (front)							
Insert (back)							
Insert (middle)							
Remove (front)							
Remove (back)							
Remove (middle)							
Clear							

* Amortized Runtime

N/A Not applicable

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access						N/A	N/A
Copy							
Destructor							
Insert (front)	N/A					N/A	N/A
Insert (back)	N/A		N/A			N/A	N/A
Insert (middle)	N/A					N/A	N/A
Remove (front)	N/A					N/A	N/A
Remove (back)	N/A		N/A			N/A	N/A
Remove (middle)	N/A					N/A	N/A
Clear							

* Amortized Runtime

N/A Not applicable

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access						N/A	N/A
Copy							
Destructor							
Insert (front)	N/A					N/A	N/A
Insert (back)	N/A					N/A	N/A
Insert (middle)	N/A					N/A	N/A
Remove (front)	N/A					N/A	N/A
Remove (back)	N/A					N/A	N/A
Remove (middle)	N/A					N/A	N/A
Clear							

→ Array, Stack and Queue don't have insert or remove operations

→ Array has a fixed size and only supports random access modification

→ Stack and Queue has no random access by design. $O(1)$ access to:

- ◆ `top()` of Stack
- ◆ `front()` and `back()` of Queue

→ Stacks have $O(1)$ push and pop

→ Queues have $O(1)$ push and pop

→ Adding and removing can be $O(1)^*$ depending on the container

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access							
Copy							
Destructor							
Insert (front)							
Insert (back)			N/A				
Insert (middle)							
Remove (front)							
Remove (back)			N/A				
Remove (middle)							
Clear							

→ Forward List only has a front pointer
 → Thus, no explicitly defined operations to insert or remove from the back.

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access	O(1)	O(1)	N/A	N/A	O(1)	N/A	N/A
Copy							
Destructor							
Insert (front)	N/A	O(n)	O(1)	O(1)	O(1)*	N/A	N/A
Insert (back)	N/A	O(1)*	N/A	O(1)	O(1)*	N/A	N/A
Insert (middle)	N/A	O(n)	O(n)	O(n)	O(n)	N/A	N/A
Remove (front)	N/A	O(n)	O(1)	O(1)	O(1)*	N/A	N/A
Remove (back)	N/A	O(1)	N/A	O(1)	O(1)*	N/A	N/A
Remove (middle)	N/A	O(n)	O(n)	O(n)	O(n)	N/A	N/A
Clear							

* Amortized Runtime

N/A Not applicable

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access			N/A	N/A			
Copy	Forward List and List require traversal to find the position for middle insertions/removals. Random access is not allowed (no [] operator).						
Destructor							
Insert (front)			O(1)	O(1)			
Insert (back)				O(1)			
Insert (middle)			O(n)	O(n)			
Remove (front)			O(1)	O(1)			
Remove (back)				O(1)			
Remove (middle)			O(n)	O(n)			
Clear							

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access							
Copy							
Destructor							
Insert (front)		O(n)					
Insert (back)		O(1)*					
Insert (middle)		O(n)					
Remove (front)		O(n)					
Remove (back)		O(1)					
Remove (middle)		O(n)					
Clear							

→ Vector shifts items for front and middle operations requiring O(n) time
 → Exceeding capacity during insertions triggers a full memory reallocation and copy operation. However, because this happens infrequently, it results in an O(1) amortized time-complexity for back insertions.

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access							
Copy							
Destructor							
Insert (front)					$O(1)^*$		
Insert (back)					$O(1)^*$		
Insert (middle)					$O(n)$		
Remove (front)					$O(1)^*$		
Remove (back)					$O(1)^*$		
Remove (middle)					$O(n)$		
Clear							

→ Deque shifts items for middle operations, requiring $O(n)$ time

→ Inserting or removing at the ends dynamically allocates or deallocates individual fixed-size memory chunks

→ Because chunk management happens infrequently, front and back operations result in an amortized $O(1)$ time complexity

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access							
Copy	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Destructor	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Insert (front)							
Insert (back)							
Insert (middle)							
Remove (front)							
Remove (back)							
Remove (middle)							
Clear	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

→ Clear is not explicitly defined for Array
 → Copying all the linear containers require $O(n)$
 → For destructor and clear operations, objects need to be destroyed by calling the destructor for each element resulting in $O(n)$. However, it can be done in $O(1)$ for trivially destructible data types (int, char, etc.).

Big-O of C++ STL Sequence Containers

Operation	Array	Vector	Forward List	List	Deque	Stack	Queue
Random Access	$O(1)$	$O(1)$	N/A	N/A	$O(1)$	N/A	N/A
Copy	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Destructor	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Insert (front)	N/A	$O(n)$	$O(1)$	$O(1)$	$O(1)^*$	N/A	N/A
Insert (back)	N/A	$O(1)^*$	N/A	$O(1)$	$O(1)^*$	N/A	N/A
Insert (middle)	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$	N/A	N/A
Remove (front)	N/A	$O(n)$	$O(1)$	$O(1)$	$O(1)^*$	N/A	N/A
Remove (back)	N/A	$O(1)$	N/A	$O(1)$	$O(1)^*$	N/A	N/A
Remove (middle)	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$	N/A	N/A
Clear	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

* Amortized Runtime

N/A Not applicable

What's Next?

- Wrap up Big-Five
- C++ Coding Expectations
- Smart Pointers
- Iterators
- Trees Review
- Binary Search Trees
- Heaps and Heap Sort